



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA INFORMÁTICA EN
INGENIERÍA DEL SOFTWARE

TRABAJO FIN DE GRADO

**Desarrollo de un DSL para definir sistemas
multiagentes**

Pedro Moreno Calderón
Junio, 2026



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA INFORMÁTICA EN
INGENIERÍA DEL SOFTWARE

TRABAJO FIN DE GRADO

**Desarrollo de un DSL para definir sistemas
multiagentes**

Autor: Pedro Moreno Calderón

Tutor: Pedro José Clemente Martín

Co-Tutor: José Barriga Corchero

Resumen

El auge de los modelos de lenguaje de gran tamaño ha impulsado la adopción de sistemas multiagente como paradigma para resolver tareas complejas mediante la colaboración de agentes especializados. Sin embargo, la construcción de estos sistemas carece hoy de metodologías estandarizadas: los desarrolladores trabajan directamente sobre los *frameworks* de programación, lo que produce bases de código difíciles de mantener y de comunicar entre equipos.

Este trabajo propone un lenguaje específico de dominio (*DSL*) para la definición de sistemas multiagente basados en LLM. El lenguaje permite modelar los componentes esenciales de un sistema —agentes, perfiles, herramientas, servidores MCP, estado compartido y estructuras de comunicación— con un vocabulario cercano al dominio y con independencia del *framework* subyacente. Un generador de código traduce cada modelo a una implementación Python funcional y lista para desplegar sobre LangGraph, incluyendo la infraestructura de contenerización y una API REST. Complementa el sistema una extensión para Visual Studio Code que ofrece resaltado de sintaxis, autocompletado contextual, mensajes de error semánticos y un diagrama interactivo del grafo de agentes generado a partir del modelo.

El DSL se implementó con Langium, incorporando restricciones semánticas en el validador para capturar invariantes no expresables en la gramática.

Palabras clave: Lenguaje específico de dominio, Sistemas multiagente, Generación de código, LangGraph, Langium, Large Language Model.

Abstract

The rise of large language models has driven the adoption of multi-agent systems as a paradigm for tackling complex tasks through the collaboration of specialised agents. However, the construction of these systems currently lacks standardised methodologies: developers work directly against programming frameworks, producing codebases that are difficult to maintain and to communicate across teams.

This work proposes a domain-specific language (DSL) for defining LLM-based multi-agent systems. The language allows modelling the essential components of a system—agents, profiles, tools, MCP servers, shared state, and communication structures—using domain-oriented vocabulary and independently of the underlying framework. A code generator translates each model into a functional Python implementation ready for deployment on LangGraph, including containerisation infrastructure and a REST API. The system is complemented by a Visual Studio Code extension that provides syntax highlighting, contextual autocompletion, semantic error messages, and an interactive diagram of the agent graph derived from the model.

The DSL was implemented with Langium, incorporating semantic restrictions in the validator to capture invariants not expressible in the grammar.

Keywords: Domain-specific language, Multi-agent systems, Code generation, LangGraph, Langium, Large Language Model.

Índice general

1. Introducción	1
1.1. Estructura del documento	1
1.2. Antecedentes y contexto actual	2
1.2.1. Desarrollo MAS previo a los LLM	4
1.2.2. Ausencia de estandarización en el desarrollo actual de agentes . . .	5
1.3. Objetivos	6
1.4. Planificación	7
1.4.1. Distribución del tiempo	7
1.4.2. Metodología y justificación	9
1.4.3. Sprints a detalle	11
1.4.4. Desviaciones y gestión de imprevistos	13
2. Estado del Arte	15
2.1. Auge de los Large Language Model (LLM)	16
2.1.1. Arquitectura y entrenamiento	16
2.1.2. Escala y capacidades emergentes	17
2.1.3. Técnicas de razonamiento y limitaciones	17
2.2. Agentes basados en LLM	18
2.2.1. Arquitectura del agente LLM	19
2.2.2. El protocolo MCP	20
2.2.3. El patrón ReAct	21
2.2.4. Limitaciones y transición a sistemas multiagente	21

2.3.	Sistemas Multiagente	22
2.3.1.	Estructuras de comunicación	22
2.3.2.	Aplicaciones representativas	23
2.4.	Frameworks de desarrollo de Multi-Agent System (MAS)	23
2.4.1.	LangChain	23
2.4.2.	LangGraph	24
2.4.3.	CrewAI	24
2.4.4.	AutoGen	25
2.4.5.	Comparativa	25
2.5.	Desarrollo dirigido por modelos y lenguajes específicos de dominio	26
2.5.1.	Lenguajes específicos de dominio	26
2.6.	Herramientas de definición de lenguajes	27
2.6.1.	Eclipse EMF, Xtext y Eugenia	27
2.6.2.	Langium	28
2.6.3.	JetBrains MPS	28
2.6.4.	Comparativa	29
2.7.	Trabajos relacionados	29
3.	Propuesta de desarrollo	31
3.1.	Motivación y justificación del enfoque	31
3.2.	Descripción del sistema y tecnologías empleadas	32
3.2.1.	Componentes del sistema	33
3.2.2.	Otras herramientas	36
3.3.	Alcance y limitaciones	37
4.	Requisitos	39
4.1.	Requisitos funcionales	40
4.1.1.	Lenguaje (<code>packages/language</code>)	40
4.1.2.	Extensión (<code>packages/extension</code>)	43
4.1.3.	Generador (<code>packages/cli</code>)	44

4.2. Requisitos no funcionales	46
5. Diseño	47
5.1. Diseño del DSL	48
5.1.1. Iteraciones del metamodelo	48
5.1.2. Metamodelo final	56
5.1.3. Sintaxis concreta	59
5.1.4. Restricciones semánticas	64
5.2. Diseño del generador	67
5.2.1. Artefactos generados	68
5.2.2. Mapeo del modelo a LangGraph	70
5.3. Diseño de la extensión	73
5.3.1. Funcionalidades base de Langium	73
5.3.2. Funcionalidades implementadas	74
6. Implementación y desarrollo	77
6.1. Implementación del DSL	78
6.1.1. Gramática	79
6.1.2. Validación semántica	92
6.2. Implementación del generador	95
6.2.1. Organización modular	95
6.2.2. Generación del estado	98
6.2.3. Generación de agentes y prompts	101
6.2.4. Generación de herramientas	106
6.2.5. Generación del grafo	108
6.2.6. Generación del soporte para producción	114
6.3. Implementación de la extensión	117
6.3.1. Resaltado y autocompletado	117
6.3.2. Ayudas al usuario	119
6.3.3. Autocompletado de herramientas MCP	120

6.3.4. Comando de generación	120
6.3.5. Diagrama del grafo	121
7. Pruebas	124
7.1. Pruebas del validador	124
7.2. Pruebas del generador	129
7.2.1. Selección de modelos: pairwise testing	129
7.2.2. Validación manual y corrección	130
7.3. Pruebas de la extensión	134
7.4. Validación con casos de uso	137
7.4.1. Respondedor automático de incidencias SRE	137
7.4.2. Asesor legal automatizado	140
8. Conclusiones y trabajos futuros	145
8.1. Conclusiones	145
8.2. Trabajos futuros	148
Anexos	153
A. Manual de instalación y manual de usuario	154
A.1. Manual de instalación	154
A.2. Manual de usuario	155
A.3. Versiones de las herramientas utilizadas	162
Bibliografía	163

Índice de tablas

1.1. Tabla de riesgos	9
1.2. Comparativa de metodologías de desarrollo	10
2.1. Comparativa de frameworks de desarrollo de MAS	25
2.2. Comparativa de <i>language workbenches</i>	29
3.1. Paquetes del proyecto	34
4.1. Requisitos funcionales del lenguaje: gramática	41
4.2. Requisitos funcionales del lenguaje: validador	42
4.3. Requisitos funcionales de la extensión	44
4.4. Requisitos funcionales del generador	45
4.5. Requisitos no funcionales	46
5.1. Resumen de las restricciones semánticas del Domain-Specific Language (DSL).	65
5.2. Mapeo de los elementos del DSL a constructos LangGraph.	71
5.3. Características base aportadas por Langium.	74
5.4. Funcionalidades implementadas en la extensión.	76
6.1. Restricciones semánticas implementadas en el validador.	94
6.2. Módulos del generador y ficheros de salida correspondientes.	97
6.3. Combinaciones de generación de nodos de agente.	105

7.1. Matriz de trazabilidad de las pruebas del validador. Cada fila se implementa con el helper <code>testRestriccion()</code> en <code>restrictions.test.ts</code>	128
7.2. Matriz de combinaciones pairwise para las pruebas del generador.	130
7.3. Matriz de trazabilidad de las pruebas del generador.	133
7.4. Matriz de trazabilidad de las pruebas de la extensión.	136
7.5. Agentes del sistema respondedor de incidencias y sus herramientas.	138
7.6. Relación entre el tamaño del modelo DSL y el código Python generado, excluyendo prompts de ambos lados.	140
7.7. Agentes del sistema asesor legal y sus herramientas.	142
7.8. Relación entre el modelo DSL y el código Python generado en el asesor legal, obtenida con <code>scripts/count_lines.py</code>	143
A.1. Opciones de persistencia del contexto.	156
A.2. Proveedores y modelos soportados por el DSL.	159
A.3. Snippets disponibles en la extensión.	160
A.4. Versiones de las herramientas requeridas.	162

Índice de figuras

1.1. Calendario de disponibilidad	8
1.2. Diagrama de Gantt del proyecto	11
2.1. Metamodelo de MAS basado en LLM propuesto en [1].	30
3.1. Flujo del sistema por fases	33
3.2. Arquitectura del sistema y relación entre paquetes	34
5.1. Metamodelo — Iteración 1	49
5.2. Metamodelo — Iteración 2	52
5.3. Grafo de control del sistema <i>Customer Service</i>	54
5.4. Metamodelo — Iteración 3	56
5.5. Modelo <code>.mad</code> completo del sistema <i>Research Assistant</i>	64
6.1. Estructura del paquete de lenguaje (<code>src/</code>).	78
6.2. Nodo raíz del metamodelo UML.	83
6.3. Nodo raíz traducido a gramática Langium.	84
6.4. Context del metamodelo UML.	84
6.5. Context traducido a gramática Langium.	85
6.6. Profile del metamodelo UML.	86
6.7. Profile traducido a gramática Langium.	86
6.8. Tool del metamodelo UML.	86
6.9. Tool traducido a gramática Langium.	87
6.10. Actor del metamodelo UML.	88

6.11. Actor traducido a gramática Langium.	89
6.12. Estructura de comunicación y transiciones del metamodelo UML.	90
6.13. Estructura de comunicación y transiciones traducida a gramática Langium.	91
6.14. Estructura del paquete del generador (<code>src/</code>).	96
6.15. Flujo de generación del estado.	99
6.16. Función <code>resolveMessageConfig()</code> en <code>stateGenerator.ts</code>	100
6.17. Ejemplo de <code>prompts.py</code> generado.	101
6.18. Flujo de generación de los agentes.	102
6.19. Ejemplo del patrón <i>Structured Output</i>	102
6.20. Ejemplo de modelos generados para distintos agentes.	103
6.21. Flujo de <code>generateNode()</code>	103
6.22. Plantilla del código de los nodos generados.	104
6.23. Campo <code>next</code> obligatorio en el esquema de nodos <code>Decentralized</code>	105
6.24. Flujo de <code>mcpClientGenerator.ts</code>	107
6.25. Implementación del patrón <i>fail-fast</i> en <code>_get_tool()</code>	107
6.26. Acceso directo sin <i>fail-fast</i>	107
6.27. Flujo de generación de la estructura <code>Layered</code>	109
6.28. Ejemplo de subgrafo <code>Layered</code> generado.	110
6.29. Flujo de generación de la estructura <code>Centralized</code>	110
6.30. Ejemplo de subgrafo <code>Centralized</code> generado.	111
6.31. Flujo de generación de la estructura <code>Decentralized</code>	111
6.32. Ejemplo de subgrafo <code>Decentralized</code> generado.	112
6.33. Flujo de generación del grafo principal.	112
6.34. Ejemplo de <code>graph.py</code> generado.	113
6.35. Flujo de <code>configGenerator.ts</code>	114
6.36. Ficheros de configuración generados por <code>configGenerator.ts</code>	115
6.37. Implementaciones de <code>generate_checkpointier()</code> generadas.	116
6.38. Ejemplo de <code>langgraph.json</code> generado.	116

6.39. Resaltado de sintaxis y autocompletado filtrado por proveedor en el editor VSCoDe.	118
6.40. Ayudas al usuario: hover provider y snippets predefinidos.	119
6.41. Botón correspondiente al manual de usuario.	120
6.42. Botón de generación en la barra superior del editor, activo sobre un fichero .mad.	121
6.43. Botón para abrir el panel de diagrama del grafo en la barra superior del editor.	122
6.44. Panel lateral de VSCoDe con el diagrama Mermaid generado a partir de un modelo de ejemplo.	123
7.1. Función auxiliar <code>testRestriccion()</code> en <code>restrictions.test.ts</code>	126
7.2. Resultado de la ejecución de las pruebas del validador con Vitest.	128
7.3. Resultado de la ejecución de las pruebas del generador con Vitest.	134
7.4. Traza de LangSmith durante la ejecución del respondedor de incidencias. El agente <code>Remediator</code> invoca <code>rollbackDeployment</code> como parte del plan de remediación.	139
7.5. Traza de LangSmith durante la ejecución del asesor legal. Un abogado especialista invoca su herramienta de búsqueda documental.	143
A.1. Botón de generación de código (cohete). Al pulsarlo se genera el proyecto Python en <code>generated/</code>	161
A.2. Botón de diagrama (gráfico de barras). Abre el diagrama Mermaid del grafo generado a partir del modelo actual en un panel lateral.	161
A.3. Botón de manual (libro). Abre el manual de usuario en un panel lateral. . .	161

Capítulo 1

Introducción

Contenidos

1.1. Estructura del documento	1
1.2. Antecedentes y contexto actual	2
1.2.1. Desarrollo MAS previo a los LLM	4
1.2.2. Ausencia de estandarización en el desarrollo actual de agentes	5
1.3. Objetivos	6
1.4. Planificación	7
1.4.1. Distribución del tiempo	7
1.4.2. Metodología y justificación	9
1.4.3. Sprints a detalle	11
1.4.4. Desviaciones y gestión de imprevistos	13

1.1. Estructura del documento

El presente documento se organiza en los siguientes capítulos:

- **Capítulo 1 — Introducción:** presenta el contexto del trabajo, los objetivos, la planificación seguida y las desviaciones producidas durante el desarrollo.



- **Capítulo 2 — Estado del arte:** revisa los trabajos relacionados con los MAS basados en LLM, los lenguajes específicos de dominio y las herramientas de generación de código existentes.
- **Capítulo 3 — Propuesta de desarrollo:** describe el metamodelo diseñado y justifica las decisiones tecnológicas adoptadas.
- **Capítulo 4 — Requisitos:** detalla los requisitos funcionales y no funcionales del sistema.
- **Capítulo 5 — Diseño:** aborda la gramática del DSL y la arquitectura del generador de código.
- **Capítulo 6 — Implementación:** describe los aspectos técnicos más relevantes del desarrollo.
- **Capítulo 7 — Pruebas:** recoge las pruebas realizadas y los resultados obtenidos.
- **Capítulo 8 — Conclusiones:** expone las conclusiones del trabajo y las posibles líneas de trabajo futuro.

1.2. Antecedentes y contexto actual

La Inteligencia Artificial (IA) está redefiniendo el panorama tecnológico global a un ritmo sin precedentes. Han transcurrido varios años desde hitos fundacionales como la publicación en 2017 del artículo “Attention Is All You Need” por parte de investigadores de Google Brain, o la aparición en 2018 de GPT-1, el primer modelo de lenguaje de gran tamaño (LLM) plenamente funcional.

Desde entonces, el crecimiento en este campo ha sido exponencial. Los LLM se han integrado progresivamente en la vida cotidiana de los usuarios, abarcando desde profesionales con un alto nivel técnico en informática hasta personas sin experiencia previa en el sector. Este avance ha abierto un amplio abanico de posibilidades en el desarrollo de software, que va desde la generación automatizada de contenido y la creación de



asistentes conversacionales, hasta la capacidad de dotar a las aplicaciones de capacidades de razonamiento complejo.

Es precisamente esta última capacidad la que introduce el concepto central de este trabajo: los agentes. Los agentes de IA son sistemas autónomos dotados de objetivos específicos y de la capacidad de percibir y comprender su entorno. Esta percepción, combinada con su capacidad de razonamiento, les permite diseñar y ejecutar planes de acción para alcanzar sus metas. No obstante, su autonomía puede verse limitada en la ejecución de tareas excesivamente prolongadas o complejas, donde es frecuente la aparición de alucinaciones o desviaciones en el razonamiento [2].

Como solución a esta limitación surgen los Sistemas Multiagente (MAS), arquitecturas que permiten la interacción y coordinación entre múltiples agentes. De este modo, en lugar de depender de un único agente monolítico, es posible desplegar un conjunto de agentes, cada uno especializado en una tarea o ámbito concreto, que colaboran para alcanzar un objetivo común.

A nivel teórico, este paradigma resulta sumamente prometedor. Casos de uso complejos, como la automatización en la gestión del tráfico aéreo en aeropuertos o el control de microrredes eléctricas en edificios, se vuelven viables bajo este enfoque. Sin embargo, su implementación práctica presenta desafíos significativos.

Por esta razón, desde finales de 2023 surgieron herramientas orientadas a la gestión de sistemas multiagente como AutoGen, LangGraph, OpenAI Swarm o PydanticAI.

Si bien, el ecosistema se encuentra aún en una fase temprana de madurez. La principal carencia actual no es la falta de herramientas, sino la ausencia de metodologías de desarrollo estandarizadas para los MAS. Como consecuencia, la construcción de estos sistemas tiende a derivar en bases de código desestructuradas y difíciles de mantener; un problema de complejidad y acoplamiento. Irónicamente, nos encontramos ante un reto inverso al de los años 2000 [3].



1.2.1. Desarrollo MAS previo a los LLM

A principios de la década de los 2000, surgió un enfoque de desarrollo de software que buscaba trascender las limitaciones del paradigma de la Programación Orientada a Objetos (POO): la Agent-Oriented Software Engineering (AOSE) [4]. Aunque se propusieron diversas metodologías para guiar la construcción de MAS, todas convergían en una limitación fundamental: la capacidad de razonamiento subyacente de la época no lograba satisfacer las necesidades prácticas de los sistemas complejos.

El escenario tecnológico de aquel entonces presentaba una clara disyuntiva. Por un lado, los sistemas de razonamiento lógico-simbólico contaban con una sólida base matemática que garantizaba resultados correctos y demostrables. Sin embargo, esta rigurosidad formal provocaba a menudo problemas de intratabilidad computacional; los tiempos de respuesta solían ser demasiado elevados, lo que en sistemas en tiempo real o críticos equivalía a no obtener ninguna respuesta válida.

Por otro lado, para mitigar este problema de rendimiento, se optó por sacrificar gran parte de la expresividad matemática de la lógica formal en favor de la eficiencia operativa. Aunque esta concesión mejoraba notablemente los tiempos de respuesta, reducía la fiabilidad del sistema y convertía a los agentes en entidades sumamente rígidas. En consecuencia, estos sistemas pasaron a depender de protocolos de comunicación altamente definidos y acoplados para poder interoperar con éxito.

En este contexto cobró especial relevancia la arquitectura Belief-Desire-Intention (BDI), un modelo inspirado en la teoría de la psicología cognitiva diseñado para conceptualizar los estados mentales de un agente de software [5]. Tomando como base conceptual arquitecturas de este tipo, florecieron metodologías formales de desarrollo para MAS como Gaia [6], Tropos [7] y MaSE [8].

A pesar de las barreras tecnológicas, surgieron implementaciones de notable éxito que demostraron el potencial del paradigma. Ejemplos destacados incluyen el proyecto OASIS, un sistema de gestión del tráfico aéreo en Singapur modelado bajo la arquitectura BDI [5], o los sistemas desarrollados por la Fuerza Aérea de los Estados Unidos, que emplearon la metodología MaSE para la coordinación de enjambres de vehículos autónomos [8].



No obstante, la adopción generalizada de estas metodologías en la industria nunca llegó a materializarse. Aunque la ingeniería de software proporcionaba marcos de trabajo estructurados, rigurosos y adaptables para el diseño, la capacidad real de razonamiento autónomo de las máquinas no acompañaba este nivel de madurez metodológica.

Como resultado, la industria del software fue abandonando progresivamente estas metodologías orientadas a agentes, las cuales quedaron relegadas casi exclusivamente al plano académico y de investigación.

1.2.2. Ausencia de estandarización en el desarrollo actual de agentes

A partir del auge de los LLM en 2022, la ingeniería de software ha sido testigo de una proliferación acelerada de los MAS. No obstante, esta innovación técnica no ha ido acompañada de un desarrollo equivalente en la forma de definirlos y representarlos formalmente [3].

A medida que un proyecto crece en extensión, también lo hace en complejidad estructural, volviéndose progresivamente más difícil de entender, mantener y evolucionar. Esta situación se ve agravada por la naturaleza cambiante del ecosistema: los frameworks de desarrollo de MAS se actualizan con frecuencia, lo que puede romper implementaciones existentes y obligar a correcciones en un código que, sin una estructura clara, resulta costoso de abordar.

A esto se suma la ausencia de un vocabulario común para describir estos sistemas. Un estudio sistemático reciente ha identificado hasta 16 patrones de diseño distintos y 12 atributos de calidad diferentes que los desarrolladores intentan optimizar de forma aislada [9]. Esta diversidad evidencia la falta de un consenso arquitectónico y de un marco formal de representación, algo esperable de un campo que aún no ha alcanzado su madurez.

Este no es un problema nuevo en la ingeniería de software. El diseño orientado a objetos atravesó una etapa similar de fragmentación antes de que la adopción de Unified Modeling Language (UML) proporcionase un lenguaje visual común para modelar



sistemas, independientemente del lenguaje de programación o el framework subyacente. El desarrollo de MAS basados en LLM se encuentra hoy en un punto análogo: existe capacidad técnica, pero no una forma compartida de representar lo que se construye.

1.3. Objetivos

Por ello, este trabajo propone un DSL que permita modelar un MAS de forma estructurada y con un vocabulario compartido. De este modo, se busca que cualquier desarrollador pueda definir y comprender un sistema de agentes de forma clara y sin ambigüedad. El DSL se acompaña de herramientas que asistan al desarrollador a lo largo de todo el proceso, desde la definición del sistema hasta la obtención de código funcional y listo para desplegar.

Nuestros principales objetivos son:

1. **Diseñar el metamodelo del DSL.** Definir, mediante un diagrama UML, los conceptos fundamentales que componen un MAS —agentes, tareas, herramientas, flujos de comunicación— y las relaciones entre ellos, con independencia de cualquier herramienta o framework concreto.
2. **Implementar la gramática y semántica del DSL en Langium.** Trasladar el metamodelo a una gramática expresiva y sin ambigüedades que permita describir instancias de un MAS de forma textual.
3. **Desarrollar un editor asistido.** Proporcionar una herramienta que permita al desarrollador crear y editar instancias del metamodelo de forma asistida, notificando al usuario tanto cuando su modelo no sea válido como cuando cierta combinación no sea posible pese a serlo sintácticamente; por ejemplo, si un proveedor concreto no puede utilizar una herramienta determinada o si el usuario está definiendo un grafo de agentes sin fin.
4. **Implementar un generador de código.** Implementar un generador de código para el framework LangGraph que traduzca el modelo definido en el DSL a una



implementación funcional.

5. **Garantizar la mantenibilidad del sistema generado.** El código generado debe seguir principios de ingeniería reconocidos —separación de responsabilidades, bajo acoplamiento y alta cohesión— de modo que pueda ser comprendido fácilmente o servir de base para modificaciones que no puedan ser soportadas a través del DSL.
6. **Preparar el código generado para el despliegue.** El código generado incluirá la configuración necesaria para contenerizarlo junto con una Application Programming Interface (API) propia, permitiendo desplegar el proyecto sin ninguna modificación adicional: *out of the box*.
7. **Verificar la corrección de la herramienta mediante pruebas.** Desarrollar pruebas unitarias y de integración sobre la gramática y el generador de código que garanticen su correcto funcionamiento ante entradas válidas e inválidas.
8. **Validar el DSL mediante casos de uso reales.** Demostrar la utilidad del DSL mediante la implementación de varios MAS que evalúen tanto la expresividad del lenguaje como la calidad del código generado.

1.4. Planificación

En todo proyecto de desarrollo software es esencial contar con una planificación y una metodología que permitan entregar el trabajo en el tiempo establecido. Sin ellas, la carga tiende a concentrarse en el tramo final, tal y como predice la ley de Parkinson. A continuación se describe la organización seguida en este trabajo.

1.4.1. Distribución del tiempo

El Trabajo de Fin de Grado tiene asignada una carga de 12 créditos en el plan de estudios, lo que equivale a unas 300 horas de trabajo. La dedicación se planificó distribuyendo aproximadamente 2 horas diarias en días lectivos y 4 horas los fines de



semana, lo que suma unas 14 horas semanales. El calendario resultante se recoge en la Figura 1.1:

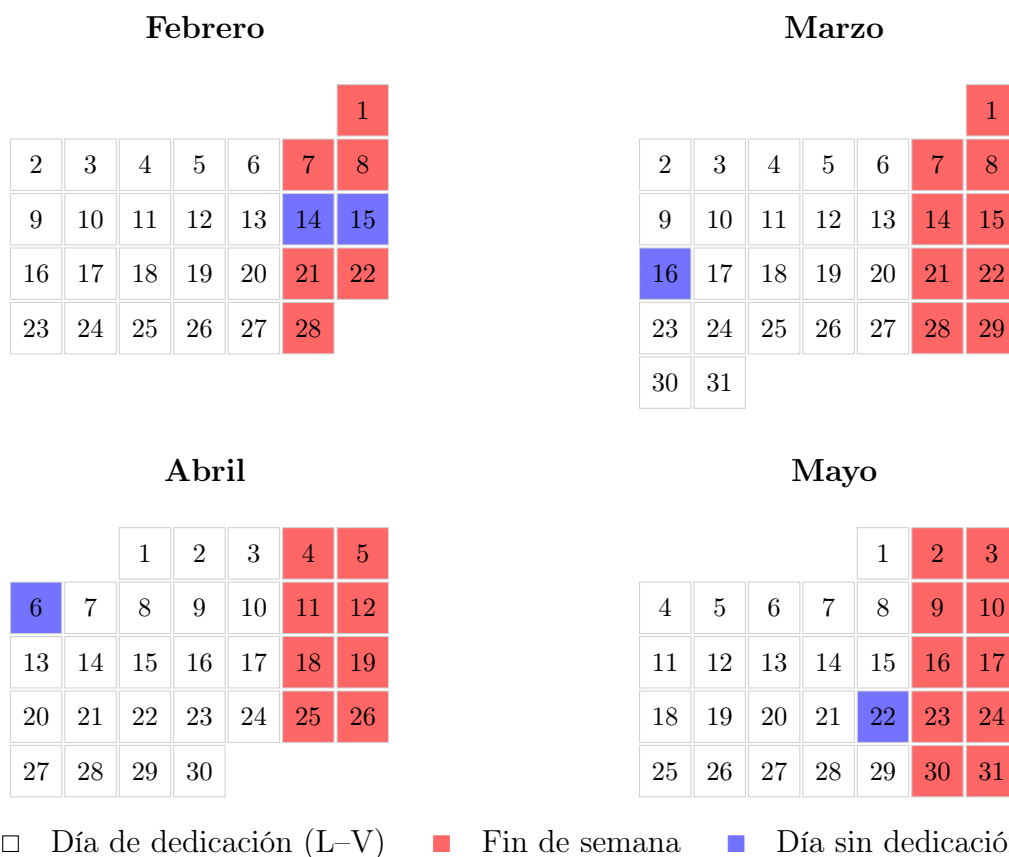


Figura 1.1: Calendario de disponibilidad

Es previsible que a lo largo del periodo de desarrollo surjan imprevistos —exámenes, entregas de otras asignaturas, compromisos personales— que reduzcan la dedicación en días concretos. Estas horas se compensaron mediante periodos de mayor disponibilidad, como la Semana Santa o el tramo posterior a la finalización de los exámenes ordinarios.

Con el objetivo de anticipar posibles contingencias, se identificaron los principales riesgos del proyecto y sus contramedidas asociadas:



Riesgo	Impacto	Probabilidad	Contramedida
Viajes y ausencias no previstas	Bajo	Alto	Las ausencias y viajes no previstos se compensarán con días de vacaciones reservados para la realización del TFG a tiempo completo.
Pérdida de información	Alto	Baja	Se dispondrá siempre de una copia actualizada diariamente en un repositorio de Github
Decisiones erróneas durante el desarrollo	Medio	Medio	Las decisiones importantes serán contrastadas frente a sus alternativas mediante diversos mecanismos como prototipos.
Alargamiento en el proceso de pruebas	Medio	Medio	Se implementará una batería automática de tests.

Tabla 1.1: Tabla de riesgos

1.4.2. Metodología y justificación

La elección de una metodología adecuada es clave para estructurar el trabajo de forma eficiente. A continuación se comparan las principales opciones consideradas:



Criterio	Cascada	Scrum	Incremental
Adaptabilidad a cambios	Baja	Alta	Alta
Overhead de gestión	Bajo	Alto	Bajo
Adecuado para trabajo individual	Sí	No	Sí
Entregas parciales funcionales	No	Sí	Sí
Requiere ceremonias formales	No	Sí	No

Tabla 1.2: Comparativa de metodologías de desarrollo

A la vista de esta comparativa, se optó por una **metodología incremental**. Scrum quedó descartado al tratarse de un trabajo individual: sus ceremonias asociadas —planificación de sprint, revisión y retrospectiva— suponen un *overhead* que no aporta valor en este contexto. El mantenimiento de un *backlog* de tareas resultó suficiente para organizar el trabajo. Bajo este enfoque, cada sprint abarca varias funcionalidades relacionadas que se desarrollan y validan de forma conjunta. La naturaleza incremental de la metodología permitió además incorporar los cambios que surgieron durante el desarrollo sin necesidad de replantear el trabajo previo.

El proyecto se articuló en cinco sprints de tres semanas cada uno. El *backlog* global fue refinándose conforme avanzaba el desarrollo, y al inicio de cada sprint se seleccionaban las tareas concretas que se abordarían durante el mismo. A continuación se resume el contenido de cada uno:

- **Sprint 1: Investigación y definición del alcance (2 feb – 22 feb)**

Investigación inicial de las tecnologías a emplear, revisión de trabajos similares existentes y definición del alcance del proyecto.

- **Sprint 2: Adaptación del DSL y MVP (23 feb – 15 mar)**



Primeras modificaciones al DSL para adaptarlo al objetivo del trabajo y elaboración de un Minimum Viable Product (MVP) funcional.

- **Sprint 3: Generador avanzado (16 mar – 5 abr)**

Mejoras en el generador de código: soporte a bifurcaciones, herramientas y paso de mensajes entre agentes. En general, los aspectos más complejos del generador.

- **Sprint 4: Validación semántica (6 abr – 26 abr)**

Continuación de las mejoras del generador y elaboración de las restricciones de validación semántica.

- **Sprint 5: Testing y extensión de Visual Studio Code (VSCode) (27 abr – 17 may)**

Inclusión de los tests automáticos y desarrollo de la extensión de VSCode como editor de modelos.

La distribución temporal de los sprints puede observarse en el diagrama de Gantt de la Figura 1.2.

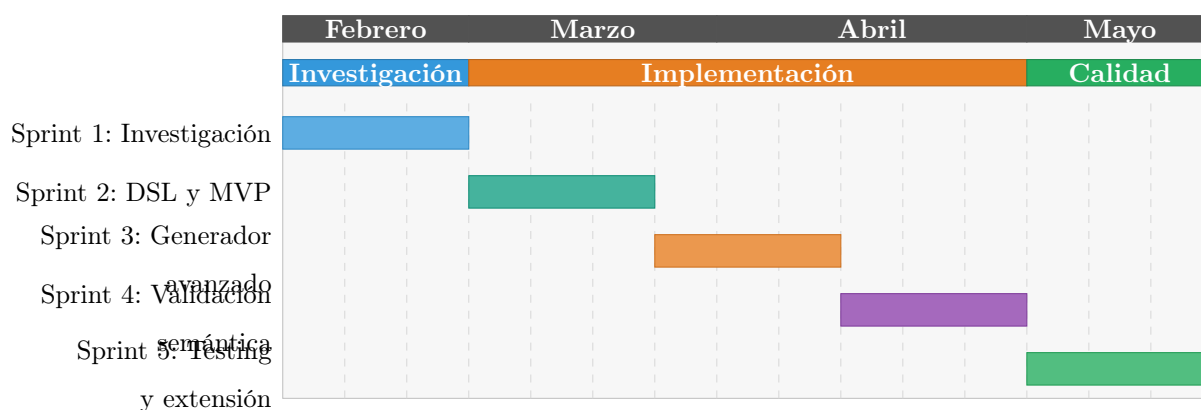


Figura 1.2: Diagrama de Gantt del proyecto

1.4.3. Sprints a detalle

A continuación se describe el trabajo realizado en cada sprint, incluyendo las decisiones técnicas más relevantes tomadas durante su desarrollo.



Sprint 1: Investigación y definición del alcance (2 feb – 22 feb)

El primer sprint se dedicó íntegramente a la fase de investigación. Se revisaron artículos científicos sobre la definición, arquitectura y funcionamiento de los MAS basados en LLM, con el objetivo de asentar una base conceptual sólida antes de comenzar el desarrollo.

Paralelamente, se evaluaron los frameworks disponibles para implementar el DSL y el generador de código. Tras analizar las alternativas, se seleccionó **Langium** como tecnología para la definición de la gramática y la validación del lenguaje, y **LangGraph** como destino del código generado. La justificación detallada de ambas elecciones se recoge en el Capítulo 2.7.

Sprint 2: Adaptación del DSL y MVP (23 feb – 15 mar)

Al inicio de este sprint se valoraron dos alternativas de diseño: reemplazar la estructura de comunicación definida en el paper del cotutor por un enfoque basado en componentes, o realizar una adaptación menos agresiva que respetase el modelo original. Se optó por la segunda vía, ya que preservaba la base teórica del trabajo y reducía el riesgo de desviación en el alcance.

Con esta decisión tomada, se desarrolló un MVP funcional capaz de generar código a partir de una definición básica: agentes con sus propiedades y una estructura de comunicación inicial. El objetivo era validar la viabilidad del enfoque antes de añadir mayor complejidad.

Sprint 3: Generador avanzado (16 mar – 5 abr)

Este sprint concentró el grueso del desarrollo del generador de código. Se añadió soporte para funcionalidades más complejas: herramientas (*tools*) asignables a agentes, distintos proveedores de modelos de IA, bifurcaciones en el flujo de ejecución y estructuras de comunicación adicionales. Se trató del sprint técnicamente más exigente del proyecto, ya que cada nueva capacidad requería extender tanto la gramática del DSL como la lógica del generador de forma coordinada.



Sprint 4: Validación semántica (6 abr – 26 abr)

Tras consolidar las funcionalidades principales, este sprint se orientó a mejorar la calidad del DSL. Se detectaron y eliminaron redundancias en el metamodelo, se simplificaron varios conceptos y se refactorizó el generador de código en consecuencia. Adicionalmente, se implementó soporte para la estructura de comunicación descentralizada, que hasta ese momento no estaba contemplada (el sistema solo soportaba las estructuras centralizada y por capas).

Por otro lado, se identificaron e implementaron las restricciones de validación semántica del lenguaje, en su mayoría detectadas de forma orgánica durante el propio desarrollo en Langium. Estas restricciones permiten notificar al usuario cuando un modelo es sintácticamente válido pero semánticamente incorrecto.

Sprint 5: Testing y extensión de VSCode (27 abr – 17 may)

El último sprint se dedicó a garantizar la calidad del sistema mediante tests automáticos. Se implementó una batería de pruebas con **Vitest** que cubre tanto las restricciones de validación del DSL como el generador de código. La extensión de VSCode se valida manualmente, ya que la automatización de pruebas para extensiones de este tipo introduce una complejidad adicional que excede el alcance del proyecto.

1.4.4. Desviaciones y gestión de imprevistos

A lo largo del proyecto se produjeron algunas desviaciones respecto a la planificación inicial, todas ellas identificadas y gestionadas sin comprometer la entrega final.

La primera desviación tuvo lugar en el Sprint 1, que se extendió una semana adicional. La causa fue la decisión de profundizar en el funcionamiento interno de las tecnologías seleccionadas antes de comenzar el desarrollo. En concreto, se dedicó tiempo a estudiar en detalle Langium y LangGraph; en el caso de este último, se completó el curso oficial de sus creadores para conocer en profundidad todas sus funcionalidades. Aunque esto retrasó el inicio del desarrollo, la inversión resultó rentable al evitar replanteamientos técnicos posteriores.



El Sprint 2 también sufrió una reducción de alcance respecto a lo planificado originalmente: el MVP iba a incluir soporte para herramientas (*tools*), pero esta funcionalidad se pospuso al sprint siguiente para compensar el retraso acumulado. La desviación se absorbió combinando una mayor dedicación horaria con una redefinición del alcance del entregable.

El Sprint 3 fue el más exigente del proyecto y se prolongó una semana más de lo previsto, al concentrar el mayor volumen de funcionalidades. No obstante, su coincidencia con el período de Semana Santa permitió dedicarle significativamente más horas de las habituales, superando con creces el objetivo semanal establecido. Gracias a este esfuerzo adicional, el retraso no se trasladó al Sprint 4, que se completó dentro del plazo previsto, dejando el calendario global sin afectación.

Capítulo 2

Estado del Arte

Contenidos

2.1. Auge de los LLM	16
2.1.1. Arquitectura y entrenamiento	16
2.1.2. Escala y capacidades emergentes	17
2.1.3. Técnicas de razonamiento y limitaciones	17
2.2. Agentes basados en LLM	18
2.2.1. Arquitectura del agente LLM	19
2.2.2. El protocolo MCP	20
2.2.3. El patrón ReAct	21
2.2.4. Limitaciones y transición a sistemas multiagente	21
2.3. Sistemas Multiagente	22
2.3.1. Estructuras de comunicación	22
2.3.2. Aplicaciones representativas	23
2.4. Frameworks de desarrollo de MAS	23
2.4.1. LangChain	23
2.4.2. LangGraph	24
2.4.3. CrewAI	24

2.4.4. AutoGen	25
2.4.5. Comparativa	25
2.5. Desarrollo dirigido por modelos y lenguajes específicos de dominio	26
2.5.1. Lenguajes específicos de dominio	26
2.6. Herramientas de definición de lenguajes	27
2.6.1. Eclipse EMF, Xtext y Eugenia	27
2.6.2. Langium	28
2.6.3. JetBrains MPS	28
2.6.4. Comparativa	29
2.7. Trabajos relacionados	29

2.1. Auge de los LLM

Durante décadas, el término *inteligencia artificial* evocaba sistemas estrechos y especializados: programas capaces de jugar al ajedrez, detectar anomalías o clasificar imágenes, pero incapaces de generalizar fuera de su dominio. La irrupción de los LLM ha transformado radicalmente este paradigma: un único modelo entrenado de forma general puede comprender instrucciones en lenguaje natural, razonar sobre problemas abstractos y asistir en tareas de programación, todo ello a través de una interfaz conversacional.

2.1.1. Arquitectura y entrenamiento

El salto cualitativo que posibilitó los LLM fue la arquitectura *Transformer*, introducida por Vaswani et al. [10]. A diferencia de los modelos recurrentes anteriores, que procesaban el texto de forma secuencial token a token, el Transformer calcula de forma simultánea la relevancia de cada token respecto a todos los demás mediante *atención multi-cabeza* (*multi-head self-attention*). Esto permite capturar dependencias semánticas a larga distancia y, al eliminar la recurrencia, habilita el entrenamiento masivamente paralelo



sobre *hardware* especializado. La arquitectura Transformer se convirtió así en la base sobre la que se construirían todos los grandes modelos de lenguaje posteriores.

El entrenamiento de un LLM se desarrolla en dos fases. La primera es el *preentrenamiento*: el modelo aprende a predecir el siguiente token en corpus masivos de texto, internalizando gramática, hechos del mundo y capacidad de razonamiento. Sin embargo, un modelo preentrenado produce texto fluido pero no necesariamente útil ni seguro. La segunda fase, Reinforcement Learning from Human Feedback (RLHF) [11], resuelve este problema: evaluadores humanos califican respuestas del modelo y sus preferencias guían un proceso de aprendizaje por refuerzo para producir respuestas más útiles y seguras. Este proceso es el que transformó GPT-3 en ChatGPT, convirtiendo modelos potentes pero impredecibles en asistentes que siguen instrucciones de forma confiable.

2.1.2. Escala y capacidades emergentes

A partir del Transformer, la investigación tomó una dirección clara: aumentar la escala de parámetros y datos. GPT-1 (2018, 117 millones de parámetros), GPT-2 (2019, 1,5B) y GPT-3 (2020, 175B) ilustran esta progresión. Ciertas capacidades —razonamiento aritmético, traducción sin entrenamiento específico, generación de código— no mejoran gradualmente con la escala, sino que aparecen de forma relativamente abrupta a partir de ciertos umbrales [12]. Este fenómeno de *capacidades emergentes* explica en parte la presión competitiva que ha impulsado a organizaciones como OpenAI, Google, Anthropic o Meta a escalar sus modelos de forma acelerada, generando el ecosistema diverso de modelos —GPT-4, Gemini, Claude, LLaMA— que existe en la actualidad.

2.1.3. Técnicas de razonamiento y limitaciones

El comportamiento del modelo en una tarea concreta está determinado en gran medida por el *prompt*. Los *system prompts* son instrucciones del desarrollador que condicionan el comportamiento del modelo durante toda la conversación; los *user prompts* son los mensajes del usuario en tiempo de interacción. La técnica *Few-Shot* consiste en incluir



ejemplos resueltos en el propio *prompt*, de modo que el modelo infiere el patrón sin necesidad de reentrenamiento. La técnica Chain of Thought (CoT) [13] instruye al modelo para descomponer su razonamiento en pasos intermedios explícitos, lo que mejora notablemente el rendimiento en tareas de razonamiento encadenado. Estas técnicas, y en especial la combinación de razonamiento con el uso de herramientas externas explorada en patrones como Reason + Act (ReACT) [14] —desarrollado en la sección 2.2—, han abierto la puerta a los sistemas agénticos.

A pesar de sus capacidades, los LLM presentan limitaciones inherentes. La más conocida es la *alucinación*: el modelo puede generar afirmaciones plausibles pero factualmente incorrectas, dado que su mecanismo de generación es probabilístico. La *ventana de contexto*, aunque ampliada en modelos recientes, impone un límite práctico a la información que el modelo puede considerar simultáneamente. Por último, los sesgos presentes en los datos de entrenamiento se reflejan en las salidas, lo que plantea desafíos de fiabilidad en aplicaciones de alto impacto.

2.2. Agentes basados en LLM

La sección anterior mostró cómo los LLM son capaces de razonar y generar respuestas sofisticadas, pero su comportamiento es fundamentalmente reactivo: reciben una entrada y producen una salida. La pregunta que emerge de forma natural es cómo pasar de un sistema conversacional a uno que pueda acometer tareas complejas de forma autónoma —como desarrollar un proyecto de *software* completo, investigar un tema consultando múltiples fuentes o gestionar un flujo de trabajo de varias etapas— sin intervención continua del usuario. La respuesta es el concepto de *agente basado en LLM*.

Un agente LLM puede describirse, en primera aproximación, como un sistema en el que el modelo de lenguaje actúa como núcleo cognitivo —el componente que razona y toma decisiones— mientras que una capa de orquestación lo dota de *percepción* del entorno, *memoria*, *capacidad de planificación* y *acción* sobre herramientas y servicios externos [15]. Si en la metáfora popular el LLM es el cerebro, el agente es el cuerpo que lo conecta al mundo.



2.2.1. Arquitectura del agente LLM

La arquitectura de referencia de un agente LLM se organiza en cuatro módulos funcionales: perfil, memoria, planificación y acción [15].

Perfil

El perfil define la identidad operativa del agente: su rol, sus capacidades declaradas y las restricciones que deben gobernar su comportamiento. En la práctica se materializa a través del *system prompt*: un conjunto de instrucciones que el desarrollador entrega al LLM antes de cualquier interacción y que condiciona de forma persistente su comportamiento. Especificar que el agente es un ingeniero de *software* experto en Python, que debe razonar en voz alta antes de actuar y documentar cada decisión, produce comportamientos cualitativamente distintos a los de un agente sin perfil definido. El perfil es, por tanto, el principal mecanismo de control que tiene el diseñador sobre el comportamiento de alto nivel del agente.

Memoria

La memoria determina qué información tiene disponible el agente en cada momento de su razonamiento. Se distinguen habitualmente dos niveles: la *memoria a corto plazo* o *en contexto*, que corresponde a la ventana de contexto del LLM y acumula el historial de razonamientos, acciones y observaciones de la sesión actual; y la *memoria a largo plazo*, externa al modelo y persistente entre sesiones, que permite al agente recuperar información relevante de interacciones anteriores.

Planificación

El módulo de planificación descompone el objetivo de alto nivel en una secuencia de pasos ejecutables. Gracias a las técnicas de razonamiento descritas en la sección anterior —CoT, Few-Shot— el agente puede generar planes en lenguaje natural sin que estos hayan sido codificados explícitamente, y replanificar cuando una acción no produce el resultado esperado.



Acción y uso de herramientas

El módulo de acción es el que conecta al agente con el entorno externo. Las herramientas disponibles para el agente son, en esencia, cualquier función, API o *software* externo que el agente puede invocar para realizar operaciones que no puede ejecutar mediante su capacidad de procesamiento interna: búsquedas en la web, ejecución de código, lectura y escritura de ficheros, consulta de bases de datos, o interacción con interfaces de usuario. El agente selecciona la herramienta adecuada en función de su razonamiento, la invoca con los parámetros necesarios y observa su resultado para continuar el proceso.

Un ejemplo concreto ilustra la necesidad de esta capacidad: un agente encargado de desarrollar *software* necesita no solo generar código sino también ejecutarlo, detectar errores, consultar documentación y crear ficheros en el sistema operativo. Ninguna de estas operaciones es accesible para un LLM sin herramientas.

2.2.2. El protocolo MCP

La proliferación de agentes y herramientas generó un problema de integración estructural conocido como el *problema $M \times N$* : con M modelos y N herramientas, integrar cada combinación requeriría un conector específico, lo que resulta inviable a escala. En noviembre de 2024, Anthropic presentó el *Model Context Protocol* (Model Context Protocol (MCP)) [16], un estándar abierto que define una interfaz universal entre los LLM y las herramientas externas: cada modelo y cada herramienta implementan el protocolo una sola vez, reduciendo el número de integraciones necesarias de $M \times N$ a $M + N$ —del mismo modo que el conector USB-C unifica las conexiones físicas entre dispositivos—. El protocolo ha tenido una adopción rápida y se ha convertido en una pieza infraestructural relevante para el desarrollo de agentes.



2.2.3. El patrón ReAct

El patrón ReACT [14] estructura el proceso mediante el cual un agente decide cuándo razonar, cuándo actuar y cómo integrar los resultados de sus acciones en su razonamiento. ReACT entrelaza en una única secuencia generada por el LLM trazas de razonamiento (*Thought*), pasos de acción (*Action*) y observaciones del entorno (*Observation*): el agente genera un pensamiento que analiza el estado actual, selecciona y ejecuta una acción —por ejemplo, invocar una herramienta de búsqueda—, observa el resultado y vuelve a razonar sobre el nuevo estado para planificar el siguiente paso. Este bucle se repite hasta que el objetivo ha sido alcanzado, lo que reduce los errores por alucinación al permitir que el agente verifique sus supuestos mediante acciones reales en lugar de comprometerse con un plan basado únicamente en conocimiento estático.

2.2.4. Limitaciones y transición a sistemas multiagente

Los agentes basados en un único LLM presentan limitaciones prácticas que afectan a su utilidad en tareas de alta complejidad. La más relevante es la relación entre especialización y fiabilidad: cuanto más amplio es el rango de tareas asignadas a un agente, mayor es la probabilidad de que incurra en alucinaciones o errores de planificación, ya que el modelo debe mantener simultáneamente contextos muy heterogéneos dentro de una ventana de contexto finita [2]. Asimismo, las cadenas de razonamiento largas tienden a perder coherencia: el agente puede olvidar el objetivo original, repetir acciones ya realizadas o quedar atrapado en bucles improductivos.

Estas limitaciones motivan de forma natural la distribución de responsabilidades entre múltiples agentes especializados, cada uno con un perfil, un conjunto de herramientas y una memoria acotados al dominio de su competencia. Es precisamente esta arquitectura la que se estudia en la siguiente sección bajo el nombre de *sistemas multiagente*.



2.3. Sistemas Multiagente

Un MAS es un sistema compuesto por múltiples agentes autónomos que perciben su entorno, toman decisiones y actúan para alcanzar objetivos individuales o colectivos [17]. En el contexto de los LLM, cada agente del sistema utiliza un modelo de lenguaje como núcleo cognitivo y se distribuyen entre ellos las responsabilidades del problema, de modo que cada uno opera sobre un perfil y un conjunto de herramientas acotados a su dominio de competencia [18]. Esta distribución resuelve buena parte de los problemas descritos en la sección anterior: la especialización reduce la heterogeneidad de contextos que cada agente debe manejar, y la colaboración entre agentes permite afrontar tareas que escapan a la capacidad de cualquiera de ellos por separado.

2.3.1. Estructuras de comunicación

La topología que organiza cómo se relacionan los agentes es uno de los elementos definitorios del MAS. La literatura identifica cuatro estructuras principales [18]:

- **En capas** (*layered*): los agentes se organizan en niveles secuenciales, de modo que la salida de cada capa es la entrada de la siguiente.
- **Centralizada**: un agente orquestador decide en cada paso qué agente debe actuar y agrega los resultados hasta dar la tarea por completada.
- **Descentralizada**: cada agente determina de forma autónoma cuál será el siguiente en actuar.
- **Memoria compartida** (*shared message pool*): todos los agentes acceden a un espacio de estado común, publicando y consumiendo información según su perfil y las necesidades de la tarea.

Estas estructuras no son excluyentes ni necesariamente lineales: los sistemas reales incorporan con frecuencia *bifurcaciones condicionales* que dirigen el flujo hacia un agente u otro en función del estado del sistema, lo que permite expresar lógica de control compleja dentro del MAS.



2.3.2. Aplicaciones representativas

Los MAS basados en LLM se aplican hoy en una variedad de dominios que ilustran la diversidad de problemas que esta arquitectura permite abordar [18]. En el ámbito del desarrollo de *software*, sistemas como MetaGPT [19] y ChatDev [20] simulan equipos completos de desarrollo asignando roles diferenciados —analista, arquitecto, programador, revisor— a cada uno de los agentes, que colaboran para producir aplicaciones funcionales a partir de un requisito de alto nivel. En investigación y análisis de información, los MAS permiten combinar agentes especializados en búsqueda, lectura crítica y síntesis para generar informes documentados sobre un tema. En simulación social, trabajos como el de Park et al. [21] muestran cómo agentes con perfiles diferenciados pueden modelar poblaciones que interactúan y exhiben comportamientos emergentes útiles para estudios de comportamiento o pruebas de políticas. Otros dominios destacados incluyen los entornos encarnados (*embodied*) y la coordinación robótica, donde varios agentes deben acordar acciones físicas en un mundo compartido.

2.4. Frameworks de desarrollo de MAS

Construir un MAS desde cero utilizando únicamente llamadas directas a la API de un LLM conduce rápidamente a un problema de escala: gestionar el estado compartido, orquestar el flujo entre agentes, integrar herramientas y mantener la coherencia del sistema requiere una cantidad creciente de código de infraestructura que no aporta valor al dominio del problema. Los frameworks de desarrollo abstraen esta complejidad. A continuación se describen los más relevantes en el contexto de este trabajo.

2.4.1. LangChain

LangChain [22] es la biblioteca de referencia para construir aplicaciones basadas en LLM. Proporciona abstracciones para cadenas de procesamiento, integración con herramientas y fuentes de datos, y soporte para agentes reactivos basados en patrones como ReACT. Su ecosistema es amplio y su comunidad muy activa, lo que lo convierte



en el punto de entrada habitual para el desarrollo con LLM.

Su limitación principal es que está orientado fundamentalmente a agentes individuales: la coordinación de múltiples agentes, los flujos con ciclos y la persistencia de estado no están contemplados de forma nativa. Esta carencia motivó el desarrollo de LangGraph como capa superior especializada.

2.4.2. LangGraph

LangGraph [23] extiende LangChain con un modelo de ejecución basado en grafos dirigidos. Cada nodo del grafo representa una unidad de procesamiento —un agente, una función o una herramienta— y las aristas definen las transiciones entre ellos. Las aristas pueden ser condicionales, lo que permite expresar bifurcaciones en función del estado del sistema. A diferencia de las cadenas lineales, LangGraph soporta ciclos, fundamentales para implementar bucles de razonamiento iterativo.

El elemento central es el *estado compartido*: un objeto tipado que se pasa entre todos los nodos y que cada uno puede leer y modificar, eliminando la necesidad de transmisiones explícitas de mensajes entre agentes. LangGraph incorpora además soporte nativo para *checkpointing*, que permite persistir el estado entre ejecuciones, y para *human-in-the-loop*, deteniendo la ejecución en puntos definidos para recibir validación humana. Su principal limitación es la curva de aprendizaje: modelar un sistema como un grafo requiere un mayor nivel de abstracción que el de frameworks de más alto nivel.

2.4.3. CrewAI

CrewAI [24] adopta un enfoque orientado a roles. Un sistema se define mediante un equipo (*crew*) compuesto por agentes, cada uno con un rol, un objetivo y un trasfondo en lenguaje natural. Las tareas se asignan a agentes concretos y el framework gestiona su ejecución de forma secuencial o jerárquica.

Su fortaleza es la facilidad de uso: con pocas líneas de código es posible definir un MAS funcional, lo que lo hace adecuado para prototipos y casos donde la lógica de coordinación no es compleja. Su limitación es el control reducido sobre el flujo: las posibilidades de



bifurcación condicional son escasas y la gestión del estado es menos explícita que en LangGraph.

2.4.4. AutoGen

AutoGen [25], desarrollado por Microsoft Research, se basa en un modelo de comunicación conversacional. Los agentes intercambian mensajes de texto y la coordinación emerge de estas conversaciones. Soporta configuraciones *human-in-the-loop* y es especialmente adecuado para tareas de debate o razonamiento iterativo entre agentes.

Su limitación es la menor estructura en la gestión del estado: al modelar la coordinación como conversaciones de texto, el control preciso del flujo y la persistencia resultan más complejos que en LangGraph.

2.4.5. Comparativa

La siguiente tabla resume las características principales de los frameworks descritos.

	LangChain	LangGraph	CrewAI	AutoGen
Soporte MAS nativo	No	Sí	Sí	Sí
Estado compartido	No	Sí (tipado)	Parcial	Parcial
Flujos cíclicos	No	Sí	No	Parcial
Bifurcaciones condicionales	No	Sí	No	No
Persistencia nativa	No	Sí	Sí	No
Human-in-the-loop	No	Sí	Parcial	Sí
Nivel de abstracción	Medio	Bajo	Alto	Medio

Tabla 2.1: Comparativa de frameworks de desarrollo de MAS

2.5. Desarrollo dirigido por modelos y lenguajes específicos de dominio

Los frameworks descritos en la sección anterior resuelven el problema de *ejecutar* un MAS, pero no el de *especificarlo*: el desarrollador sigue necesitando escribir código imperativo en TypeScript o Python para definir los agentes, sus herramientas y su topología de comunicación, y a medida que los sistemas crecen en complejidad esta especificación se vuelve difícil de mantener y de razonar sobre ella.

El *Desarrollo Dirigido por Modelos* (Model Driven Development (MDD)) aborda este problema invirtiendo la perspectiva habitual del desarrollo de *software*: en lugar de tratar el código fuente como el artefacto central del proceso, los modelos pasan a ser los artefactos de primer orden y el código se convierte en un producto derivado generado automáticamente a partir de ellos [26]. Un modelo captura la estructura y el comportamiento de un sistema a un nivel de abstracción superior al del código, expresado en términos del dominio del problema en lugar de en términos de la plataforma de implementación. Las transformaciones automáticas se encargan de traducir esos modelos a código ejecutable, reduciendo el esfuerzo manual y garantizando la coherencia entre la especificación y la implementación.

2.5.1. Lenguajes específicos de dominio

Un DSL es un lenguaje de programación o especificación diseñado para expresar conceptos de un dominio concreto, en contraposición a los lenguajes de propósito general como Java o Python, concebidos para abordar cualquier problema de computación [27]. Ejemplos ampliamente conocidos son SQL para la consulta de bases de datos relacionales, HTML para la descripción de documentos web, o las expresiones regulares para la especificación de patrones de texto.

Los DSL se clasifican habitualmente en dos categorías. Los DSL *internos* o *embebidos* se construyen sobre la sintaxis de un lenguaje de propósito general existente, aprovechando su infraestructura de herramientas. Los DSL *externos* definen una sintaxis propia e



independiente, lo que ofrece mayor libertad expresiva a costa de requerir una cadena de herramientas dedicada: una *gramática* que especifica formalmente qué construcciones son válidas en el lenguaje, un *analizador sintáctico* o *parser* que transforma el texto escrito en el DSL en una estructura de datos, y un *Abstract Syntax Tree (AST)* que representa de forma jerárquica la estructura del programa analizado y sobre el que operan tanto el validador semántico como el generador de código.

En el contexto del MDD, los DSL actúan como el lenguaje en el que se expresan los modelos del dominio. El usuario especifica su sistema en términos del DSL —sin conocer los detalles de la plataforma de implementación— y un generador de código transforma esa especificación en código ejecutable para la plataforma objetivo.

2.6. Herramientas de definición de lenguajes

El diseño de un DSL externo requiere una cadena de herramientas que cubra la definición de la gramática, la construcción del analizador sintáctico, la generación del AST y la integración con un entorno de desarrollo que ofrezca autocompletado, validación y resaltado de sintaxis. Los *language workbenches* proporcionan esta infraestructura de forma integrada, permitiendo al desarrollador centrarse en la semántica del dominio.

2.6.1. Eclipse EMF, Xtext y Eugenia

Eclipse Modeling Framework (EMF) [28] es el marco de metamodelado de referencia en el ecosistema Eclipse. Proporciona Ecore como lenguaje de metamodelado —alineado con EMOF, el subconjunto esencial del estándar MOF— y genera automáticamente código Java a partir de los metamodelos definidos. Sobre esta base, Xtext [29] permite definir la gramática de un DSL textual mediante una notación similar a EBNF; a partir de ella genera el analizador, la infraestructura de validación y una integración completa con el IDE Eclipse, incluyendo autocompletado y navegación. Eugenia [30], parte del proyecto Epsilon, complementa este ecosistema con soporte para DSL con sintaxis gráfica: actúa como una capa sobre el *Graphical Modeling Framework*



(GMF), permitiendo definir el editor visual mediante anotaciones directamente en el metamodelo Ecore en lugar de tener que construir los modelos intermedios de GMF manualmente.

Se trata del ecosistema más maduro y con mayor presencia en la literatura académica para el desarrollo de DSL. Su principal inconveniente es la complejidad de la cadena de herramientas: la configuración del entorno Eclipse, la gestión de dependencias entre plugins y la propia infraestructura EMF pueden resultar inestables y difíciles de mantener, especialmente en proyectos de tamaño reducido.

2.6.2. Langium

Langium [31] es un framework para la definición de DSL basado en TypeScript, desarrollado por TypeFox. Sigue el mismo planteamiento declarativo que Xtext —una gramática que genera parser, AST e infraestructura de validación— pero sobre la plataforma Node.js y el protocolo estándar LSP (*Language Server Protocol*). Al generar un servidor de lenguaje LSP, el DSL resultante es compatible de forma nativa con cualquier editor que implemente dicho protocolo; en particular, Langium genera automáticamente una extensión para VSCode lista para distribuir.

Es una herramienta de aparición relativamente reciente que está ganando tracción en la comunidad de ingeniería de lenguajes. Su ecosistema TypeScript moderno, la generación automática de la extensión de editor y la ligereza del entorno de desarrollo la posicionan como una alternativa sólida a las soluciones basadas en Eclipse.

2.6.3. JetBrains MPS

JetBrains MPS (*Meta Programming System*) [32] adopta un enfoque radicalmente distinto al de Xtext o Langium: en lugar de definir una gramática textual y generar un parser, MPS es un *editor proyeccional*. El usuario no edita texto libre que luego se analiza sintácticamente, sino que manipula directamente la estructura del AST a través de proyecciones visuales que pueden ser textuales, tabulares o gráficas simultáneamente. Esto elimina los problemas habituales de parsing y permite mezclar notaciones dentro de



un mismo programa.

MPS genera un entorno de edición completo basado en la plataforma IntelliJ y tiene una adopción industrial notable. Su principal desventaja es la elevada curva de aprendizaje: el modelo mental proyeccional difiere significativamente del desarrollo de software convencional, y la configuración del entorno es considerablemente más compleja que la de las alternativas basadas en gramáticas.

2.6.4. Comparativa

La siguiente tabla resume las características principales de los tres *language workbenches* descritos.

	Eclipse EMF + Xtext	JetBrains MPS	Langium
Ecosistema	Java / JVM	Java / JVM	TypeScript / Node.js
Enfoque	Gramática textual	Editor proyeccional	Gramática textual
DSL gráfico	Sí (Eugenia)	Sí (nativo)	No
Integración de editor	Eclipse	IntelliJ / MPS	VSCoDe (LSP)
Madurez	Alta	Alta	Media-alta
Complejidad de setup	Alta	Muy alta	Baja

Tabla 2.2: Comparativa de *language workbenches*

2.7. Trabajos relacionados

El trabajo más directamente relacionado con el presente Trabajo Fin de Grado (TFG) es el de Barriga et al. [1], que propone la aplicación del MDD al desarrollo de MAS basados en LLM. Su contribución principal es un metamodelo que formaliza la estructura

y el comportamiento de este tipo de sistemas.

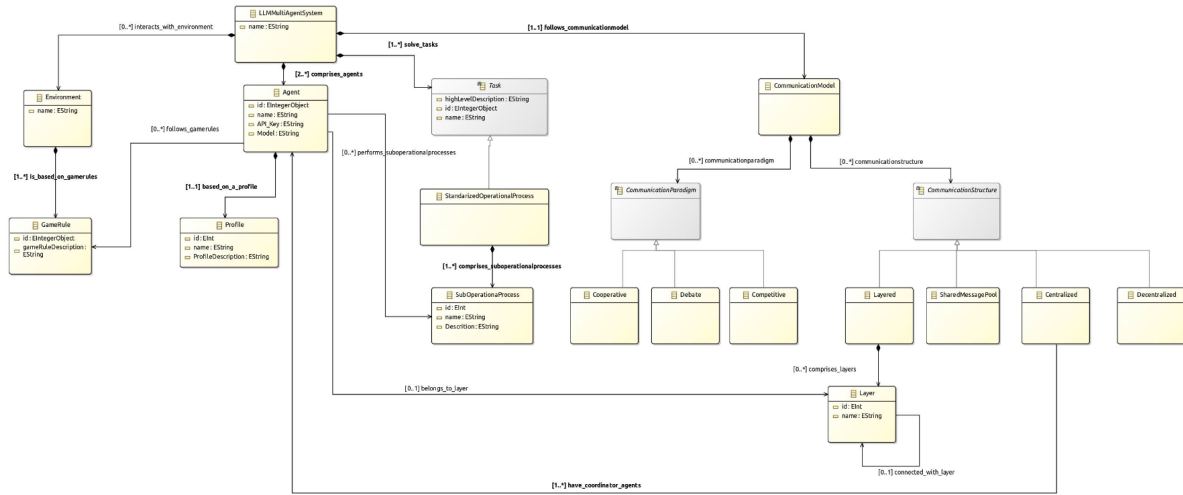


Figura 2.1: Metamodelo de MAS basado en LLM propuesto en [1].

El metamodelo define conceptos como *LLMMultiAgentSystem* como raíz del sistema, *Agent* con atributos de modelo y clave de API, *Profile* para la caracterización del agente, y *CommunicationModel* articulado en paradigma y estructura de comunicación. Incorpora además *GameRule* para definir reglas de comportamiento que los agentes deben respetar, y una jerarquía de procesos (*Task*, *StandardizedOperationalProcess*, *SubOperationProcess*) orientada a la descomposición estructurada de tareas.

Los propios autores reconocen que el trabajo se encuentra en curso: el metamodelo está siendo refinado y su aplicabilidad evaluada. Como trabajo futuro identifican el aumento de su expresividad, la definición de un marco de modelado gráfico y el desarrollo de las transformaciones modelo-texto que permitan la generación automática del MAS.

Capítulo 3

Propuesta de desarrollo

Contenidos

3.1. Motivación y justificación del enfoque	31
3.2. Descripción del sistema y tecnologías empleadas	32
3.2.1. Componentes del sistema	33
3.2.2. Otras herramientas	36
3.3. Alcance y limitaciones	37

En este capítulo se explica cuál es el sistema a desarrollar, justificando el enfoque de DSL y MDD, y detallando los paquetes de los que está conformado así como las tecnologías empleadas en cada uno de ellos.

3.1. Motivación y justificación del enfoque

Actualmente, el desarrollo de MAS es más un producto de artesanía que de ingeniería. Cuando se desarrolla una API, los equipos pueden definir los recursos y rutas necesarios con independencia del framework o las tecnologías empleadas — esto es lo que se conoce como el contrato de la API (OpenAPI, JSON Schema) —; sin embargo, cuando se trata de un MAS, no existen herramientas con este nivel de madurez.



Si bien están comenzando a surgir algunos estándares como Agent Communication Protocol (ACP) [33], estos resuelven el problema de cómo se comunican los agentes entre sí, pero no el de cómo definir un sistema completo. Esto deriva en que no exista una forma estándar de describir estos sistemas, haciendo que cada equipo defina sus componentes por separado: diagramas de flujo para la orquestación, gestión del estado para el almacenamiento, definición de las capacidades de cada agente...

Bajo esta premisa, parece adecuado el desarrollo de un DSL que estandarice la forma de definir los MAS con independencia del framework destino, con el objetivo de proporcionar una visión unificada del sistema: qué agentes operarán, qué resuelve cada uno y cómo puede hacerlo, con qué otros agentes se comunica, cómo se gestionará el estado compartido, etc.

3.2. Descripción del sistema y tecnologías empleadas

El sistema a desarrollar proporciona una interfaz mediante la cual se pueden crear y validar modelos conforme al DSL definido. Junto al editor, se implementa un generador de código hacia LangGraph como primera transformación concreta del modelo. Dado que el DSL actúa como descripción independiente del framework, este podría dar lugar a distintas implementaciones según el generador empleado; LangGraph constituye en este caso la primera de ellas.

El editor y el generador se distribuyen en forma de extensión de VSCode, dado que es uno de los Integrated Development Environment (IDE) más populares del mundo y cuenta además con una amplia compatibilidad con editores derivados como VSCodium o Cursor. Una alternativa habría sido desarrollar una aplicación web que permitiese definir, validar y generar modelos con independencia del IDE del desarrollador. Sin embargo, la elección tecnológica adoptada para el desarrollo del DSL —Langium, justificada más adelante— ofrece la generación automática de una extensión base de VSCode, lo que reduce drásticamente el esfuerzo de implementación frente a una aplicación web equivalente, que habría requerido construir desde cero la interfaz, el editor y la integración con el servidor de lenguaje.

El sistema opera en dos fases bien diferenciadas. En la primera, el usuario edita el archivo `.mad` en el editor; el Language Server Protocol (LSP) valida el modelo en tiempo real y muestra los errores de forma inline hasta que el modelo es correcto. En la segunda, el usuario invoca el generador explícitamente; el Command Line Interface (CLI) lee el modelo validado y produce el proyecto LangGraph listo para desplegar. A continuación se ilustra el flujo de cada fase.

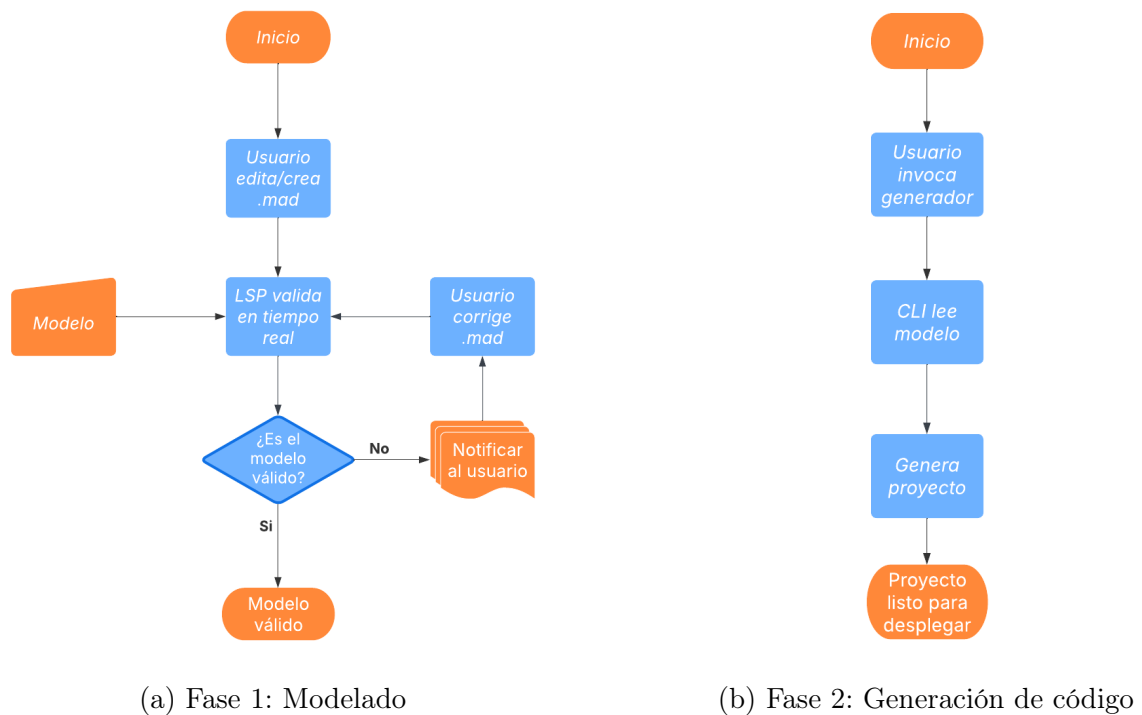


Figura 3.1: Flujo del sistema por fases

3.2.1. Componentes del sistema

El sistema se dividirá en tres paquetes de software independientes entre sí.

Paquete	Tecnología principal	Función
language	Langium (TypeScript)	Define la gramática del DSL, validación semántica, autocompletado y hover. Actúa como Language Server (LSP).
extension	VSCoDe Extension API	Integra el Language Server en VSCoDe, habilitando soporte del DSL en el editor.
cli	Node.js CLI	Lee archivos .mad y genera código Python/LangGraph listo para ejecutar.

Tabla 3.1: Paquetes del proyecto

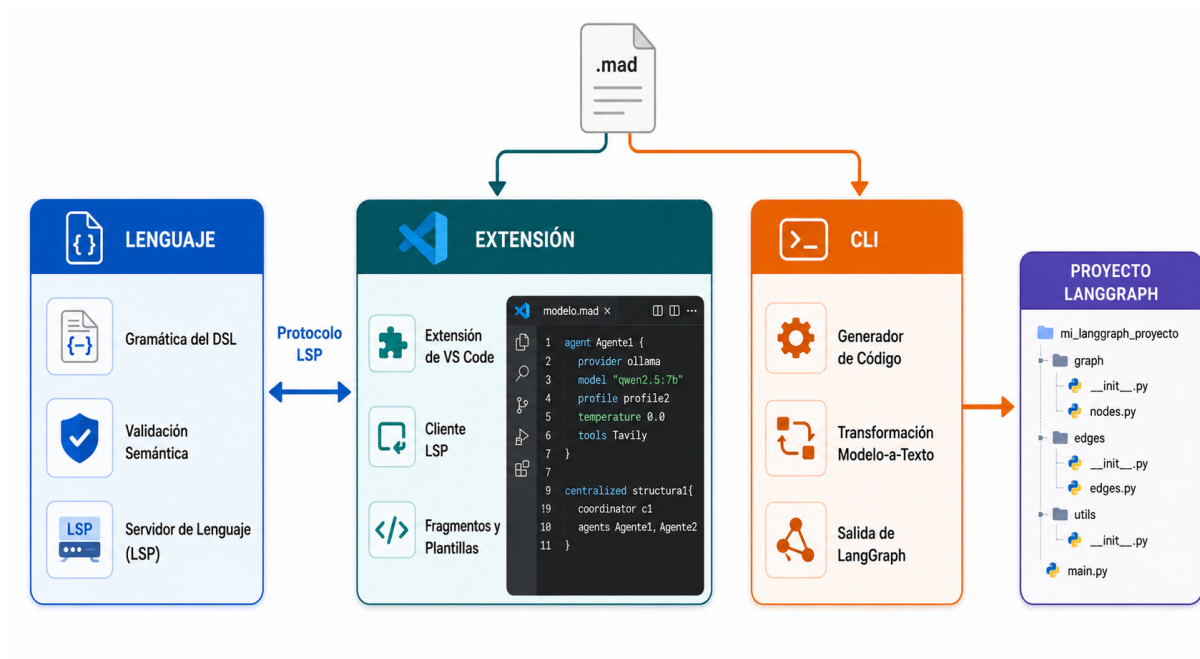


Figura 3.2: Arquitectura del sistema y relación entre paquetes

A continuación se presenta en detalle cada uno de los componentes que constituyen el sistema.

Language Este componente definirá la gramática y sintaxis del DSL a desarrollar, además de las restricciones del lenguaje. Es la base del sistema y será independiente del



generador, ya que el DSL debe ser independiente del framework de salida, por lo que el metamodelo establecido deberá soportar generadores de código para cualquier framework.

Para llevar a cabo su desarrollo se ha optado por emplear Langium [31], dado que en el ámbito del desarrollo de DSL es una de las opciones más modernas que existen. Sus principales ventajas son su curva de aprendizaje reducida, la capacidad de generar la base de una extensión de VSCode fácilmente y el soporte de LSP out of the box.

Entre las opciones valoradas, Eclipse EMF + Xtext fue descartado dado que, pese a contar con experiencia previa, el entorno resultó inestable y está atado a Java. Por su parte, JetBrains MPS [32] fue descartado por su alta curva de aprendizaje.

La decisión tecnológica de este componente era la más importante del proyecto, dado que la extensión y el generador de código girarán en torno a él.

Extension Constituye el editor de modelos del sistema. Al haber empleado Langium para el DSL, se aprovecha su capacidad de generar automáticamente una extensión base de VSCode, a la cual se añadirán las funciones necesarias para que actúe como editor del DSL.

Esta pieza es el nexo entre el lenguaje y el generador, dado que desde el editor se podrá invocar el generador para producir el código.

Uno de los problemas que suelen enfrentar los DSL es la barrera de entrada de los nuevos usuarios. Para reducirla, la extensión incluirá una serie de ayudas como snippets, plantillas, descripciones de cada componente, etc.

CLI Al igual que la extensión, el paquete CLI forma parte del andamiaje que genera `generator-langium`, proporcionando la estructura base del generador de código sobre la que se construye la transformación concreta. Será el encargado de transformar los modelos en código Python listo para desplegar sobre LangGraph [23]. Se ha seleccionado LangGraph como framework de salida por dos razones principales: permite integrar soporte para LangSmith Studio, que proporciona una Graphic User Interface (GUI) para depurar el sistema generado, y facilita el despliegue mediante la generación de un contenedor Docker con una API incluida. Además, LangGraph es uno de los frameworks



MAS con mayor curva de aprendizaje, por lo que facilitar su código mediante un DSL aporta un valor tangible al desarrollador.

El generador producirá una estructura de proyecto como la siguiente:

```
/
├── agents.py
├── checkpointer.py    gestión de la persistencia
├── config.py
├── graph.py
├── langgraph.json    necesario para LangSmith Studio
├── prompt.py
├── requirements.txt
├── state/
│   ├── __init__.py
│   ├── state.py
│   └── reducers.py    lógica de reducción del estado compartido
├── subgraph/
│   ├── answerWriter.py
│   ├── ExtractorGraph.py
│   ├── specializedAssistants.py
│   └── ValidationGraph.py
```

3.2.2. Otras herramientas

Cabe destacar el uso de las siguientes tecnologías de soporte:

Node.js Langium exige Node.js v16 o superior como requisito de entorno; sin él no es posible instalar el generador Yeoman ni ejecutar ninguno de los paquetes del proyecto. En tiempo de ejecución tiene dos roles concretos:

- **Runtime del Language Server:** el proceso del servidor de lenguaje (`packages/language`) se ejecuta como un proceso Node.js independiente con el



que VSCode se comunica vía IPC siguiendo el protocolo LSP.

- **Runtime del CLI:** el generador de código (`packages/cli`) se distribuye y ejecuta como un binario Node.js, permitiendo invocar la generación desde la terminal con `multi-agent-dsl-cli generate <archivo.mad>`.

Vitest El generador de proyectos de Langium (`generator-langium`) incluye como opción de andamiaje la configuración de tests con Vitest, integrándolo como framework de testing del ecosistema. Se emplea para validar tanto la gramática y las restricciones semánticas del lenguaje como la corrección del código generado por el CLI.

3.3. Alcance y limitaciones

El DSL permitirá definir un MAS indicando los agentes que lo forman junto a sus parámetros de configuración: el perfil con el que se ejecuta, la estructura de comunicación a la que pertenecen, la información que intercambian entre ellos y los mecanismos con los que se gestionarán los mensajes.

Quedan fuera del alcance:

- **Planificación explícita:** cómo descompone cada agente sus tareas internamente no se puede definir en el DSL; esta lógica queda delegada al LLM en tiempo de ejecución a través del prompt de perfil.
- **Human in the loop:** en esta versión inicial no se modelará la solicitud de aprobación humana por parte de un agente antes de ejecutar un proceso, ya que su inclusión añadiría complejidad al metamodelo sin aportar al objetivo principal de validar el enfoque DSL.
- **Modelos y proveedores:** si bien LangGraph da soporte para una gran variedad de modelos y proveedores, solo se incluirán los más extendidos, ya que añadir todos produciría una carga extra en la fase de testeo sin aportar valor diferencial.



- **Condiciones avanzadas en bifurcaciones:** el DSL permitirá definir bifurcaciones con condición, pero los mecanismos de condición se limitarán a casos básicos; lógica condicional compleja queda fuera del alcance.
- **Implementación de herramientas:** el modelo permitirá definir las conexiones con las herramientas a las que tenga acceso un agente (path de una función Python, servidor y método de transporte de una función MCP), pero no modelará qué hace cada herramienta; el generador no producirá código de herramientas.
- **Generadores adicionales:** se ha optado por implementar un único generador, hacia LangGraph, como prueba de concepto que demuestre que el DSL puede dar lugar a una transformación completa y desplegable; la incorporación de generadores hacia otros frameworks queda como extensión natural del trabajo.
- **Un sistema por archivo:** cada archivo `.mad` definirá un único modelo; no se permitirá distribuir un modelo en distintos archivos.
- **Estructura `sharedMessagePool`:** el DSL no contempla esta estructura de comunicación, en la que los agentes publican y consumen sobre un pool común de mensajes; su incorporación al metamodelo y al generador queda como línea futura.
- **Tipos compuestos en el estado:** los atributos del bloque `context` se limitan a tipos primitivos (`string`, `int`, `boolean`); la incorporación de listas y de literales (conjuntos cerrados de valores admitidos) queda como ampliación natural del lenguaje.
- **Tests automatizados de la interfaz de la extensión:** la verificación de los comandos y elementos visuales de la extensión se realiza mediante inspección manual, dado que automatizar pruebas sobre la interfaz de VSCode requiere infraestructura específica que escapa al objetivo del proyecto.

Capítulo 4

Requisitos

Contenidos

4.1. Requisitos funcionales	40
4.1.1. Lenguaje (<code>packages/language</code>)	40
4.1.2. Extensión (<code>packages/extension</code>)	43
4.1.3. Generador (<code>packages/cli</code>)	44
4.2. Requisitos no funcionales	46

Los requisitos recogidos en este capítulo no se derivaron de un análisis previo cerrado, sino mediante un proceso iterativo: partiendo del metamodelo propuesto en [1] como base, se definieron varios modelos de MAS de ejemplo y se implementó manualmente el código LangGraph equivalente para cada uno. De ese ejercicio emergieron, por un lado, los elementos que el DSL debía permitir expresar y, por otro, los artefactos que el generador debía producir. Algunas de las necesidades detectadas se descartaron deliberadamente del alcance del proyecto (sección 3.3) y se mantienen documentadas como mejoras futuras, por lo que no aparecen en las tablas que siguen.

En este capítulo se presentan los requisitos resultantes; se organizan en funcionales y no funcionales. Los identificadores de los requisitos funcionales siguen el esquema **RF-XNN**, donde la letra indica el paquete al que pertenecen: L para `packages/language`, E para la



extensión y G para el generador. Los no funcionales se identifican con el prefijo RNF.

4.1. Requisitos funcionales

4.1.1. Lenguaje (packages/language)

Este paquete cubre dos ámbitos: la gramática, que define qué puede expresarse en el DSL, y el validador, que comprueba la corrección semántica de los modelos. A continuación se presentan los requisitos de cada uno por separado. Los requisitos del validador incluyen entre paréntesis un identificador de restricción (R01–R15) que referencia la restricción concreta implementada en el servicio de validación de Langium. Estas restricciones son equivalentes conceptualmente a las que en entornos de metamodelado clásicos se expresarían en Object Constraint Language (OCL); en Langium se implementan directamente como funciones TypeScript. La verificación de los requisitos de este paquete se realiza mediante tests automatizados.

Gramática

ID	Requisito
RF-L01	El lenguaje debe permitir definir un bloque context con atributos tipados (string , int , boolean) con descripción obligatoria
RF-L02	El lenguaje debe permitir definir perfiles (profile) reutilizables que actúan como prompt de sistema de los agentes
RF-L03	El lenguaje debe permitir declarar agentes con proveedor, modelo, perfil, herramientas, tiempo máximo de respuesta, temperatura, máximo número de intentos y referencias al estado



ID	Requisito
RF-L04	El lenguaje debe permitir declarar herramientas Python locales (<code>pythonTool</code>) y servidores MCP (<code>mcpServer</code>)
RF-L05	El lenguaje debe permitir definir estructuras de comunicación <code>layered</code> , <code>centralized</code> y <code>decentralized</code>
RF-L06	El lenguaje debe permitir definir transiciones entre estructuras con condiciones opcionales sobre atributos del estado
RF-L07	El lenguaje debe permitir declarar un nodo <code>Summarizer</code> para gestión del historial de mensajes
RF-L08	El lenguaje debe permitir declarar la estrategia de persistencia del estado (<code>inMemorySave</code> , <code>postgresSave</code> , <code>mongoSave</code>)
RF-L09	El lenguaje debe definir como proveedores válidos OpenAI, Anthropic, Google (<code>google_genai</code>) y Ollama

Tabla 4.1: Requisitos funcionales del lenguaje: gramática

Validador

ID	Requisito
RF-L10	El validador debe comprobar que el modelo LLM declarado es válido para el proveedor indicado
RF-L11	El validador debe comprobar que todas las estructuras de comunicación participan en al menos una transición (R01)
RF-L12	El validador debe comprobar que no existen URLs duplicadas entre servidores MCP (R02)
RF-L13	El validador debe comprobar que existe exactamente un punto de inicio <code>START</code> (R03)



ID	Requisito
RF-L14	El validador debe comprobar que no existen arcos duplicados con condición entre el mismo par origen-destino (R04)
RF-L15	El validador debe comprobar que la única transición desde un origen no lleva condición (R05)
RF-L16	El validador debe comprobar que el tipo del valor en una condición es compatible con el tipo del atributo referenciado (R06)
RF-L17	El validador debe comprobar que no existe más de una transición sin condición desde un mismo origen (R07)
RF-L18	El validador debe comprobar que la temperatura de cualquier actor está entre 0.0 y 1.0 (R08)
RF-L19	El validador debe comprobar que solo existe un Summarizer en el sistema (R09)
RF-L20	El validador debe comprobar que los valores numéricos opcionales (maxToken , timeOut , maxRetries , tokenTrigger , maxMessages) son positivos (R10)
RF-L21	El validador debe comprobar que las estructuras layered no contienen ciclos (R11)
RF-L22	El validador debe comprobar que las apiKeyName de los servidores MCP son únicas (R12)
RF-L23	El validador debe comprobar que todos los elementos del sistema tienen nombres únicos (R13)
RF-L24	El validador debe comprobar que el nodo Summarizer no tiene perfil asignado (R14)
RF-L25	El validador debe comprobar que cada agente pertenece a una única estructura de comunicación (R15)

Tabla 4.2: Requisitos funcionales del lenguaje: validador



4.1.2. Extensión (packages/extension)

Este paquete actúa como nexo entre el lenguaje y el generador, proporcionando al usuario la interfaz de edición y las ayudas de productividad dentro de VSCode. La verificación de estos requisitos se realiza mediante inspección manual, dado que automatizar tests sobre la interfaz de la extensión queda fuera del alcance del proyecto.

ID	Requisito
RF-E01	La extensión debe activar el Language Server al abrir cualquier archivo <code>.mad</code>
RF-E02	La extensión debe proporcionar resaltado de sintaxis para archivos <code>.mad</code> mediante gramática TextMate
RF-E03	La extensión debe proporcionar autocompletado de referencias filtrado por el tipo esperado en cada posición
RF-E04	La extensión debe mostrar documentación contextual (hover) sobre las palabras clave del DSL y sobre los nodos del modelo
RF-E05	La extensión debe proporcionar snippets para las construcciones más habituales del lenguaje
RF-E06	La extensión debe mostrar los errores y warnings del validador subrayados en tiempo real mientras el usuario escribe
RF-E07	La extensión debe exponer un comando accesible desde la barra del editor para generar el código Python del modelo activo
RF-E08	El comando de generación debe abortar y notificar al usuario si el modelo contiene errores de validación
RF-E09	La extensión debe exponer un comando para visualizar el modelo como diagrama Mermaid en un panel lateral
RF-E10	El diagrama debe actualizarse automáticamente al guardar el archivo <code>.mad</code> si el panel de previsualización ya está abierto



ID	Requisito
RF-E11	La extensión debe advertir al usuario cuando use combinaciones de proveedor y estructura que pueden producir comportamientos incorrectos

Tabla 4.3: Requisitos funcionales de la extensión

4.1.3. Generador (packages/cli)

Este paquete transforma los modelos validados en proyectos Python listos para desplegar sobre LangGraph, produciendo un artefacto por cada elemento declarado en el modelo. La verificación de estos requisitos se realiza mediante tests automatizados sobre el código generado.

ID	Requisito
RF-G01	El generador debe producir <code>prompts.py</code> con las cadenas de texto de los perfiles de los agentes
RF-G02	El generador debe producir <code>state.py</code> con el <code>TypedDict</code> del estado compartido derivado del <code>context</code> del modelo
RF-G03	El generador debe producir <code>agents.py</code> con la instanciación de los LLMs y sus esquemas de salida estructurada (<code>BaseModel</code>)
RF-G04	El generador debe producir <code>graph.py</code> con el grafo LangGraph completo incluyendo nodos, aristas y funciones de enrutamiento
RF-G05	El generador debe producir <code>.env.template</code> con todas las variables de entorno necesarias según los proveedores y herramientas usados
RF-G06	El generador debe producir <code>requirements.txt</code> con las dependencias Python exactas según el contenido del modelo



ID	Requisito
RF-G07	El generador debe producir <code>langgraph.json</code> para el despliegue con <code>langgraph dev</code> y <code>langgraph build</code>
RF-G08	El generador debe producir <code>checkpointinter.py</code> con la configuración de persistencia correspondiente al tipo declarado en el modelo
RF-G09	El generador debe producir código correcto para la estructura de comunicación <code>layered</code>
RF-G10	El generador debe producir código correcto para la estructura de comunicación <code>centralized</code> , etiquetando los mensajes de los agentes con su nombre
RF-G11	El generador debe producir código correcto para la estructura de comunicación <code>decentralized</code> , usando <code>Command</code> de <code>LangGraph</code> para el enrutamiento
RF-G12	El generador debe combinar múltiples estructuras de comunicación como subgrafos dentro de un grafo principal
RF-G13	El generador debe producir el cliente MCP (<code>mcpClients.py</code>) para cada <code>mcpServer</code> declarado en el modelo
RF-G14	El generador debe ser invocable desde la línea de comandos mediante <code>multi-agent-dsl-cli generate <archivo.mad></code>

Tabla 4.4: Requisitos funcionales del generador



4.2. Requisitos no funcionales

ID	Requisito
RNF01	El Language Server debe procesar y validar el modelo en tiempo real sin demoras perceptibles durante la edición
RNF02	El código generado debe ser ejecutable directamente con <code>langgraph dev</code> sin requerir modificaciones manuales
RNF03	El generador debe ser determinista: el mismo modelo <code>.mad</code> debe producir siempre el mismo código Python
RNF04	Los paquetes <code>language</code> y <code>cli</code> deben disponer de tests automatizados que cubran las restricciones del validador y los artefactos del generador
RNF05	La extensión debe ser compatible con VSCode ≥ 1.67
RNF06	Los modelos de ejemplo deben poder generarse y ejecutarse de extremo a extremo sin intervención manual más allá de rellenar las API keys
RNF07	La extensión y el CLI deben ser compatibles con Windows, macOS y Linux
RNF08	Los mensajes de error del validador deben identificar el elemento afectado y describir la causa del problema para facilitar su corrección
RNF09	La arquitectura del sistema debe permitir incorporar nuevos generadores sin modificar los paquetes <code>language</code> ni <code>extension</code>

Tabla 4.5: Requisitos no funcionales

Capítulo 5

Diseño

Contenidos

5.1. Diseño del DSL	48
5.1.1. Iteraciones del metamodelo	48
5.1.2. Metamodelo final	56
5.1.3. Sintaxis concreta	59
5.1.4. Restricciones semánticas	64
5.2. Diseño del generador	67
5.2.1. Artefactos generados	68
5.2.2. Mapeo del modelo a LangGraph	70
5.3. Diseño de la extensión	73
5.3.1. Funcionalidades base de Langium	73
5.3.2. Funcionalidades implementadas	74

En este capítulo se describe el proceso de diseño del sistema, abordado de forma iterativa e incremental a lo largo de los distintos sprints del proyecto. El capítulo se organiza en tres secciones, una por cada componente del sistema: el DSL, el generador de código y la extensión de VSCode.



5.1. Diseño del DSL

El diseño del DSL se llevó a cabo mediante una serie de iteraciones sobre el metamodelo propuesto en [1], tomándolo como punto de partida e incorporando los cambios necesarios para adaptarlo al alcance y objetivos de este trabajo. A continuación se presentan las distintas evoluciones del metamodelo junto con las decisiones que las motivaron.

5.1.1. Iteraciones del metamodelo

Iteración 1: Ajuste del alcance. Partiendo del metamodelo propuesto en [1], la primera iteración consistió en ajustar sus conceptos al alcance definido para este trabajo. En dicho metamodelo, los MAS se modelaban como sistemas que operan en un `Environment`, gobernado por un conjunto de `GameRule` que especifican cómo deben comportarse los agentes. El sistema estaba compuesto por `Agent`, cada uno con nombre, modelo y clave API. Los agentes estructuraban su comportamiento mediante el concepto de `Task`, que dividía el flujo de trabajo en `StandardizedOperationalProcess`, compuesto a su vez por `SubOperationProcess`, representando cada uno un paso operacional específico. Por último, las interacciones entre agentes se articulaban a través de un `CommunicationModel`, dividido en `CommunicationParadigm`, que definía la actitud de los agentes (cooperativo, debate o competitivo), y `CommunicationStructure`, que definía el orden en que se comunicaban.

Tomando este metamodelo como base, se tomaron las siguientes decisiones de diseño:

- **Eliminación de `Task`, `StandardizedOperationalProcess` y `SubOperationProcess`.** Su inclusión añadiría complejidad al metamodelo sin contribuir al objetivo principal de validar el enfoque DSL. La planificación interna de cada agente queda delegada al LLM en tiempo de ejecución a través de su perfil.
- **Eliminación de `CommunicationParadigm`.** El comportamiento actitudinal de los agentes puede expresarse de forma mucho más rica y flexible a través del prompt de perfil, en lugar de limitarlo a tres subtipos predefinidos.



- **Profile como elemento de primer nivel.** Los perfiles se desvincularon del agente y pasaron a declararse desde el nodo raíz del sistema, permitiendo que varios agentes compartan el mismo perfil sin duplicar su definición.
- **Eliminación del atributo apiKey de Agent.** Incluir un valor secreto directamente en el modelo constituye una mala práctica de seguridad; las claves de API se gestionan mediante variables de entorno en el proyecto generado.

El resultado de estos ajustes constituye la base sobre la que se desarrollarán las iteraciones siguientes, recogida en la Figura 5.1. En ella, un MAS queda representado como un conjunto de **Agent** organizados en una **CommunicationStructure**, cada uno guiado por un **Profile** que actúa como su prompt de sistema. Los conceptos de **Environment** y **GameRule** permanecen en el modelo pero quedan sin desarrollar en esta iteración, pendientes de revisión en ciclos posteriores.

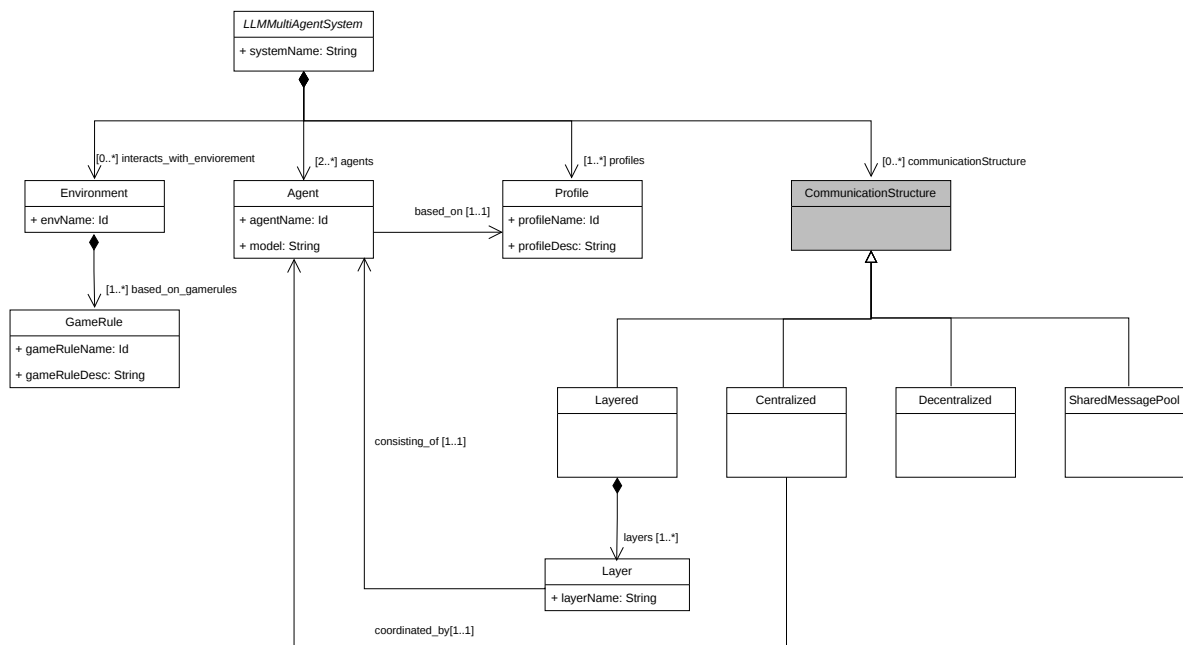


Figura 5.1: Metamodelo — Iteración 1

Iteración 2: Producción del MVP. El objetivo de esta iteración era claro: llevar el metamodelo hasta el punto de soportar un MVP funcional del generador. Para ello



se diseñaron dos modelos de MAS sencillos y se implementó manualmente el código equivalente, con el fin de detectar las carencias del metamodelo actual.

- **Research Assistant.** Sistema con estructura **Centralized** en el que un coordinador delegaba en distintos agentes según la fase de elaboración de un documento de investigación. Del análisis de este modelo se extrajeron las siguientes conclusiones:

- Los agentes solo podían escribir en el campo de mensajes, que se saturaba fácilmente, haciendo que los agentes ignorasen su propio trabajo previo.
- El uso de herramientas resultaba imprescindible para sistemas con cierta complejidad.
- El coordinador debía ser un tipo de nodo diferenciado del resto de agentes, con un prompt estructurado que no dependiese exclusivamente del criterio del diseñador; de lo contrario, el correcto enrutamiento entre agentes quedaba como responsabilidad del usuario, que no tiene por qué conocer el funcionamiento interno del sistema.

- **CV Reviewer.** Sistema con estructura **Layered** que implementaba un pipeline secuencial en el que varios agentes analizaban un CV, extraían sus características clave y notificaban al candidato por correo electrónico el resultado de la evaluación. Este modelo confirmó y añadió lo siguiente:

- Reforzó la necesidad de herramientas para sistemas de interés práctico.
- Los agentes requerían un mecanismo de almacenamiento de información más allá del campo de mensajes.
- Los agentes necesitaban atributos adicionales de configuración. Por otro lado, especificar el modelo mediante un **String** libre resultaba frágil: el usuario podía introducir cualquier valor, lo que impedía conocer el proveedor en tiempo de generación. Separar proveedor y modelo en atributos distintos resolvía ambos problemas.



A partir de los problemas detectados se introdujeron los siguientes cambios en el metamodelo:

- **Incorporación de Tool.** Se modeló el concepto de herramienta con dos subtipos: `PythonTool`, que referencia una función Python definida por el usuario mediante su nombre y ruta, y `McpServer`, que declara la URL del servidor MCP y una lista de nombres de herramienta (`String`) que se desean exponer. Separar el servidor de la lista de herramientas evita abrir conexiones independientes al mismo servidor para cada herramienta y previene la duplicación de claves API; sin esta jerarquía, el agente recibiría todas las herramientas del servidor indiscriminadamente, lo que podría sobrecargarlo.
- **Eliminación de Environment y GameRule.** Con la incorporación de `Tool` y `Profile`, estos conceptos dejan de aportar valor: la percepción del entorno queda cubierta por las herramientas, y las restricciones de comportamiento se expresan intrínsecamente a través del perfil del agente.
- **Incorporación de Context.** Se introdujo un nodo raíz `Context` compuesto de `Attribute`, sobre los que los agentes pueden leer y escribir. Encapsular los atributos dentro de `Context` en lugar de declararlos directamente facilita futuras extensiones como la persistencia. Al tratarse de una abstracción del DSL, el mecanismo es independiente del framework destino: en `LangGraph` se corresponde con el estado compartido del grafo; un concepto análogo —aunque menos estructurado— aparece en otros frameworks como `CrewAI`, donde los agentes consumen el output de tareas previas como contexto.
- **Refinamiento de Agent e introducción de Actor.** Se añadieron atributos de configuración al agente (`maxToken`, `timeOut`, `maxRetries`, `temperature`) y, dado que un agente ordinario y un coordinador producen código diferente, se introdujo la abstracción `Actor` de la que derivan `Agent` y `Coordinator`.
- **Separación de proveedor y modelo.** El proveedor se incorporó como atributo independiente, de modo que determina las dependencias incluidas en el código



generado. El modelo se define mediante una lista de valores estáticos que, aunque puede quedar desfasada, proporciona autocompletado y validación en el editor, lo cual se consideró de mayor utilidad para el diseñador.

El metamodelo resultante de esta iteración se recoge en la Figura 5.2.

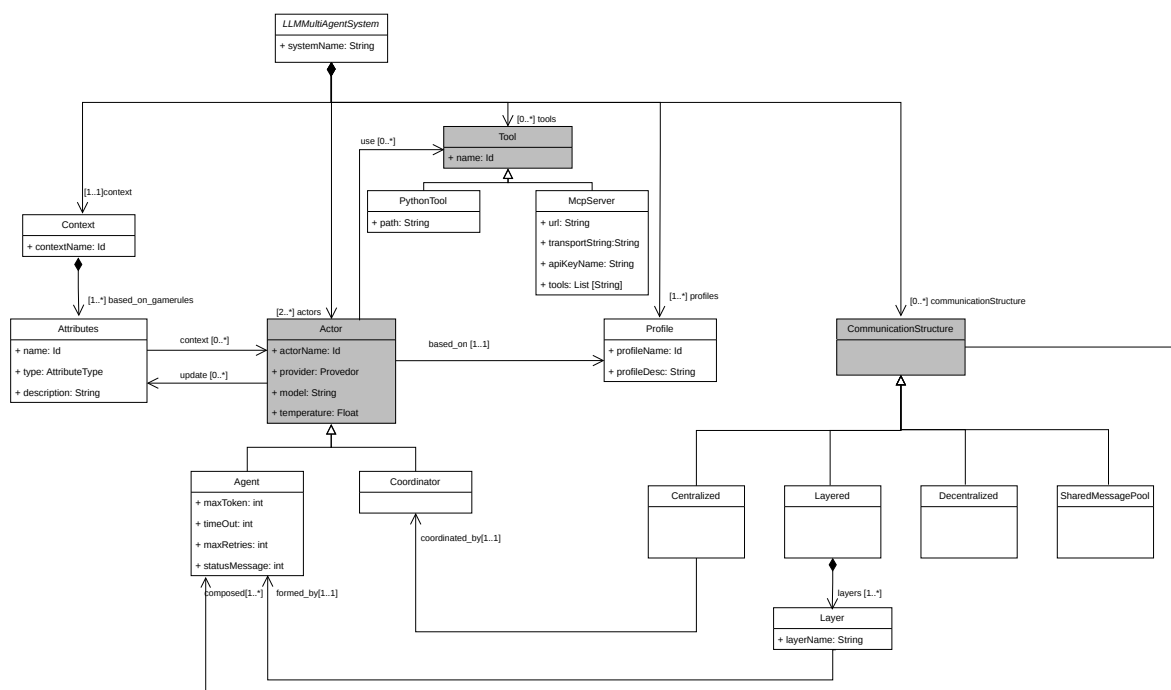


Figura 5.2: Metamodelo — Iteración 2

Iteración 3: Preparación para entornos de producción. Durante el desarrollo avanzado del generador se elaboraron varios modelos adicionales que pusieron de manifiesto las últimas carencias del metamodelo respecto a un sistema listo para producción. El modelo que mejor ilustró estas carencias fue **Customer Service**.

Este modelo representaba un sistema de atención al cliente compuesto por dos estructuras de comunicación encadenadas: una estructura **Layered** de un único agente que validaba el mensaje del usuario y extraía campos relevantes (identificador del pedido, tipo de problema, etc.), seguida de una estructura **Centralized** que enrutaba al agente más adecuado según el tipo de incidencia. La transición entre estructuras era condicional: si el mensaje no era válido o los datos eran incorrectos, el sistema respondía directamente



al usuario sin llegar a la estructura centralizada. La Figura 5.3 muestra el grafo de control resultante.

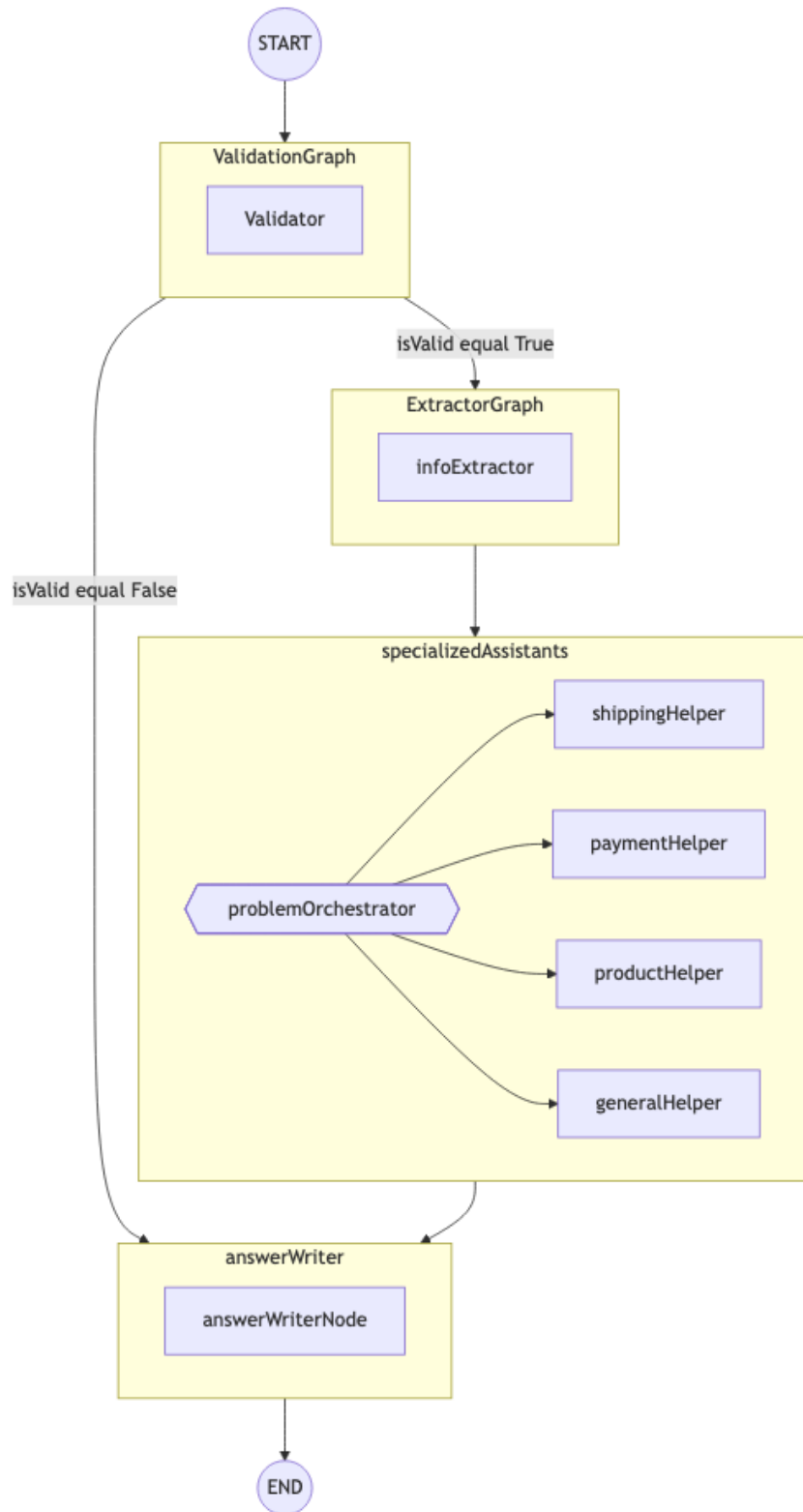


Figura 5.3: Grafo de control del sistema *Customer Service*.



Del análisis de este modelo se identificaron tres carencias:

- **Ausencia de transiciones.** El metamodelo permitía declarar varias `CommunicationStructure`, pero no ofrecía ningún mecanismo para definir el orden de ejecución entre ellas ni condiciones de transición basadas en el estado del sistema.
- **Ausencia de persistencia.** El estado de la conversación no quedaba almacenado en ningún sitio. En el ejemplo concreto, si el usuario cerraba la sesión y volvía a consultar su problema, toda la conversación habría desaparecido.
- **Gestión del historial de mensajes.** El campo de mensajes puede crecer hasta superar la ventana de contexto del modelo, especialmente en modelos de bajo coste donde conviene limitar el contexto. Los sistemas de producción suelen abordar esto mediante truncado, resumen o ambos mecanismos de forma combinada.

Para dar soporte a estas características se introdujeron los siguientes cambios:

- **Incorporación de Transition.** Se añadió una nueva clase `Transition` que parte del nodo raíz y conecta dos `CommunicationStructure` mediante referencias `from` y `to`. Opcionalmente, una transición puede incluir una condición formada por un atributo de referencia del `Context`, un tipo de comparación (`ConditionType`: igual, mayor o menor que) y un valor. Si existe condición, ambos campos deben estar definidos; si no, ninguno lo estará — restricción que queda a cargo del validador semántico.
- **Incorporación de Persistence.** Se añadió un atributo `persistence` de tipo `PersistenceType` al `Context`, consistente en una enumeración estática con los mecanismos soportados: `inMemorySave`, `mongoSave` y `postgresSave`. El conjunto se limitó deliberadamente para mantener acotado el alcance de las pruebas.
- **Incorporación de Summarizer y maxMessages.** Para la gestión del historial se introdujeron dos mecanismos independientes y combinables: un atributo entero

opcional `maxMessages` en el `Context` que establece el límite de mensajes antes del truncado, y un nuevo subtipo de `Actor` denominado `Summarizer`, con un atributo `tokenTrigger` que indica el umbral de tokens a partir del cual se activa el resumen del historial. El sistema puede operar con ambos mecanismos, con uno solo o con ninguno.

El metamodelo resultante de esta iteración se recoge en la Figura 5.4.

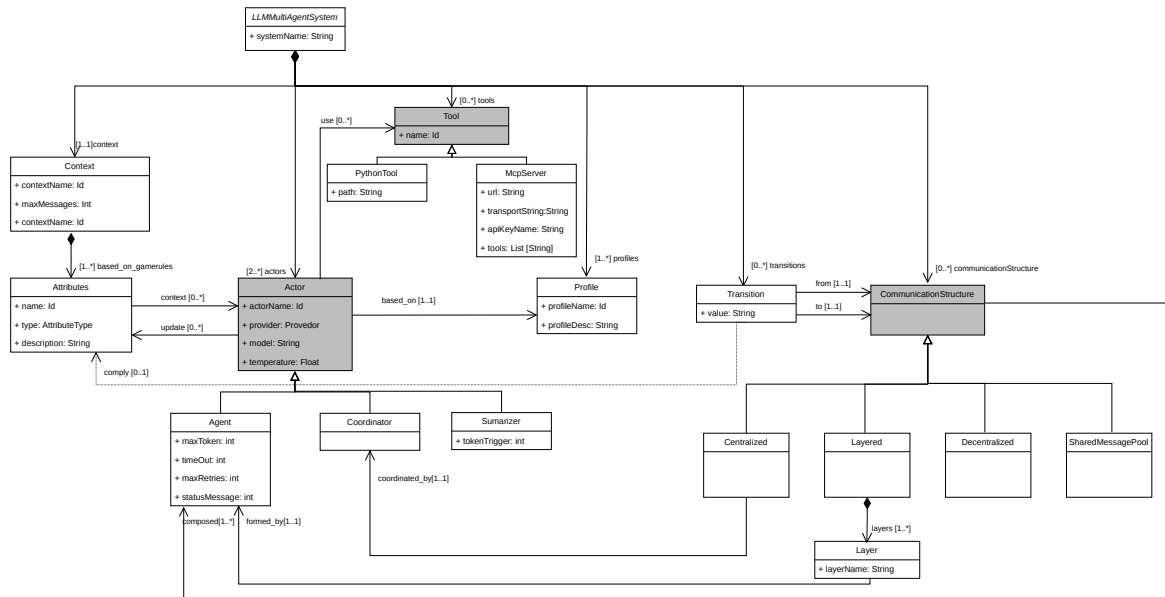


Figura 5.4: Metamodelo — Iteración 3

5.1.2. Metamodelo final

El metamodelo final del DSL, recogido en la Figura 5.4, está compuesto por las entidades que se describen a continuación, agrupadas por responsabilidad.

LLMMultiAgentSystem. Es la entidad raíz del modelo. Actúa como contenedor del sistema y agrega los seis tipos de elementos que lo componen: `Context`, `Profile`, `Tool`, `Actor`, `CommunicationStructure` y `Transition`. El atributo `systemName` lo identifica de forma única.



Context y Attribute. `Context` declara el estado compartido del sistema — el único canal de información que los agentes tienen para leer y escribir entre sí. Está compuesto por cero o más `Attribute`, cada uno con un nombre (`name`), un tipo primitivo (`type`: `int`, `string` o `boolean`) y una descripción obligatoria (`description`) que el generador utiliza como metadato en los esquemas Pydantic generados. Adicionalmente, `Context` incluye dos atributos opcionales de gestión del historial: `maxMessages`, que activa el truncado conservando solo los últimos N mensajes, y `persistence`, una enumeración estática (`inMemorySave`, `postgresSave`, `mongoSave`) que determina el mecanismo de persistencia del estado.

Profile. Define el prompt de sistema de un actor. Es un elemento de primer nivel reutilizable: varios actores pueden referenciar el mismo perfil sin duplicar su definición. Contiene un identificador (`name`) y el texto del prompt (`profileDescription`), declarado como cadena multilínea.

Actor, Agent, Coordinator y Summarizer. `Actor` es la abstracción base de todos los participantes del grafo. Agrupa los atributos comunes: `name`, `provider` (proveedor LLM: `openai`, `anthropic`, `ollama` o `google_genai`), `model` (identificador del modelo concreto), una referencia opcional a `Profile` y `temperature`. El perfil se declara como opcional a nivel de `Actor` porque `Summarizer` no debe tener perfil asignado (restricción R14); para `Agent` y `Coordinator`, su presencia se recomienda y el validador emite una advertencia si se omite. El proveedor se modela como enumeración estática porque condiciona directamente los imports y las variables de entorno del proyecto generado.

De `Actor` derivan tres subtipos:

- **Agent:** el nodo genérico del grafo. Incorpora atributos de configuración (`maxToken`, `timeOut`, `maxRetries`, `statusMessage`) y referencias al estado: `stateContext` indica los atributos que el agente lee —el generador los inyecta en su prompt en cada invocación— y `stateUpdate` indica los que escribe —el generador construye con ellos un esquema `BaseModel` de salida estructurada—. También puede referenciar herramientas mediante `tools`.



- **Coordinator**: actor especializado que actúa como orquestador en una estructura **Centralized**. Solo decide a qué agente delegar en cada turno; no resuelve problemas por sí mismo ni tiene referencias al estado ni herramientas propias. Sus atributos son los mínimos heredados de **Actor**.
- **Summarizer**: actor especializado en comprimir el historial de mensajes. Cuando el historial supera el umbral definido por **tokenTrigger**, este nodo genera un resumen y lo almacena en el estado. No puede tener perfil asignado (restricción R14), dado que su comportamiento está completamente determinado por el generador.

PythonTool y McpServer. **Tool** es la abstracción base de las herramientas. De ella derivan dos subtipos. **PythonTool** referencia una función Python local del proyecto mediante su nombre y la ruta al módulo (**modulePath**); el generador produce el import correspondiente. **McpServer** declara la URL de un servidor MCP remoto, el protocolo de transporte (**transport**), una clave API opcional (**apiKeyName**) y la lista de nombres de herramientas concretas a exponer (**tools**); el generador produce un cliente **MultiServerMCPClient** con esta configuración.

CommunicationStructure, Layered, Centralized, Decentralized y SharedMessagePool. **CommunicationStructure** es la abstracción base de las estructuras de comunicación. Los cuatro subtipos del DSL se inspiran en las topologías de comunicación identificadas en [18], en una interpretación simplificada de cada una:

- **Layered**: pipeline secuencial. Los agentes se ejecutan en el orden definido por las capas (**Layer**), cada una ocupada por exactamente un agente. La salida de una capa sirve de entrada a la siguiente, sin bifurcaciones.
- **Centralized**: un **Coordinator** implementado como nodo LLM recibe el contexto acumulado del sistema y, en cada turno, decide de forma autónoma a qué agente especializado delegar la ejecución. No hay lógica de enrutamiento codificada: la decisión emerge del propio modelo.



- **Decentralized:** cada agente está conectado al resto de agentes de la estructura. Tras ejecutarse, cada agente devuelve un `Command` de `LangGraph` indicando explícitamente el siguiente nodo, sin ningún coordinador central.
- **SharedMessagePool:** todos los agentes comparten un pool de mensajes común. Esta estructura está contemplada en el metamodelo pero pendiente de implementación en el generador (véase el Capítulo ??).

Transition y Condition. `Transition` define una arista entre dos `CommunicationStructure` a nivel de sistema, expresando el flujo de control del grafo. Contiene una referencia de origen (`from`, que puede ser `START`) y una de destino (`to`, que puede ser `END`). Opcionalmente, puede incluir una `Condition` formada por una referencia a un `Attribute` del estado, un operador de comparación (`equal`, `greater` o `lower`) y un valor literal. Cuando existe condición, todos sus campos son obligatorios; cuando no existe, la transición se considera incondicional — invariante verificada por el validador semántico (restricciones R03–R07).

5.1.3. Sintaxis concreta

Definidos en el metamodelo los conceptos del lenguaje y sus relaciones, queda por determinar cómo se materializan en una notación textual concreta. Los ficheros del DSL llevan la extensión `.mad` (*Multi-Agent DSL*). Se ha optado por una sintaxis basada en palabras clave y bloques delimitados por llaves, en la línea de TypeScript o JSON, con la intención de resultar familiar tanto para programadores como para usuarios menos habituados al ecosistema MDD.

Las principales decisiones sintácticas son:

- **Modelo único por fichero.** Cada `.mad` declara exactamente un modelo, encerrado entre llaves. No existen mecanismos de *import* ni espacios de nombres.
- **Orden fijo en el bloque raíz.** Los elementos deben aparecer en el orden `Context` → `Profile` → herramientas → actores → estructuras de comunicación



→ transiciones. Este orden refleja sus dependencias —un actor referencia un perfil previamente declarado, una estructura referencia actores ya declarados— y permite leer el modelo de arriba a abajo sin saltos.

- **Referencias cruzadas por nombre.** Los elementos se referencian por su identificador, sin tipos cualificados ni rutas. El validador semántico se encarga de resolverlas.
- **Asignación sin operador.** Los atributos se escriben como `clave valor` (p. ej. `model "gpt-4o"`), sin signo `=`, reduciendo el ruido visual.
- **Listas separadas por comas.** Los campos que admiten varios valores —`stateContext`, `stateUpdate`, `tools`, `agents`— usan comas como separador en lugar de delimitadores adicionales.
- **Comentarios al estilo C.** Se admite `//` para línea y `/* ...*/` para bloque.

A continuación se muestran las construcciones principales del lenguaje, una vez ya descrito su significado en el metamodelo, prestando atención únicamente a la forma textual y a las particularidades sintácticas de cada una.

Contexto. Bloque clásico de `clave valor` en su interior, con la declaración de persistencia al final:

```
context customerService {  
  attribute orderId type string description "Id del pedido con problemas."  
  attribute paymentAmount type int description "Precio del producto."  
  attribute isValid type boolean description "Si el mensaje del cliente es valido."  
  
  persistence inMemorySave  
}
```

Perfiles. A diferencia del resto de elementos, `profile` prescinde de las llaves: tras la palabra clave `description` se escribe directamente la cadena del prompt. Esta forma se



eligió porque el contenido habitual es un texto largo, normalmente multilínea, y un bloque adicional aportaría ruido.

```
profile ResearcherPrompt description
"Eres un agente investigador. Tu unica responsabilidad es buscar
informacion actualizada y relevante sobre el tema indicado por el usuario..."
```

Herramientas. Una `pythonTool` se declara en una sola línea —solo necesita nombre y ruta de módulo—, mientras que un `mcpServer` usa el bloque con llaves para acomodar sus múltiples atributos:

```
pythonTool orderChecker modulePath "validator"

mcpServer Tavily {
  url "https://mcp.tavily.com/mcp/?tavilyApiKey={key}"
  transport "streamable_http"
  apiKeyName "TAVILY_API_KEY"
  tools "tavily_search", "tavily_extract"
}
```

Actores. El bloque de un `agent` combina atributos simples con listas de referencias a atributos del contexto y herramientas, todas separadas por comas:

```
agent Researcher {
  provider openai
  model "gpt-4o"
  profile ResearcherPrompt
  description "Agente investigador que busca informacion con Tavily."
  stateUpdate searchSummary, sources
  tools Tavily
}
```

Los `coordinator` y `summarizer` siguen la misma estructura sintáctica, omitiendo los campos no aplicables a su rol.



Estructuras de comunicación. Cada estructura tiene su propia palabra clave (`layered`, `centralized`, `decentralized`, `shared_message_pool`) seguida del bloque con sus contenidos específicos. En `layered` cada agente que participa requiere su propia declaración `layer`, y la cláusula opcional `next` establece únicamente el encadenamiento entre ellas; por ese motivo un mismo agente puede aparecer en una línea como destino de un `next` y, en otra línea aparte, como su propia capa. En `centralized` se declara primero el coordinador y luego la lista de agentes:

```
layered ResearchPipeline {  
    layer Researcher next Writer  
    layer Writer  
}  
  
centralized specializedAssistants {  
    coordinator problemOrchestrator  
    agents shippingHelper, paymentHelper, productHelper, generalHelper  
}
```

Transiciones. Las transiciones se declaran como líneas sueltas a nivel del modelo, no como un bloque colectivo. La sintaxis es `from origen to destino`, con la cláusula opcional `when condición`. Las palabras reservadas `START` y `END` sustituyen a las referencias en las transiciones extremas:

```
from START to ValidationGraph  
from ValidationGraph to answerWriter when isValid equal False  
from ValidationGraph to ExtractorGraph when isValid equal True  
from ExtractorGraph to specializedAssistants  
from specializedAssistants to answerWriter  
from answerWriter to END
```

Ejemplo completo. La Figura 5.5 reúne todas las construcciones anteriores en un modelo `.mad` completo correspondiente al sistema *Research Assistant* introducido en la Iteración 2. Los *prompts* de los perfiles se han recortado por brevedad.



```
{
  context researchAssistant {
    attribute topic type string description "Tema sobre el que el usuario
    quiere investigar."
    attribute searchSummary type string description "Resumen estructurado
    de los hallazgos."
    attribute sources type string description "Lista de URLs y titulos
    consultados."
    attribute report type string description "Informe final redactado."

    persistence inMemorySave
  }

  profile ResearcherPrompt description
  "Eres un agente investigador. Busca informacion actualizada sobre el tema [...]"

  profile WriterPrompt description
  "Eres un agente redactor. Redacta un informe a partir de searchSummary [...]"

  mcpServer Tavily {
    url "https://mcp.tavily.com/mcp/?tavilyApiKey={key}"
    transport "streamable_http"
    apiKeyName "TAVILY_API_KEY"
    tools "tavily_search", "tavily_extract"
  }

  agent Researcher {
    provider openai
    model "gpt-4o"
    profile ResearcherPrompt
    description "Busca informacion en la web con Tavily."
    stateUpdate searchSummary, sources
    tools Tavily
  }
}
```



```

agent Writer {
  provider openai
  model "gpt-4o"
  profile WriterPrompt
  description "Elabora el informe final a partir de los hallazgos."
  stateContext topic, searchSummary, sources
  stateUpdate report
}

layered ResearchPipeline {
  layer Writer
  layer Researcher next Writer
}

from START to ResearchPipeline
from ResearchPipeline to END
}

```

Figura 5.5: Modelo .mad completo del sistema *Research Assistant*.

5.1.4. Restricciones semánticas

La gramática del DSL captura todas las reglas que pueden expresarse de forma sintáctica, pero no es suficiente para garantizar que un modelo bien formado a nivel léxico sea también coherente. Existen propiedades que solo pueden comprobarse sobre el árbol de sintaxis abstracta una vez analizado el conjunto del modelo, como la unicidad de nombres, la conectividad del grafo o la compatibilidad de tipos en las condiciones. Estas comprobaciones constituyen las *restricciones semánticas* del lenguaje, y se aplican tras el parseo y antes de delegar el modelo al generador, de manera que cualquier fichero .mad que llegue a la fase de generación queda garantizado como semánticamente coherente.

La Tabla 5.1 resume las quince restricciones (R01–R15) que conforman el sistema de validación. Todas ellas se notifican como **errores** y bloquean la generación de código hasta que el usuario las resuelva. A continuación se describen agrupadas según su naturaleza.



Código Restricción		Nodo objetivo
R01	Estructuras de comunicación conectadas	LLMMultiAgentSystem
R02	URL única por servidor MCP	McpServer
R03	Existe una única transición START que no conecta directamente con END	Transition
R04	Sin arcos duplicados con condición	Transition
R05	Las transiciones únicas no pueden tener condición	Transition
R06	Compatibilidad de tipos en condiciones	Condition
R07	Una sola transición sin condición por origen	Transition
R08	Temperatura entre 0.0 y 1.0	Actor
R09	Un único Summarizer por sistema	Summarizer
R10	Valores positivos en campos numéricos	Agent, Summarizer, Context
R11	Estructura Layered sin ciclos	Layered
R12	apiKeyName única por servidor MCP	McpServer
R13	Nombres únicos en todo el sistema	Cualquier elemento con name
R14	El Summarizer no tiene perfil asignado	Summarizer
R15	Un agente pertenece a una sola estructura	Agent

Tabla 5.1: Resumen de las restricciones semánticas del DSL.

Integridad del grafo de comunicación. Las restricciones **R01**, **R03**, **R04**, **R05**, **R07** y **R11** protegen la coherencia del grafo formado por las estructuras de comunicación y las transiciones que las conectan. **R01** verifica que toda estructura declarada esté conectada a alguna transición, evitando estructuras inalcanzables que ensucian el modelo sin aportar nada al sistema. **R03** asegura que existe exactamente una transición desde **START** —si no existiera, el grafo no tendría punto de entrada definido; si hubiera varias, la



ambigüedad sería irresoluble— y, además, prohíbe que esa transición vaya directamente a **END**, porque un sistema sin estructuras intermedias carece de utilidad. **R04** impide arcos duplicados con condición entre el mismo par origen-destino, dado que serían semánticamente equivalentes y producirían comportamiento no determinista. **R05** prohíbe que una transición sea la única salida de un nodo y a la vez tenga condición: la condición carecería de sentido al no existir un camino alternativo cuando no se cumpliera. **R07** relaja la regla anterior permitiendo varias transiciones con condición desde un mismo origen junto a una única transición sin condición, lo que permite expresar un comportamiento de *router* con cláusula *else*. Por último, **R11** comprueba que dentro de un bloque **Layered** el encadenamiento mediante **next** no forme ciclos, ya que una capa que apuntara a una anterior produciría un bucle infinito en tiempo de ejecución y contradiría la naturaleza secuencial de esta estructura.

Unicidad e identidad. Las restricciones **R02**, **R09**, **R12** y **R13** garantizan que determinados elementos sean únicos en su categoría. **R02** y **R12** impiden, respectivamente, que dos servidores MCP compartan URL o nombre de clave de API, evitando colisiones en los clientes generados. **R09** permite un único **Summarizer** por sistema, dado que el resumen del historial es una operación global que solo tiene sentido aplicada una vez. **R13** impone unicidad de nombres sobre todo el espacio del sistema —contexto, atributos, perfiles, herramientas, actores y estructuras—. Esta última restricción es especialmente importante porque Langium, por defecto, admite identificadores duplicados y resuelve las referencias a la primera ocurrencia encontrada de forma silenciosa, lo que produciría código generado con identificadores colisionantes sin que el usuario fuera advertido.

Tipado y dominio de valores. Las restricciones **R06**, **R08** y **R10** acotan los tipos y rangos de los valores literales del modelo. **R06** comprueba que, en una condición de transición, el tipo del literal coincida con el tipo declarado del atributo referenciado: comparar un atributo **int** con un literal **string**, o viceversa, produciría una condición no evaluable en tiempo de ejecución. **R08** acota la temperatura de los actores al intervalo



[0,0,1,0], el rango comúnmente aceptado por la mayoría de proveedores LLM. **R10** exige que los campos numéricos de configuración (`maxToken`, `timeOut`, `maxRetries`, `tokenTrigger` y `maxMessages`) tomen valores estrictamente positivos, dado que cero o valores negativos carecen de interpretación válida.

Restricciones específicas sobre actores y estructuras. **R14** impide que un `Summarizer` tenga perfil asignado: su comportamiento está completamente determinado por el generador, y un perfil personalizado se ignoraría silenciosamente. **R15** verifica que cada agente pertenezca a una sola estructura de comunicación. El generador produce código distinto para un agente según la estructura que lo contiene —capas con `add_edge` en `Layered`, devolución de `Command` en `Decentralized`, etc.—, por lo que la pertenencia simultánea a varias estructuras resultaría incompatible con el modelo de generación.

Advertencias adicionales. Más allá de los errores duros, el validador emite también una serie de advertencias (*warnings*) que no bloquean la generación pero alertan al usuario de configuraciones potencialmente problemáticas: el uso de Ollama junto con herramientas, cuyo soporte depende del modelo concreto cargado localmente; el uso de Google Gemini en estructuras `Centralized` o `Decentralized`, donde se ha observado empíricamente que tiende a producir bucles infinitos; y la utilización de identificadores de modelo no contemplados en la lista curada por proveedor. Estos avisos pretenden orientar al usuario en sus decisiones de configuración sin imponer restricciones rígidas que podrían quedar obsoletas con la evolución de los proveedores.

5.2. Diseño del generador

El generador es el componente que transforma un modelo `.mad` semánticamente válido en un proyecto Python ejecutable con LangGraph. A diferencia de un generador monolítico que emitiría un único fichero, se optó por distribuir el código en varios módulos de responsabilidad delimitada: esto simplifica la lectura del código producido, facilita que el usuario retoque manualmente la parte que desee sin interferir en el resto, y satisface el



formato que LangSmith Studio requiere para visualizar el grafo.

5.2.1. Artefactos generados

El proyecto generado está organizado en tres paquetes y un conjunto de ficheros sueltos en la raíz. A continuación se describe cada uno junto con la condición que determina su generación.

- **Paquete state.** Contiene todo lo relativo al estado compartido del grafo:
 - `state.py`. Siempre generado. Define el estado del grafo como una clase `TypedDict` cuyos campos corresponden a los atributos del bloque `Context` más el historial de mensajes.
 - `reducers.py`. Generado únicamente si el modelo declara un `Summarizer` o un valor de `maxMessages`. Contiene los reductores necesarios para aplicar el resumen o el truncado del historial.
 - `__init__.py`. Siempre generado. Reexporta el estado y los reductores para que el resto de módulos puedan importarlos desde un único punto.
- **Paquete subgraph.** Contiene un fichero por cada estructura de comunicación declarada en el modelo:
 - `<nombre>.py`. Cada fichero implementa un subgrafo de `LangGraph` independiente correspondiente a la estructura. La separación permite compilar y probar cada estructura de forma aislada.
- **Paquete tools.** Contiene el código relacionado con las herramientas del modelo. Se recomienda al usuario colocar aquí también sus propias herramientas Python:
 - `mcpClients.py`. Generado solo si el modelo declara algún `McpServer`. Configura e instancia los clientes MCP y expone las herramientas seleccionadas para que los agentes las consuman.



- `agents.py`. Contiene la definición de cada agente: la cadena `LangGraph` (modelo LLM con sus herramientas) y el esquema `BaseModel` de salida estructurada que permite sobrescribir los atributos del estado.
- `prompts.py`. Reúne todos los prompts de sistema declarados como `Profile` en el modelo. Mantenerlos en un fichero propio facilita su ajuste sin necesidad de tocar el código de los agentes.
- `graph.py`. Ensambla el grafo principal: conecta los subgrafos mediante las transiciones del modelo, añade las bifurcaciones condicionales y compila el grafo con el `checkpointer`. Expone la variable `graph` que `LangSmith Studio` utiliza para localizar el grafo.
- `checkpointer.py`. Instancia el mecanismo de persistencia correspondiente al valor de `persistence` del `Context`. Si no se ha declarado persistencia, no se genera.
- `config.py`. Centraliza la carga de variables de entorno del fichero `.env` y las expone como variables con nombre. Evita que cada módulo cargue el paquete `os` y acceda directamente al entorno.
- `.env`. Plantilla con todas las claves de API y valores de configuración que el usuario debe completar antes de ejecutar el sistema. El generador deduce su contenido a partir de los proveedores y servidores MCP del modelo.
- `langgraph.json`. Fichero de configuración de `LangSmith Studio`. Indica la ruta al módulo `graph.py` y la variable que expone el grafo compilado, lo que permite visualizarlo e interactuar con él desde la interfaz de `Studio`.
- `requirements.txt`. Lista las dependencias Python necesarias, deducidas del conjunto de proveedores y herramientas presentes en el modelo. Permite instalar el entorno completo con un único comando (`pip install -r requirements.txt`).



5.2.2. Mapeo del modelo a LangGraph

Descritos los artefactos que produce el generador, queda explicar cómo se traduce cada elemento del metamodelo a constructos concretos de LangGraph y en qué fichero acaba materializándose. La Tabla 5.2 recoge el mapeo completo; los párrafos posteriores entran en los casos cuya traducción no es directa.

Elemento del DSL	Constructo / fichero generado
<code>Context</code>	No emite código por sí solo; actúa como contenedor de <code>Attribute</code> , <code>maxMessages</code> y <code>persistence</code> .
<code>Context.maxMessages</code>	Reductor de truncado del historial en <code>state/reducers.py</code> .
<code>Context.persistence</code>	Checkpointeer correspondiente al valor declarado, instanciado en <code>checkpointer.py</code> .
<code>Attribute</code>	Campo de la clase del estado en <code>state/state.py</code> .
<code>Attribute.description</code>	Anotación del campo equivalente en el esquema de salida estructurada en <code>agents.py</code> .
<code>Profile</code>	Constante de tipo <code>str</code> en <code>prompts.py</code> .
<code>Actor.profile</code> (<code>based_on</code>)	Referencia a la constante anterior en el cuerpo del nodo.
<code>Agent</code>	Función de nodo en <code>agents.py</code> , con cuerpo dependiente de la estructura que lo contiene.
<code>Agent.stateContext</code>	Valores del estado inyectados en el prompt de entrada del nodo.
<code>Agent.stateUpdate</code>	Esquema de salida estructurada en <code>agents.py</code> .
<code>Coordinator</code>	Nodo supervisor con plantilla fija, emitido directamente en el fichero del subgrafo <code>Centralized</code> al que pertenece.



Elemento del DSL	Constructo / fichero generado
Summarizer	Reductor del estado en <code>state/reducers.py</code> ; no se materializa como nodo del grafo.
PythonTool	Import del módulo en <code>agents.py</code> y <code>bind_tools</code> al agente que la declara.
McpServer	Cliente MCP en <code>tools/mcpClients.py</code> y <code>bind_tools</code> al agente que lo declara.
Layered	Subgrafo en <code>subgraph/<nombre>.py</code> con encadenamiento lineal mediante <code>add_edge</code> .
Centralized	Subgrafo en <code>subgraph/<nombre>.py</code> con coordinador supervisor y agentes hoja que devuelven <code>Command</code> .
Decentralized	Subgrafo en <code>subgraph/<nombre>.py</code> en el que cada agente decide el siguiente mediante <code>Command(goto=...)</code> .
Transition (incondicional)	<code>add_edge</code> entre subgrafos en <code>graph.py</code> .
Transition (condicional)	<code>add_conditional_edges</code> con función de enrutamiento generada en <code>graph.py</code> .

Tabla 5.2: Mapeo de los elementos del DSL a constructos LangGraph.

El cuerpo del nodo Agent varía con la estructura que lo contiene. La traducción de un `Agent` a código no es uniforme: depende del tipo de `CommunicationStructure` en la que se ha declarado. En `Layered` el agente se limita a escribir su parte del estado, y la transición al siguiente nodo queda resuelta por el `add_edge` del subgrafo. En `Centralized` el agente actúa como nodo hoja: tras su ejecución devuelve un `Command` que retorna el control al coordinador. En `Decentralized` cada agente devuelve un `Command` que indica explícitamente el siguiente nodo, prescindiendo de coordinador. Adicionalmente, si el agente declara `tools`, estas se vinculan al modelo del nodo para que el LLM pueda



invocarlas durante su ejecución. Esta variabilidad explica que **Agent** no se traduzca a una plantilla fija, sino que requiera lógica de generación condicionada al contexto del modelo.

El coordinador se emite junto a su estructura. A diferencia del resto de actores, cuyo código vive en `agents.py`, el **Coordinator** se emite directamente en el fichero del subgrafo **Centralized** al que pertenece. Esta excepción se justifica por dos motivos. Primero, la lógica del coordinador es invariable salvo por la lista concreta de agentes a los que delega: recibe el estado, consulta al LLM con un prompt estructurado que enumera los agentes disponibles y devuelve un **Command** con el identificador del elegido. Segundo, la lista de agentes solo es conocida en el contexto de la estructura; mantener el coordinador junto a esa estructura evita acoplamientos innecesarios entre `agents.py` y los subgrafos.

El Summarizer se materializa como reductor, no como nodo. A diferencia de **Agent** y **Coordinator**, el **Summarizer** no se traduce a un nodo del grafo, sino a un reductor del campo de mensajes en `state/reducers.py`. La razón es que el resumen no es una operación cuya activación deba decidirse desde el flujo de control: debe ejecutarse de forma transversal cada vez que el historial supera el umbral `tokenTrigger`. Modelarlo como reductor garantiza esta automaticidad y evita obligar al diseñador a intercalar el nodo en cada bifurcación del grafo. Es también el motivo por el que el metamodelo limita el sistema a un único **Summarizer** (restricción R09): un reductor del estado por definición es global.

Las condiciones de transición se agrupan por origen. Una **Transition** sin condición se traduce directamente a un `add_edge(origen, destino)` en `graph.py`. Cuando una o varias transiciones que parten del mismo origen incluyen condición, todas ellas se agrupan en una única llamada a `add_conditional_edges`, acompañada de una función de enrutamiento generada que evalúa los predicados sobre el estado y devuelve el identificador del destino correspondiente. Esta agrupación es obligatoria porque `LangGraph` exige un único conmutador de salida por nodo, no múltiples aristas



condicionales independientes; la restricción R07 del validador semántico se diseñó precisamente para cubrir el caso en el que esa función debe contemplar también una rama *else* sin condición.

5.3. Diseño de la extensión

El diseño de la extensión se articula en torno a las capacidades que Langium proporciona por defecto y las que se decidió añadir a partir de la experiencia de uso durante el desarrollo de los modelos de prueba.

5.3.1. Funcionalidades base de Langium

Langium genera automáticamente una extensión de VSCode a partir de la gramática del lenguaje, con una serie de capacidades básicas ya operativas. Para determinar qué funcionalidades adicionales debían desarrollarse, se siguió un proceso de uso iterativo: se elaboraron distintos modelos y se identificaron los aspectos que dificultaban el trabajo del diseñador. La Tabla 5.3 recoge las capacidades disponibles sin ningún desarrollo adicional.

Función	Descripción
Infraestructura LSP	Proceso LSP completo para comunicarse con VSCode mediante IPC! (IPC!).
Parser	Módulo de análisis sintáctico generado automáticamente a partir de la gramática.
Validación automática	El validador comprueba continuamente si el modelo es sintácticamente correcto y notifica los errores detectados en tiempo real.
Resaltado básico	El editor resalta algunas construcciones del lenguaje, como las cadenas de texto o los nombres de clase del metamodelo.



Función	Descripción
Autocompletado base	El editor propone posibles coincidencias para las referencias cruzadas del modelo.

Tabla 5.3: Características base aportadas por Langium.

Del uso de esta extensión base se detectaron las siguientes limitaciones:

- **Resaltado inconsistente.** El esquema de colores proporcionado no distinguía con claridad entre categorías distintas: las palabras clave del lenguaje y los identificadores introducidos por el usuario recibían el mismo tratamiento visual.
- **Autocompletado insuficiente.** El autocompletado no resolvía referencias cruzadas, sino que proponía todas las palabras escritas hasta ese momento en el modelo. El diseñador acababa buscando manualmente en el fichero el nombre del elemento que quería referenciar.
- **Guía insuficiente para nuevos usuarios.** Un diseñador sin conocimiento previo del metamodelo no disponía de ayuda para saber qué atributos admite cada construcción ni cuáles son opcionales.
- **Dependencia de la terminal para generar código.** El flujo de generación exigía ejecutar un comando en la terminal, lo que resultaba tedioso en ciclos de desarrollo iterativos.
- **Ausencia de representación visual.** El diseñador no disponía de ningún mecanismo para comprobar rápidamente la estructura global del modelo que estaba definiendo.

5.3.2. Funcionalidades implementadas

A partir de las limitaciones detectadas se incorporaron las funcionalidades recogidas en la Tabla 5.4.



Función	Descripción
Resaltado consistente	Esquema de colores coherente: las palabras clave del lenguaje se muestran en azul y los valores introducidos por el usuario en blanco, excepto cadenas de texto y valores enumerados, que adoptan su propio color.
Autocompletado mejorado	El editor propone las palabras clave válidas en cada posición del cursor y las referencias existentes en los contextos que las admiten.
Hover documentado	Al situar el cursor sobre una primitiva del lenguaje, el editor muestra todos sus atributos con indicación de cuáles son obligatorios u opcionales y una descripción del propósito de cada uno.
Snippets y templates	El diseñador dispone de plantillas de código para cada construcción del lenguaje. Escribiendo <code>template</code> se ofrece un modelo orientativo completo; para cada elemento individual (agente, coordinador, servidor MCP, etc.) existe un snippet con todos sus campos ya estructurados.
Autocompletado de herramientas MCP	Al declarar un <code>McpServer</code> , el editor sugiere las herramientas más comunes de los servidores MCP de uso habitual (como <code>Tavily</code>), evitando que el diseñador tenga que consultar la documentación del servidor para conocer sus nombres exactos.
Botón de generación	Botón en el editor que invoca el generador sin necesidad de recurrir a la terminal.



Función	Descripción
Diagrama del grafo	Panel lateral que muestra el grafo del modelo actual con sus estructuras de comunicación, transiciones y condiciones, actualizado automáticamente al guardar el fichero.

Tabla 5.4: Funcionalidades implementadas en la extensión.

Capítulo 6

Implementación y desarrollo

Contenidos

6.1. Implementación del DSL	78
6.1.1. Gramática	79
6.1.2. Validación semántica	92
6.2. Implementación del generador	95
6.2.1. Organización modular	95
6.2.2. Generación del estado	98
6.2.3. Generación de agentes y prompts	101
6.2.4. Generación de herramientas	106
6.2.5. Generación del grafo	108
6.2.6. Generación del soporte para producción	114
6.3. Implementación de la extensión	117
6.3.1. Resaltado y autocompletado	117
6.3.2. Ayudas al usuario	119
6.3.3. Autocompletado de herramientas MCP	120
6.3.4. Comando de generación	120
6.3.5. Diagrama del grafo	121



En este capítulo se abordan los aspectos más concretos de la implementación del sistema descrito en el capítulo anterior. El capítulo se divide en tres secciones, una por cada componente del sistema.

6.1. Implementación del DSL

Esta sección describe los aspectos más relevantes de la implementación del DSL sobre Langium. El código fuente del paquete de lenguaje se organiza en el directorio `src/` de la siguiente forma:

```
src/
├── generated/    generado por Langium, no modificar
├── index.ts
├── models.ts
├── multi-agent-dsl-completion-provider.ts
├── multi-agent-dsl-hover-provider.ts
├── multi-agent-dsl-module.ts    configuración e inyección de servicios
├── multi-agent-dsl-scope-provider.ts
├── multi-agent-dsl-validator.ts
└── multi-agent-dsl.langium
```

Figura 6.1: Estructura del paquete de lenguaje (`src/`).

El fichero central es `multi-agent-dsl.langium`, que define la gramática completa del lenguaje; Langium lo procesa para generar automáticamente el contenido de `generated/`, que no debe modificarse manualmente. El fichero `index.ts` actúa como punto de entrada público del paquete y tampoco requiere modificaciones. `multi-agent-dsl-validator.ts`, `multi-agent-dsl-module.ts` y `models.ts` implementan la lógica del lenguaje y se describen en las subsecciones siguientes.



Los ficheros `completion-provider`, `hover-provider` y `scope-provider`, aunque residen en este paquete por la arquitectura de Langium, son servicios orientados al editor y se tratan en la Sección 6.3.

6.1.1. Gramática

Si bien la forma habitual de definir la sintaxis abstracta de un DSL es mediante un metamodelo —frecuentemente expresado como un diagrama de clases UML, como es el caso de este trabajo—, Langium no utiliza UML para el metamodelo, a diferencia de herramientas como Eclipse con EMF.

En Langium, el metamodelo y la gramática se definen conjuntamente en el mismo fichero. Esto plantea una dificultad concreta: Langium no dispone de un mecanismo único y directo para representar clases abstractas cuyos subtipos hereden atributos.

Subclases y atributos heredados en Langium

El DSL contiene varias jerarquías de subtipos: la jerarquía de actores (`Agent`, `Coordinator`, `Summarizer`), la de herramientas (`PythonTool`, `McpServer`) y la de estructuras de comunicación (`Layered`, `Centralized`, `Decentralized`). En todos los casos se planteó el mismo problema: cómo representar en Langium clases base con atributos compartidos entre subtipos. Las alternativas que se describen a continuación se ilustran con la jerarquía de actores por ser la más completa, pero la solución adoptada se aplica a todas ellas:

- **Interface:** En Langium, una `interface` es una declaración de tipo TypeScript que define un contrato estructural: sus atributos no son gramática, sino una especificación de qué propiedades debe tener el nodo resultante en el AST. A priori parece la forma más natural de implementar herencia de atributos: se define una interfaz con los atributos compartidos y cada clase concreta usa `implements`. El principal problema es que, a pesar del uso de `implements`, los atributos **no se heredan ni se inyectan en la gramática**: deben redefinirse explícitamente en



cada regla hija. Esto genera acoplamiento, ya que cualquier modificación de un atributo compartido obliga a actualizarlo en cada clase por separado:

```
interface ActorBase {
  name: string
  provider: string
  model: string
  profile?: @Profile
  temperature?: number
}

Agent implements ActorBase:
  'agent' name=ID '{'
    ProviderModel
    ('profile' profile=[Profile])?
    ('temperature' temperature=FLOAT)?
    ('tools' tools+=[Tool:ID] (',' tools+=[Tool:ID]))*)?
  '};

// Coordinator y Summarizer implementan ActorBase de igual forma,
// redefiniendo name, provider, model, profile y temperature en cada regla.
```

- **Fragment + Type:** Otra alternativa es combinar las primitivas `fragment` y `type`. Un `fragment` es un bloque de gramática reutilizable que no genera un nodo propio en el AST, sino que incrusta sus atributos directamente en el nodo de la regla que lo incluye. Por su parte, `type` es un alias de unión de tipos TypeScript puro, sin impacto en el *parsing*. Combinados, `fragment` define los atributos compartidos una única vez y cada clase concreta lo incluye, eliminando el acoplamiento de la opción anterior. `type` permite agrupar las clases concretas bajo un tipo común para usarlo en referencias cruzadas o comprobaciones de tipo en el validador, aunque su aportación es limitada: únicamente resulta útil cuando se necesita determinar si un nodo pertenece a un conjunto de clases concretas, algo que en muchos casos puede resolverse con una unión explícita en la propia regla:



```

type Actor = Agent | Summarizer | Coordinator;

fragment ActorBase:
    name=ID '{'
        ProviderModel
        ('profile' profile=[Profile])?
        ('temperature' temperature=FLOAT)?;

Agent:
    'agent' ActorBase
        ('tools' tools+=[Tool:ID] (',' tools+=[Tool:ID]))*?
    '};

// Coordinator y Summarizer incluyen ActorBase directamente,
// sin redefinir ninguno de sus atributos compartidos.

```

- **Metamodelo Eclipse + EMF:** La última alternativa consiste en definir el metamodelo con Eclipse + EMF directamente en Ecore, lo que se corresponde con el diseño UML del metamodelo. A diferencia de las opciones anteriores, esta alternativa **sí resolvería** el problema de la herencia de atributos, ya que Ecore permite herencia real entre clases mediante `eSuperTypes`, y el fichero `.langium` sería generado automáticamente a partir del metamodelo, delegando en el generador las decisiones sobre `interface`, `fragment` y `type`. El principal inconveniente es la complejidad añadida: requiere mantener Eclipse + EMF como dependencia del proyecto y sincronizar el metamodelo con la gramática generada ante cada cambio.

Finalmente, se optó por `Fragment + Type`. La opción `Interface` quedó descartada dado que el acoplamiento resultante era excesivo: funcionalidades transversales como el autocompletado de proveedor y modelo habrían debido implementarse tres veces —una por cada subtipo de `ActorBase`— a pesar del uso de `implements`, puesto que los atributos no se heredan en la gramática. Con `fragment`, en cambio, dicha lógica se implementa una única vez, al incrustarse los atributos directamente en el nodo correspondiente del AST. La



alternativa Eclipse + EMF habría resuelto la herencia de forma nativa, pero introducía un coste de mantenimiento adicional —sincronizar el modelo Ecore con el fichero `.langium` ante cada cambio— que no se consideró justificado para el alcance de este trabajo.

Modelos y proveedores

Los modelos de LLM cambian con frecuencia y, al definirse como cadenas de texto libres, el usuario no dispone de orientación sobre qué valores son válidos sin consultar la documentación de cada proveedor. Para gestionar este problema se estudiaron las siguientes alternativas:

- **Enumerado estático:** Incorporar los modelos como un enumerado en la gramática. Proporciona autocompletado y validación sin coste adicional, pero cualquier actualización del catálogo de modelos requiere modificar la gramática, regenerar el AST y redistribuir la extensión.
- **String libre:** Permitir que el usuario introduzca el modelo libremente, delegando en él la responsabilidad de conocer los valores válidos. Esto obliga a consultar la documentación de cada proveedor, lo que va en contra del objetivo del DSL de simplificar la configuración del sistema.
- **String con validación:** El valor introducido se contrasta con un catálogo de modelos válidos mantenido en un fichero independiente (`models.ts`). La gramática incluye un atributo `provider` separado, lo que permite al *completion provider* sugerir únicamente los modelos compatibles con el proveedor seleccionado. Actualizar el catálogo no requiere regenerar el AST ni modificar el generador, aunque sí implica redistribuir la extensión.
- **Endpoint con modelos actualizados:** Consultar dinámicamente un endpoint propio que exponga el catálogo de modelos vigente de cada proveedor. Eliminaría la necesidad de redistribuir la extensión ante cada cambio, pero requeriría desarrollar y mantener una infraestructura de servidor cuyos costes quedan fuera del alcance de este trabajo.

Se optó por el *string con validación*. El endpoint ofrecería la solución más completa, pero su coste de infraestructura no está justificado para el alcance del proyecto. El enumerado estático implica regenerar el AST con cada actualización del catálogo. El string libre traslada al usuario la responsabilidad de conocer los modelos válidos, lo que contradice el propósito del DSL. El string con validación equilibra mantenimiento sencillo —basta con actualizar `models.ts`— con una experiencia de usuario asistida mediante autocompletado contextual.

Paso del metamodelo a Langium

Una vez tomadas las decisiones anteriores, la traducción del metamodelo a gramática Langium fue en su mayor parte un proceso directo. A continuación se presentan las distintas construcciones.

Raíz. El nodo raíz es el punto de entrada del modelo y agrupa el resto de construcciones del sistema.

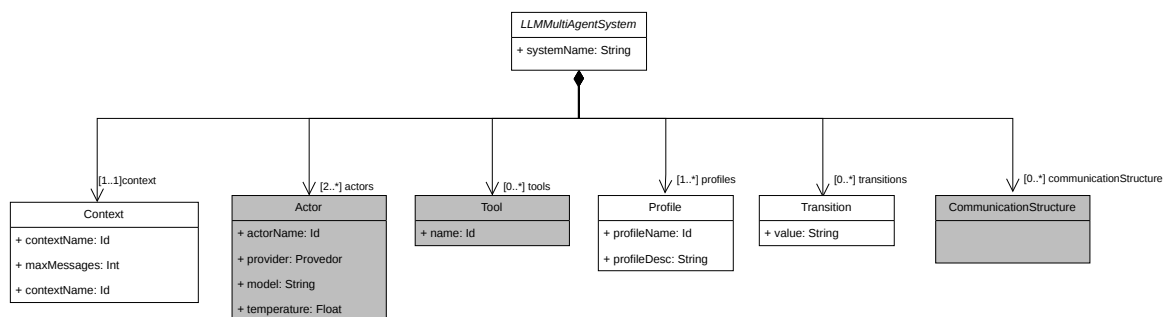


Figura 6.2: Nodo raíz del metamodelo UML.

En Langium, la regla de entrada impone un orden de declaración estricto: los elementos deben aparecer en el modelo exactamente en el orden en que se definen en la gramática. Por ello se adoptó un orden lógico de dependencias: primero el contexto, los perfiles y las herramientas —que no referencian ningún otro elemento—; a continuación los actores, que referencian perfiles y herramientas; y por último las estructuras de comunicación y las transiciones, que referencian actores.



```
entry LLMMultiAgentSystem:
  '{'
    context=Context
    profiles+=Profile*
    tools+=(PythonTool | MCPServer)*
    actors+=(Agent | Summarizer | Coordinator)*
    communicationStructures+=(Layered | SharedMessagePool | Centralized |
    Decentralized)+
    transitions+=CommTransition*
  '}';
```

Figura 6.3: Nodo raíz traducido a gramática Langium.

Context. Siguiendo en orden de definición en Langgraph, el contexto se encarga de definir si el sistema incluye trimeado, el tipo de persistencia y los atributos que leen y escriben los agentes.

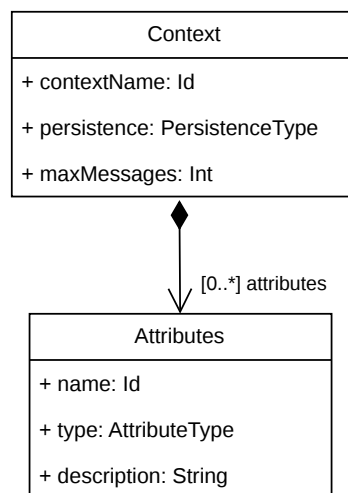


Figura 6.4: Context del metamodelo UML.



```
AttributeType returns string:
    'int' | 'string' | 'boolean';

type PersistenceValue = InMemorySaver | PostgreSaver | MongoDBSaver;

InMemorySaver:
    {infer InMemorySaver} 'inMemorySave';

PostgreSaver:
    {infer PostgreSaver} 'Postgre';

MongoDBSaver:
    {infer MongoDBSaver} 'MongoDB';

Context:
    'context' name = ID '{'
        attributes+=Attribute*
        ('maxMessages' maxMessages=INT)?
        'persistence' persistence=( InMemorySaver | PostgreSaver | MongoDBSaver)
    '}'

Attribute:
    'attribute' name = ID 'type' type = AttributeType 'description'
    description = STRING
```

Figura 6.5: Context traducido a gramática Langium.

Profile. El perfil es una construcción auxiliar que asocia un nombre a una descripción textual utilizada como instrucción de sistema del actor. Su estructura en el metamodelo es directa y la traducción a Langium resulta inmediata.

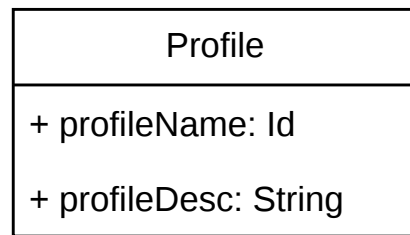


Figura 6.6: Profile del metamodelo UML.

```

Profile:
  'profile' name=ID 'description' profileDescription=STRING;
  
```

Figura 6.7: Profile traducido a gramática Langium.

Tool. La jerarquía de herramientas sigue el mismo patrón `fragment + type` descrito en la sección anterior. En el metamodelo UML la clase abstracta `Tool` agrupa los atributos compartidos; en Langium esto se expresa mediante `fragment ToolBase` para los atributos comunes y `type Tool = PythonTool | MCPServer` como alias de unión.

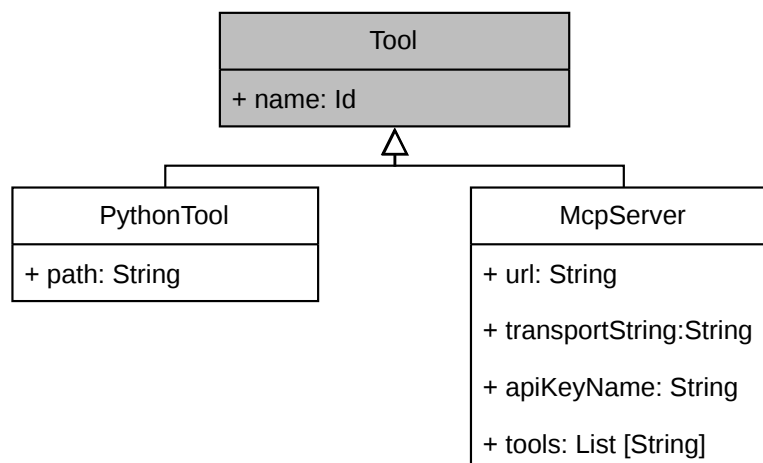


Figura 6.8: Tool del metamodelo UML.



```

type Tool = PythonTool | MCPServer ;

fragment ToolBase:
    name=ID;

PythonTool:
    'pythonTool' ToolBase 'modulePath' modulePath=STRING;

MCPServer:
    'mcpServer' name=ID '{'
        'url' url=STRING
        'transport' transport=STRING
        ('apiKeyName' apiKeyName=STRING)?
        'tools' tools+=STRING (',' tools+=STRING)*
    '}' ;

```

Figura 6.9: Tool traducido a gramática Langium.

Actor. La jerarquía de actores aplica igualmente el patrón `fragment + type`. En el metamodelo, `Actor` es la clase abstracta que agrupa los atributos comunes de `Agent`, `Coordinator` y `Summarizer`; en Langium esta clase abstracta se desdobra en dos construcciones: `fragment ActorBase`, que incrusta los atributos compartidos directamente en el nodo AST de cada subtipo, y `type Actor = Agent | Summarizer | Coordinator`, que es el alias de unión equivalente. De forma análoga, `ProviderModel` aparece en la gramática como un segundo fragmento anidado dentro de `ActorBase`, reflejando los atributos de proveedor y modelo definidos en el metamodelo.

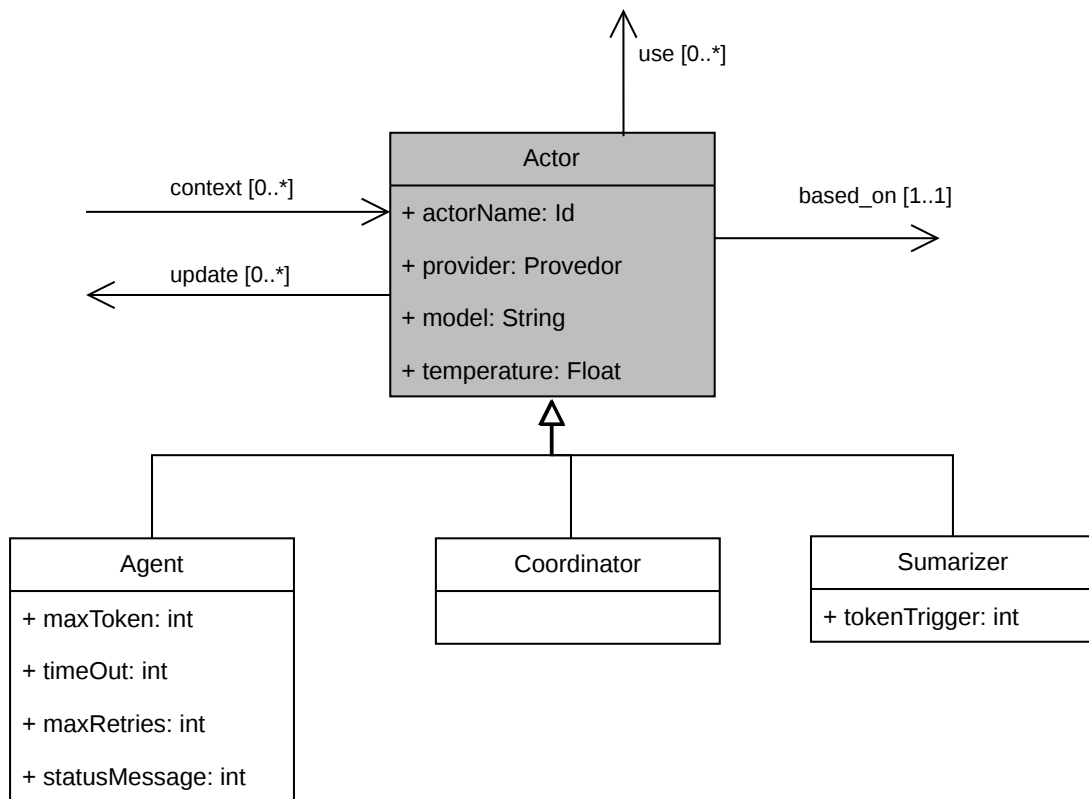


Figura 6.10: Actor del metamodelo UML.



```

ProviderType returns string:
    'openai' | 'anthropic' | 'ollama' | 'google_genai';

fragment ProviderModel:
    'provider' provider=ProviderType
    'model' model=STRING;

type Actor = Agent | Summarizer | Coordinator;

fragment ActorBase:
    name=ID '{'
        ProviderModel
        ('profile' profile=[Profile])?
        ('temperature' temperature = FLOAT)?

Summarizer:
    'summarizer' ActorBase
        ('tokenTrigger' tokenTrigger=INT)?
    '}'

Agent:
    'agent' ActorBase

    //Recomendable
    ('description' description=STRING)?
    ('stateContext' stateContext+=[Attribute] (',' stateContext+=[Attribute]))*?
    ('stateUpdate' stateUpdate+=[Attribute] (',' stateUpdate+=[Attribute]))*?

    // Opcional
    ('statusMessage' statusMessage = STRING)?
    ('maxToken' maxToken = INT)?
    ('timeOut' timeOut = INT)?
    ('maxRetries' maxRetries = INT)?
    ('tools' tools+=[Tool:ID] (',' tools+=[Tool:ID]))*?
    '}'

Coordinator:
    'coordinator' ActorBase
    '}'

```

Figura 6.11: Actor traducido a gramática Langium.



Estructura de comunicación y transiciones. Esta construcción concentra dos diferencias de nomenclatura respecto al metamodelo UML. En primer lugar, la clase abstracta `CommunicationStructure` no se declara como regla de gramática en Langium: sus subtipos (`Layered`, `Centralized`, `Decentralized`, `SharedMessagePool`) son reglas independientes que se listan como alternativas en línea en la regla raíz (`communicationStructures+=(...)+`); `CommunicationStructure` existe únicamente como alias de tipo para permitir las referencias cruzadas desde `CommTransition`. En segundo lugar, la clase `Transition` del metamodelo recibe el nombre `CommTransition` en Langium para evitar ambigüedad con el término genérico *transition*; sus referencias `from/to` apuntan a `CommunicationStructure`, admitiendo además los centinelas `START` y `END` para marcar los extremos del flujo. La condición opcional, modelada como atributo `value: String` en el UML, se refina en Langium con una regla `Condition` estructurada (atributo, operador y literal tipado).

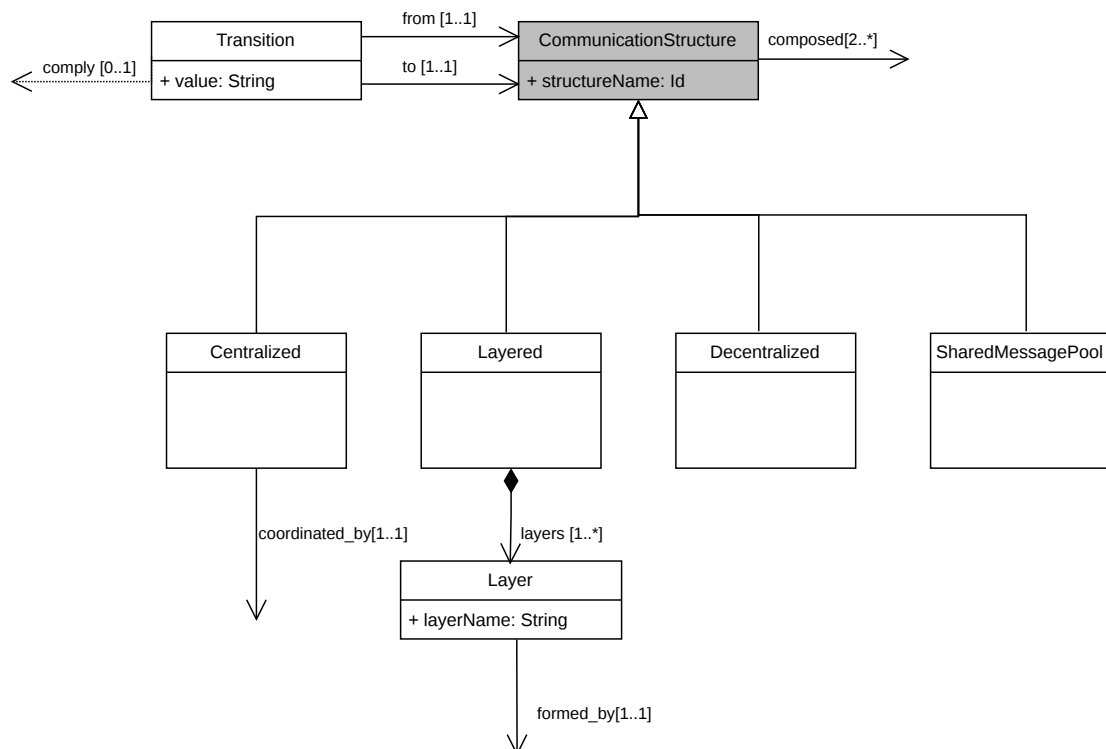


Figura 6.12: Estructura de comunicación y transiciones del metamodelo UML.

```

CommTransition:
    'from' (isStart?='START' | source=[CommunicationStructure])
    'to' (isEnd?='END' | target=[CommunicationStructure])
    ('when' condition=Condition)?;

ConditionType returns string:
    'equal' | 'greater' | 'lower';

Condition:
    attribute=[Attribute] operator=ConditionType value=ConditionLiteral;

ConditionLiteral:
    BoolLiteral | IntLiteral | StringLiteral;

BoolLiteral: {infer BoolLiteral} value=('True' | 'False');
IntLiteral: {infer IntLiteral} value=INT;
StringLiteral: {infer StringLiteral} value=STRING;

Layered:
    'layered' name=ID '{'
        layers+=Layer+
    '}'

Centralized:
    'centralized' name=ID '{'
        'coordinator' coordinator=[Coordinator]
        Agents
    '}'

SharedMessagePool:
    'shared_message_pool' name=ID '{'
        Agents
    '}'

Decentralized:
    'decentralized' name=ID '{'
        'agents' agents+=[Agent] ', ' agents+=[Agent] (' ', agents+=[Agent])*
    '}'

Layer:
    'layer' agent=[Agent] ('next' next=[Agent])?;

```

6.1.2. Validación semántica

Las restricciones semánticas se implementan como métodos de la clase `MultiAgentDslValidator` en `multi-agent-dsl-validator.ts`. La función `registerValidationChecks()` asocia cada tipo de nodo del AST a uno o varios de estos métodos mediante el objeto `ValidationChecks<MultiAgentDslAstType>`; Langium invoca automáticamente el método correspondiente cada vez que el nodo aparece en el modelo. Cada método recibe el nodo concreto y un `ValidationAcceptor` y llama a `accept()` con la severidad (`'error'` o `'warning'`), el mensaje, el nodo afectado y, opcionalmente, la propiedad exacta sobre la que debe aparecer el subrayado en el editor. Para evitar duplicación entre los tres subtipos de actor, las comprobaciones de temperatura, valores positivos y modelo de proveedor se extraen a funciones auxiliares compartidas (`checkTemperature`, `checkPositive` y `checkProviderModel`).

Además de las restricciones formales del diseño, se han añadido tres *warnings* que alertan sobre combinaciones de proveedor y estructura que, siendo sintácticamente válidas, presentan problemas conocidos en tiempo de ejecución.

La siguiente tabla resume las restricciones y *warnings* implementados junto con la función validadora asociada y la severidad del diagnóstico emitido.

Código	Descripción	Función(es) validadora(s)	Severidad
R01	Todas las estructuras conectadas a alguna transición	<code>checkCommunicationStructuresConnected</code>	error
R02	URL única por <code>MCPServer</code>	<code>checkMcpServer</code>	error
R03	Exactamente un <code>START</code> en el sistema	<code>uniqueStartPoint</code>	error
R04	Sin arcos duplicados desde el mismo origen	<code>checkDuplicatedArcs</code>	error

Código	Descripción	Función(es) validadora(s)	Severidad
R05	Transición única desde un origen no puede llevar condición	<code>checkUniqueTransition</code>	error
R06	Tipo del valor compatible con el tipo del atributo	<code>checkConditionTypeCompatibility</code>	error
R07	Como máximo una transición sin condición por origen	<code>checkTransitionCompatibility</code>	error
R08	Temperatura en [0,0, 1,0] (Agent)	<code>checkAgentTemperature</code>	error
R08	Temperatura en [0,0, 1,0] (Coordinator)	<code>checkCoordinatorTemperature</code>	error
R08	Temperatura en [0,0, 1,0] (Summarizer)	<code>checkSummarizerTemperature</code>	error
R09	Un único Summarizer en el sistema	<code>checkSummarizerUnique</code>	error
R10	<code>maxToken</code> , <code>timeOut</code> , <code>maxRetries</code> > 0 (Agent)	<code>checkAgentPositiveValues</code>	error
R10	<code>tokenTrigger</code> > 0 (Summarizer)	<code>checkSummarizerPositiveValues</code>	error
R10	<code>maxMessages</code> > 0 (Context)	<code>checkContextPositiveValues</code>	error
R11	Sin ciclos en estructura Layered	<code>checkLayeredNoCycles</code>	error
R12	<code>apiKeyName</code> única por MCPServer	<code>checkMcpApiKeyUnique</code>	error

Código	Descripción	Función(es) validadora(s)	Severidad
R13	Nombres únicos en el espacio de nombres global	<code>checkUniqueNames</code>	error
R14	<code>Summarizer</code> sin perfil asignado	<code>checkSummarizerNoProfile</code>	error
R15	Un agente pertenece a una sola estructura de comunicación	<code>checkAgentUniqueStructure</code>	error
W01	<code>ollama</code> con <code>tools</code> : posible falta de soporte	<code>checkAgentOllamaToolsWarning</code>	warning
W02	<code>Coordinator</code> con <code>google_genai</code> : riesgo de bucle	<code>checkCoordinatorGoogleWarning</code>	warning
W03	<code>Decentralized</code> con <code>google_genai</code> : riesgo de bucle	<code>checkDecentralizedGoogleWarning</code>	warning

Tabla 6.1: Restricciones semánticas implementadas en el validador.

Las restricciones R08 y R10 requieren métodos separados para `Agent`, `Coordinator` y `Summarizer` a pesar de que los tres subtipos comparten la misma lógica. Esto se debe a la naturaleza de `type Actor`: como se vio en la Sección 6.1.1, `type` es un alias de unión TypeScript puro que no genera ningún nodo real en el AST. El `ValidationRegistry` de Langium solo acepta tipos de nodo concretos como claves de registro —`Agent`, `Coordinator`, `Summarizer`—, de modo que no existe ningún tipo `Actor` sobre el que registrar un único método. Cabe destacar que en todos los casos la lógica de la restricción fue abstraída a una única función de forma que aseguremos la consistencia. En esencia estamos aplicando el principio Don't Repeat Yourself (DRY).



6.2. Implementación del generador

Esta sección describe la implementación del generador de código LangGraph, incluyendo su organización interna y las decisiones técnicas tomadas en cada módulo.

6.2.1. Organización modular

El generador se estructura en módulos independientes, uno por cada fichero de salida. El punto de entrada es `main.ts`, que invoca a `generator.ts`, encargado de orquestar la ejecución de todos los generadores en el orden adecuado. Las funciones auxiliares compartidas entre módulos se centralizan en `util.ts`. El subdirectorio `templates/` contiene fragmentos de código Python estáticos que se insertan directamente en la salida sin transformación, lo que evita duplicar lógica de generación para bloques que son siempre idénticos.

A continuación podemos observar como queda dividido el paquete:



```
src/
├── generator.ts
├── generators/
│   ├── agentsGenerator.ts
│   ├── checkpointerGenerator.ts
│   ├── configGenerator.ts
│   ├── edges/
│   │   ├── index.ts
│   │   ├── layered.ts
│   │   ├── centralized.ts
│   │   └── decentralized.ts
│   ├── graphGenerator.ts
│   ├── jsonGenerator.ts
│   ├── mcpClientGenerator.ts
│   ├── promptGenerator.ts
│   ├── requirementsGenerator.ts
│   └── stateGenerator.ts
├── main.ts
├── templates/
│   ├── dbBlock.ts
│   └── reducers.ts
└── util.ts
```

Figura 6.14: Estructura del paquete del generador (`src/`).

La siguiente tabla resume la correspondencia entre cada módulo y el fichero que genera.

Módulo	Fichero generado	Descripción
<code>main.ts</code>	—	Punto de entrada; invoca al orquestador
<code>generator.ts</code>	—	Orquesta los generadores en orden
<code>util.ts</code>	—	Funciones auxiliares compartidas
<code>stateGenerator.ts</code>	<code>state.py</code> <code>reducers.py</code>	Definición del estado compartido
<code>agentsGenerator.ts</code>	<code>agents.py</code>	Funciones de nodo de cada actor
<code>promptGenerator.ts</code>	<code>prompts.py</code>	Constantes de prompt de sistema
<code>graphGenerator.ts</code>	<code>graph.py</code>	Construcción del grafo y transiciones
<code>edges/</code>	<code><subgraph>.py</code>	Un subgrafo por cada CommunicationStructure
<code>mcpClientGenerator.ts</code>	<code>mcpClients.py</code>	Cientes MCP configurados
<code>checkpointerGenerator.ts</code>	<code>checkpointer.py</code>	Configuración del checkpointer
<code>configGenerator.ts</code>	<code>config.py</code> , <code>.env</code>	Variables de entorno y configuración
<code>jsonGenerator.ts</code>	<code>langgraph.json</code>	Definición del proyecto LangGraph
<code>requirementsGenerator.ts</code>	<code>requirements.txt</code>	Dependencias Python del proyecto
<code>templates/</code>	—	Fragmentos estáticos insertados en <code>state.py</code>

Tabla 6.2: Módulos del generador y ficheros de salida correspondientes.



6.2.2. Generación del estado

`stateGenerator.ts` es el módulo responsable de generar el subdirectorio `state/`, que contiene los siguientes ficheros:

- `state.py`: define el estado compartido del sistema como un `TypedDict` con el campo de mensajes y los atributos modelados en el contexto.
- `reducers.py`: se genera únicamente cuando el modelo incluye un `Summarizer` o define `maxMessages`; contiene el nodo de resumen, el reductor de recorte de mensajes o ambos, según corresponda.
- `__init__.py`: expone el subdirectorio como paquete Python.

Este módulo consume los nodos `Context` y `Attribute` del AST. La siguiente figura muestra el flujo de generación.

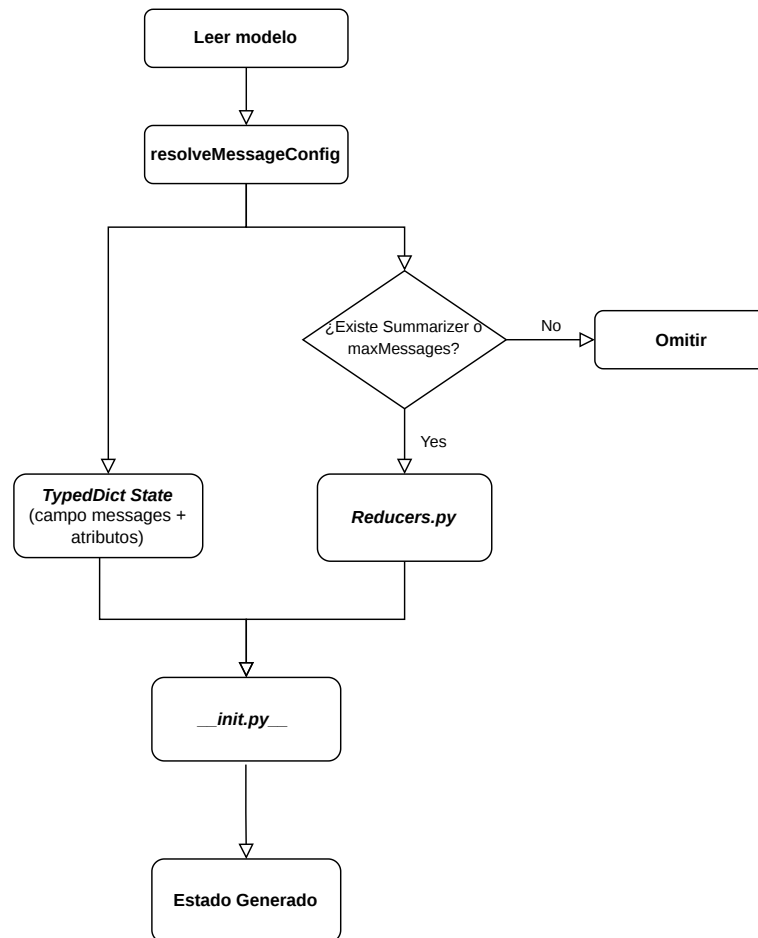


Figura 6.15: Flujo de generación del estado.

La función `resolveMessageConfig()` determina la estrategia de gestión de mensajes en función de si el modelo define un `Summarizer`, un límite de mensajes (`maxMessages`), ambos o ninguno.



```
export function resolveMessageConfig(model: LLMMultiAgentSystem): MessageConfig {  
  const summarizer = model.actors.find(isSummarizer);  
  const hasTrim = model.context.maxMessages !== undefined;  
  if (summarizer && hasTrim) return buildMix(summarizer);  
  if (summarizer) return buildSummarize(summarizer);  
  if (hasTrim) return TRIM;  
  return NONE;  
}
```

Figura 6.16: Función `resolveMessageConfig()` en `stateGenerator.ts`.

Cuando solo se define `maxMessages`, se devuelve la constante `TRIM`, un reductor estático tomado directamente de `templates/` que recorta el historial al número de mensajes configurado. Cuando el modelo incluye un `Summarizer`, las funciones `buildSummarize()` y `buildMix()` reciben el nodo como parámetro para incorporar sus valores (modelo, proveedor, temperatura) en el código generado.

Para la implementación del `Summarizer` se evaluaron tres alternativas:

- **Wrapper sobre el reductor:** envolver el reductor de mensajes con un wrapper que detecte el desbordamiento y dispare el resumen en cada adición de mensaje. Se descartó porque el recuento de tokens se ejecutaría en cada actualización del estado, con el coste computacional asociado.
- **Nodo dedicado al inicio:** insertar el nodo de resumen como primer nodo del grafo, antes del primer nodo funcional. Se descartó porque resumir al inicio del turno implica condensar el historial antes de que se procese la nueva entrada.
- **Nodo dedicado al final con router:** incluir la detección del límite de tokens en el router de salida; si se supera, el flujo se redirige al nodo de resumen antes de terminar. Esta es la opción adoptada, ya que desacopla la lógica de resumen del generador del estado y la concentra en el generador del grafo.

Paralelamente surgió la cuestión de dónde almacenar el resumen generado. La primera aproximación fue guardarlo como un `SystemMessage` dentro del historial; sin embargo,



la API de Anthropic solo admite un único mensaje de sistema por conversación, lo que entra en conflicto con el perfil del agente, que también se inyecta como `SystemMessage`. Por ello se optó por añadir un campo `summary` explícito al estado, solución que además coincide con el patrón recomendado en la documentación de LangGraph.

6.2.3. Generación de agentes y prompts

Los agentes modelan su comportamiento a través de un *prompt* de sistema. Todos los prompts se agrupan en un único fichero generado, `prompts.py`, separado del fichero de agentes para mejorar la legibilidad del código generado, ya que los prompts suelen ocupar varias líneas. Este fichero es producido por `promptGenerator.ts`, que recorre todos los nodos `Profile` y genera una constante de cadena por cada uno. El resultado es un fichero como el siguiente:

```
RESEARCHER = """
Eres un agente investigador. Dado un tema o pregunta, tu tarea es
recopilar informacion relevante y estructurarla en puntos clave.
Se conciso y cita las fuentes cuando las tengas.
"""

ANALYST = """
Eres un agente analista. Recibes informacion recopilada y debes
identificar patrones, contradicciones y conclusiones. Presenta tu
analisis de forma ordenada.
"""

WRITER = """
Eres un agente redactor. Recibes un analisis estructurado y lo
transformas en un texto claro y fluido adaptado al usuario final.
No agregues informacion nueva, solo mejora la presentacion.
"""
```

Figura 6.17: Ejemplo de `prompts.py` generado.

Para la generación de los agentes se utiliza `agentsGenerator.ts`, que produce el fichero `agents.py`. Este módulo consume únicamente los nodos `Actor` de tipo `Agent` del AST y sigue el siguiente flujo de generación:

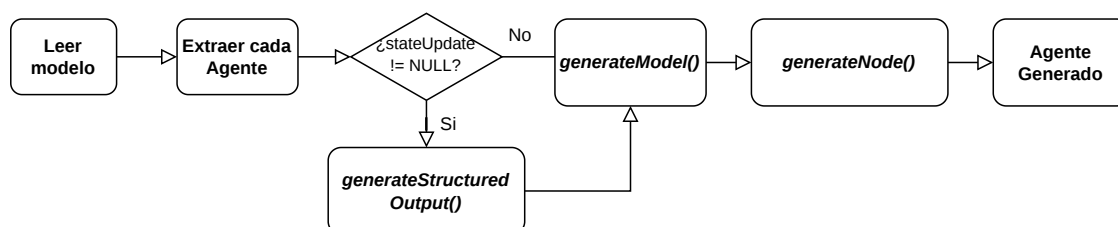


Figura 6.18: Flujo de generación de los agentes.

El primer paso consiste en comprobar si el agente declara alguna referencia de tipo `stateUpdate`; en caso afirmativo, es necesario generar el mecanismo para actualizar los atributos referenciados. Para ello se utiliza el patrón *Structured Output* de LangChain, en el que el modelo devuelve una instancia de un esquema Pydantic en lugar de texto libre. Internamente, el LLM recibe dicho esquema como un esquema JSON que le obliga a producir una respuesta que se ajuste a él.

```

class InfoExtractorOutput(BaseModel):
    orderId: str = Field(description="Id del pedido con un problema")
    issueDescription: str = Field(description="Descripcion del problema")
    preferredSolution: str = Field(description="Solucion preferida por el cliente, " \
    "si la hay.")
    productID: str = Field(description="Id del producto con problemas")
  
```

Figura 6.19: Ejemplo del patrón *Structured Output*.

A continuación, `generateModel()` construye el objeto modelo para cada agente, configurando sus parámetros concretos: temperatura, número máximo de tokens, `timeOut`, etc. En este paso también se vinculan las herramientas a las que tiene acceso el agente.



```
modelValidator = init_chat_model(model="openai:gpt-5-nano", temperature=0)
    .bind_tools([orderChecker, ValidatorOutput], tool_choice="required")
modelInfoExtractor = init_chat_model(model="openai:gpt-5-nano", temperature=0)
    .bind_tools([getInfoOrder, InfoExtractorOutput], tool_choice="required")
modelAnswerWriter = init_chat_model(model="openai:gpt-5-nano", temperature=0)
```

Figura 6.20: Ejemplo de modelos generados para distintos agentes.

Por último, `generateNode()` genera la lógica de cada nodo: qué datos recibe del estado, cómo invoca las herramientas y qué devuelve. Su funcionamiento se detalla en la Figura 6.21:

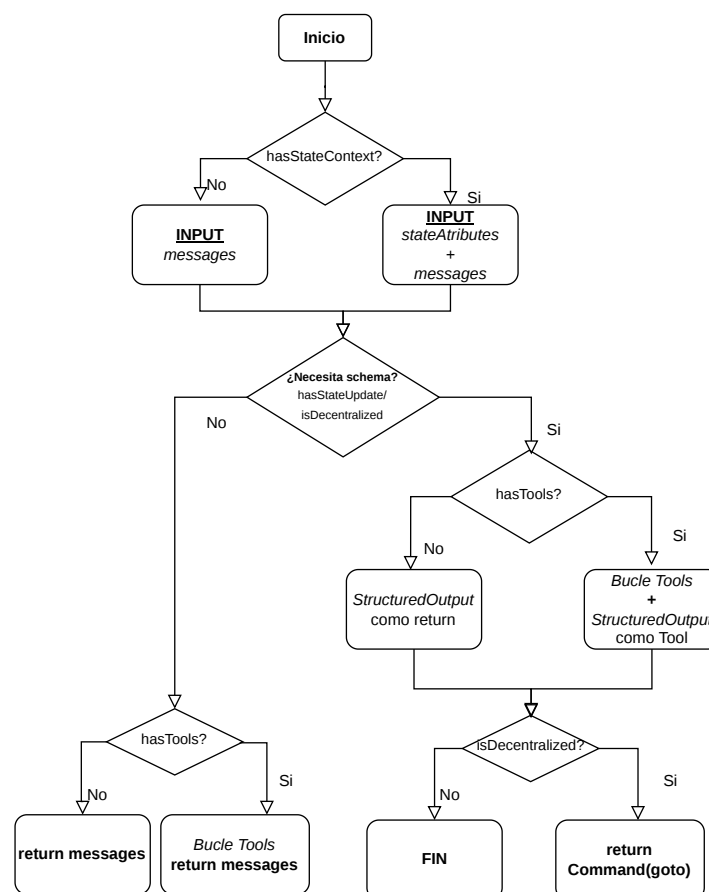


Figura 6.21: Flujo de `generateNode()`.



La plantilla resultante para cada nodo es la siguiente:

```
def <nodeName>(state):
    """<descripcion del agente>"""
    [? si <statusMessage> get_stream_writer()({"status": "<mensaje>"})

    # INPUT
    messages = [SystemMessage(content=
        [? si hay Summarizer: _system_prompt(PROFILE, state)]
        [? si no: PROFILE]
    )]
    messages += state["messages"]
    [? si <stateContext> messages += [HumanMessage(content=f"""
        attr1: {state.get("attr1", "No registrado")}
        attr2: {state.get("attr2", "No registrado")}
        ...
    """)]

    # INVOCACION
    [? si NO hay <tools>]
        result = modelAgente.invoke(messages)
    [? si HAY <tools>]
        while True:
            response = await modelAgente.ainvoke(messages)
            messages.append(response)

            [? si hasStructured]
                # Schema bindeado como tool -> senal de fin
                for tc in response.tool_calls:
                    if tc["name"] == "<Agente>Output":
                        <RETURN ESTRUCTURADO>
            [? si NO hasStructured]
                # Fin cuando el modelo deja de pedir <tools>
                if not response.tool_calls:
                    <RETURN MENSAJES>

            for tc in response.tool_calls:
                tool = _tools_by_name[tc["name"]]
                result = await tool.ainvoke(tc["args"])
                messages.append(ToolMessage(content=str(result),
                    tool_call_id=tc["id"]))

    # RETURN
    <RETURN segun el caso>
```

Figura 6.22: Plantilla del código de los nodos generados.

La sección `stateContext` de la plantilla merece una aclaración. La implementación inicial accedía a los atributos mediante `state[attr]`, lo que era sintácticamente limpio y reflejaba la semántica del `TypedDict` declarado en `state.py`; sin embargo, al ejecutar los modelos sobre LangGraph surgió un problema: LangGraph solo materializa en el estado los campos que se pasen explícitamente en el *input* del grafo, por lo que los campos

`Optional[X]` no existen como claves hasta que algún nodo los escriba, y el acceso por corchetes lanzaba un `KeyError` en cualquier agente con `stateContext` que precediera a los nodos responsables de rellenar esos campos. Se valoró inicializar todos los atributos a `None` en el estado inicial, pero se descartó: implicaba añadir lógica de *bootstrap* fuera de los nodos del modelo, y no resolvía el problema semántico —el LLM recibiría un `None` indistinguible de un valor legítimamente ausente—. La solución adoptada fue `state.get(attr, "No registrado")`: evita el `KeyError` cuando el campo todavía no existe y proporciona al LLM una cadena con significado claro que le permite razonar correctamente sobre el estado parcial.

La siguiente tabla recoge las distintas combinaciones posibles en función de los flags `isDecent` (pertenece a una estructura de comunicación descentralizada), `hasTools` y `stateUpdate`:

isDecent	hasTools	stateUpdate	función	bucle	schema-as-tool	return final
×	×	×	def sync	—	—	messages
×	×	✓	def sync	—	with_structured_output	dict stateUpdate
×	✓	×	async def	while	—	messages cuando no hay tool_calls
×	✓	✓	async def	while	sí	dict stateUpdate al invocar schema
✓	×	(✓ forzado)	def sync	—	with_structured_output	Command(goto, update)
✓	✓	(✓ forzado)	async def	while	sí	Command(goto, update) al invocar schema

Tabla 6.3: Combinaciones de generación de nodos de agente.

Como se observa en la tabla, la estructura `Decentralized` fuerza `stateUpdate` independientemente de si está declarado en el modelo. Esto se debe a que, en dicha estructura, cada agente decide a qué nodo continuar; para ello, el esquema debe incluir obligatoriamente el campo `next`:

```
next: Literal["agente1", "agente2", "END"] = Field(...)
```

Figura 6.23: Campo `next` obligatorio en el esquema de nodos `Decentralized`.



Sin ese campo, el grafo no sabría qué valor asignar a `goto` en el `Command(goto=...)`. Por ello, el generador construye el `BaseModel` en todo caso: aunque el agente `Decentralized` no declare ningún `stateUpdate` en el `.mad`, el esquema se genera con al menos el campo `next` (y `messages` si tampoco dispone de herramientas).

Otra decisión relevante fue cómo integrar el uso de herramientas en los agentes. En `LangGraph`, el patrón habitual consiste en modelar las herramientas como nodos separados del grafo. Sin embargo, en este trabajo se optó por encapsularlas dentro del propio nodo del agente mediante un bucle, de forma que el grafo solo expone un nodo por agente. Si se hubiera seguido el patrón estándar, el generador tendría que:

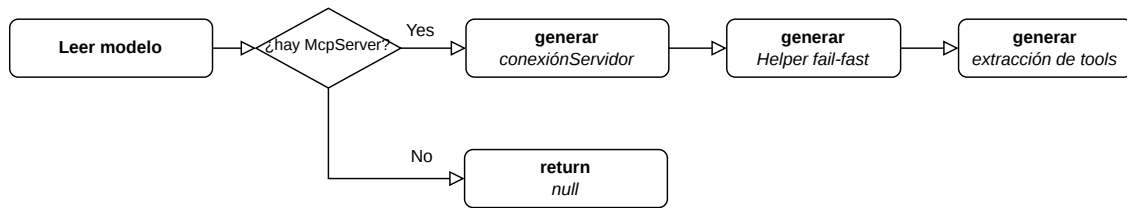
- Emitir un `ToolNode` por cada agente con herramientas.
- Emitir una `conditional_edge` que dirigiera el flujo hacia el `ToolNode` o hacia el siguiente agente declarado en el `.mad`.

Esta decisión está motivada por el nivel de abstracción del metamodelo: cuando el grafo se escribe a mano, el desarrollador tiene visión completa de la topología y puede decidir la estructura de cada agente individualmente. El metamodelo del DSL, en cambio, describe la coreografía entre agentes —capas, secuencias, coordinador— sin entrar en el flujo interno modelo-herramientas-modelo de cada nodo. Exponer los `ToolNode` en el grafo generado introduciría artefactos que no corresponden a ningún concepto del metamodelo y complicaría el generador del grafo sin aportar valor al usuario.

6.2.4. Generación de herramientas

Como se indicó anteriormente, el DSL no tiene como objetivo generar implementaciones de herramientas; no obstante, sí es necesario generar el código que permite importar las herramientas de los servidores MCP declarados en el modelo.

De ello se encarga `mcpClientGenerator.ts`, que produce `mcpClients.py` cuando el modelo define al menos un `McpServer`. El flujo de generación se muestra en la siguiente figura:

Figura 6.24: Flujo de `mcpClientGenerator.ts`.

Como se aprecia en la figura, existe una función intermedia entre la conexión al servidor y la extracción de las herramientas. Dicha función resuelve cada herramienta por su nombre contra el catálogo real del servidor MCP en el momento de importar el módulo.

```

def _get_tool(name: str):
    tool = next((t for t in _all_tools if t.name == name), None)
    if tool is None:
        raise RuntimeError(
            f"Tool '{name}' no encontrada en el servidor MCP. "
            f"Tools disponibles: {[t.name for t in _all_tools]}"
        )
    return tool
  
```

Figura 6.25: Implementación del patrón *fail-fast* en `_get_tool()`.

La alternativa habría sido acceder directamente con `next()` sin valor por defecto:

```

tavily_search = next(t for t in _all_tools if t.name == "tavily_search")
  
```

Figura 6.26: Acceso directo sin *fail-fast*.

Sin embargo, cuando la herramienta no existe, `next()` sin valor por defecto lanza `StopIteration` sin indicar qué herramienta faltaba ni cuáles estaban disponibles, lo que se traduce en una traza opaca sin información útil para el usuario. Esta situación puede producirse en varios escenarios:

- Si el usuario escribe mal el nombre de la herramienta.
- Un cambio de nombre en el servidor remoto.
- Un servidor parcialmente degradado que responda con un subconjunto de su catálogo.



- Un fallo de conexión silencioso: si la lista de herramientas llega vacía, todas las resoluciones fallan en cascada con el mismo mensaje opaco.

Por ello se optó por implementar `_get_tool()`, donde cada caso de error produce un mensaje distinto y concreto. El coste adicional en el generador fue mínimo, lo que lo convirtió en una solución adecuada.

6.2.5. Generación del grafo

El grafo conecta todos los nodos del sistema: las conexiones entre agentes dentro de una misma estructura de comunicación las define dicha estructura, y las conexiones entre estructuras distintas las definen las transiciones del modelo. Desde el punto de vista de la generación, LangGraph ofrece dos primitivas para establecer estas conexiones: `add_edge`, para transiciones incondicionales (el flujo siempre pasa de A a B), y `add_conditional_edges` junto con una función *router*, para transiciones condicionales (el destino depende del valor de un atributo del estado en tiempo de ejecución). La elección entre una u otra depende de si la transición en el `.mad` incluye una `Condition` o no. La decisión arquitectónica relevante fue cómo organizar el grafo generado.

Se optó por generar un subgrafo por cada estructura de comunicación, alojados en el directorio `subgraph/`. El módulo `edges/index.ts` orquesta los generadores específicos (`layered.ts`, `centralized.ts` y `decentralized.ts`), uno por cada tipo de estructura. Por su parte, `graphGenerator.ts` importa dichos subgrafos y genera los enrutadores entre estructuras.

Concentrar todo en un único generador y un único fichero de salida habría presentado dos inconvenientes claros:

- El generador TypeScript habría resultado excesivamente largo y difícil de mantener, aunque su estructura interna fuera equivalente.
- Un único fichero Python generado iría en contra de la práctica habitual de subgrafos reutilizables en LangGraph y reduciría la legibilidad del código para el usuario.

Una decisión complementaria fue si generar un estado por subgrafo o un estado compartido



para todo el sistema. LangGraph promueve el uso de un estado independiente por subgrafo, donde únicamente ciertos atributos se fusionan con el estado del grafo general; el propósito es aislar cada subgrafo para que sus agentes no accedan a atributos de otros subgrafos. Sin embargo, esta opción se detectó en una fase avanzada del desarrollo, cuando incorporarla habría requerido readaptar el DSL. Además, la gestión de estados por subgrafo es un mecanismo específico de LangGraph que no tiene reflejo directo en el metamodelo, por lo que introducirlo habría roto la correspondencia entre el `.mad` y el código generado. Dado que el estado compartido no afecta a la corrección de la ejecución, se optó por mantener un único estado para todos los subgrafos.

Subgrafos Dado que el artículo base de este trabajo propone las estructuras de comunicación sin especificar cómo deben implementarse, se tomaron las siguientes decisiones de diseño para cada una:

Layered. Esta estructura admite dos interpretaciones: varios agentes ejecutándose en paralelo dentro de una misma capa antes de pasar a la siguiente, o un único agente por capa que avanza de forma secuencial. La primera opción produce el mismo resultado que anidar varias estructuras `SharedMessagePool`, por lo que se optó por la segunda: cada capa contiene un único nodo. Esta topología es ampliamente utilizada en sistemas reales y, combinada con las transiciones condicionales, permite modelar una gran variedad de comportamientos.

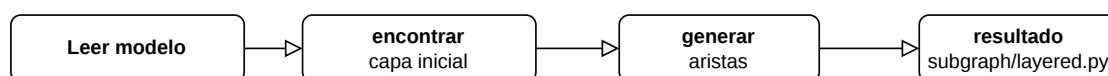


Figura 6.27: Flujo de generación de la estructura `Layered`.



```
def build_ResearchPipeline():
    builder = StateGraph(State)

    builder.add_node("researcher", nodeResearcher)
    builder.add_node("writer", nodeWriter)

    builder.add_edge(START, "researcher")
    builder.add_edge("researcher", "writer")
    builder.add_edge("writer", END)

    return builder.compile()
```

Figura 6.28: Ejemplo de subgrafo **Layered** generado.

Centralized. Esta estructura puede concebirse como un sistema orquestado por un enrutador determinístico o por un coordinador central agéntico. La primera opción es equivalente a combinar varias estructuras **Layered** con transiciones condicionales, por lo que se optó por la segunda: un **Coordinator** que decide, en cada paso, si la tarea ha sido resuelta o qué agente debe ejecutarse a continuación.

Para que el coordinador pueda tomar decisiones informadas, recibe como entrada todos los atributos actualizados por los agentes junto con el historial de mensajes. Se consideró que esta información es suficiente para que, combinada con su prompt, el coordinador determine el siguiente nodo. Se valoró permitir que el usuario especificara explícitamente qué atributos recibe el coordinador, pero se descartó para simplificar el modelado, dado que en la mayoría de los casos el coordinador necesita acceso a la mayor parte de los atributos escritos.

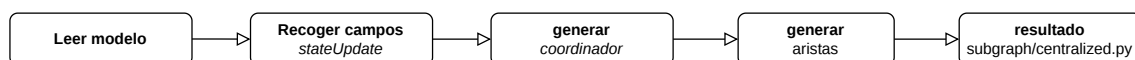


Figura 6.29: Flujo de generación de la estructura **Centralized**.



```
def build_specializedAssistants():
    builder = StateGraph(State)

    builder.add_node("coordinator", coordinator_node)
    builder.add_node("shippinghelper", nodeShippingHelper)
    builder.add_node("paymenthelper", nodePaymentHelper)
    builder.add_node("producthelper", nodeProductHelper)
    builder.add_node("generalhelper", nodeGeneralHelper)

    builder.add_edge(START, "coordinator")
    builder.add_edge("shippinghelper", "coordinator")
    builder.add_edge("paymenthelper", "coordinator")
    builder.add_edge("producthelper", "coordinator")
    builder.add_edge("generalhelper", "coordinator")

    return builder.compile()
```

Figura 6.30: Ejemplo de subgrafo **Centralized** generado.

Decentralized. En esta estructura, cada agente determina cuál es el nodo siguiente. Es la topología más específica de las implementadas: requiere que los agentes devuelvan `Command(goto)`, lo que ya está cubierto por el mecanismo de `stateUpdate` forzado descrito anteriormente. Al no existir un nodo coordinador central, no es necesario generar enrutadores entre agentes; basta con instanciar los nodos y registrar sus aristas de salida directamente.



Figura 6.31: Flujo de generación de la estructura **Decentralized**.

```
def build_knowledgeTriage():
    builder = StateGraph(State)

    builder.add_node("classifier", nodeClassifier)
    builder.add_node("scienceexpert", nodeScienceExpert)
    builder.add_node("historyexpert", nodeHistoryExpert)
    builder.add_node("generalexpert", nodeGeneralExpert)

    builder.add_edge(START, "classifier")

    return builder.compile()
```

Figura 6.32: Ejemplo de subgrafo Decentralized generado.

Grafo principal. `graphGenerator.ts` es el responsable de construir el grafo de nivel superior: registra cada subgrafo compilado como un nodo del grafo principal y después procesa las `CommTransition` del modelo para añadir las conexiones entre estructuras. El flujo de generación se muestra en la Figura 6.33.

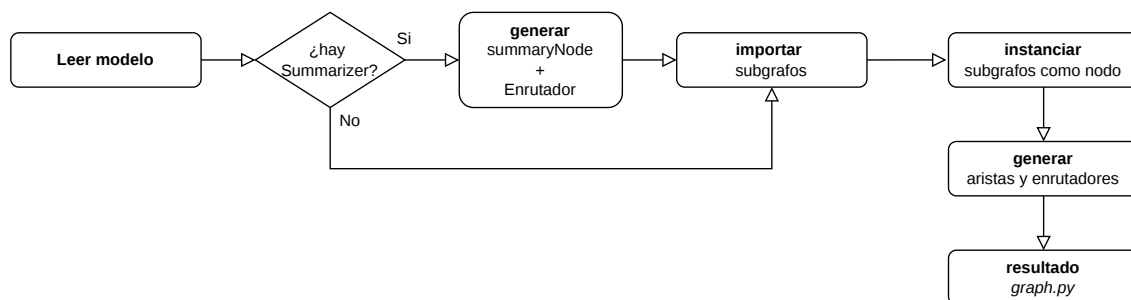


Figura 6.33: Flujo de generación del grafo principal.

La decisión clave en este módulo es cómo traducir cada transición. Si la `CommTransition` no incluye `Condition`, la conexión es incondicional y se genera con `add_edge(origen, destino)`. Si incluye `Condition`, el destino depende del valor de un atributo del estado en tiempo de ejecución, por lo que se genera un *router* — una función que lee dicho atributo y devuelve el nombre del siguiente nodo — y se registra mediante `add_conditional_edges(origen, router, {valor: destino,`



...}). El siguiente ejemplo muestra un `graph.py` generado que combina ambos tipos de transición:

```
# Construcción del grafo
builder = StateGraph(State)

# Subgrafos como nodos
ValidationGraph = build_ValidationGraph()
ExtractorGraph = build_ExtractorGraph()
answerWriter = build_answerWriter()
specializedAssistants = build_specializedAssistants()

builder.add_node("validationgraph", ValidationGraph)
builder.add_node("extractorgraph", ExtractorGraph)
builder.add_node("answerwriter", answerWriter)
builder.add_node("specializedassistants", specializedAssistants)

# Edges de inicio
builder.add_edge(START, "validationgraph")

# Routers
def route_validationgraph(state: State) -> str:
    if state["isValid"] == False:
        return "answerwriter"
    if state["isValid"] == True:
        return "extractorgraph"
    return END

# Transiciones
builder.add_conditional_edges(
    "validationgraph",
    route_validationgraph,
    {
        "answerwriter": "answerwriter",
        "extractorgraph": "extractorgraph",
        END: END
    }
)

builder.add_edge("extractorgraph", "specializedassistants")
builder.add_edge("specializedassistants", "answerwriter")
builder.add_edge("answerwriter", END)

# Compilar
graph = builder.compile()
```

Figura 6.34: Ejemplo de `graph.py` generado.



6.2.6. Generación del soporte para producción

Esta subsección describe los generadores de soporte que producen la infraestructura necesaria para ejecutar el sistema generado en un entorno real.

Variables de entorno. `configGenerator.ts` genera dos ficheros complementarios: `.env` y `config.py`. El fichero `.env` actúa como plantilla que el usuario rellena con sus claves de API; `config.py` carga dicho fichero mediante `load_dotenv()` y expone cada variable como una constante importable. Esta separación evita repetir `load_dotenv()` y `os.getenv()` en cada módulo: basta con `from config import OPENAI_API_KEY`. El generador sigue el siguiente flujo:

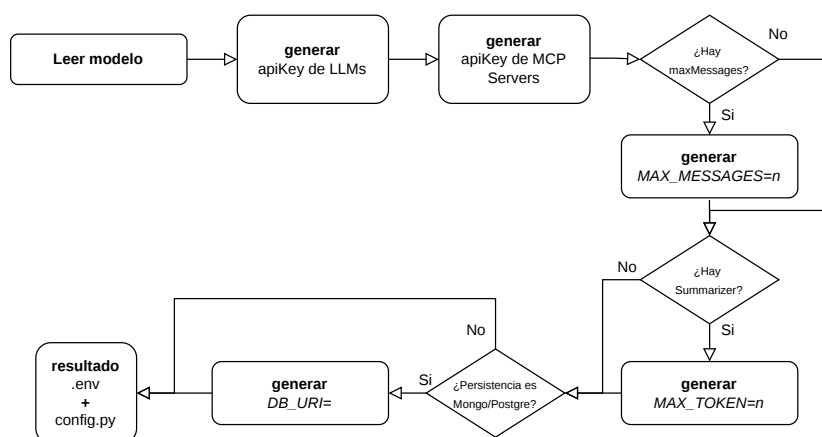


Figura 6.35: Flujo de `configGenerator.ts`.

Las claves `LANGSMITH_API_KEY` y `LANGSMITH_PROJECT` se incluyen siempre en ambos ficheros, independientemente del modelo. Estas variables dan soporte a LangSmith Studio, la interfaz gráfica oficial de LangGraph que permite depurar y visualizar la ejecución del grafo en tiempo real. El resto de claves se generan en función de los proveedores declarados en el `.mad`.



```
LANGSMITH_API_KEY=  
LANGSMITH_PROJECT=  
OPENAI_API_KEY=  
MAX_MESSAGES=
```

(a) .env generado.

```
# config.py -- ARCHIVO GENERADO AUTOMATICAMENTE  
from dotenv import load_dotenv  
import os  
  
load_dotenv()  
  
os.environ["LANGCHAIN_TRACING_V2"] = "true"  
LANGSMITH_API_KEY = os.getenv("LANGSMITH_API_KEY")  
LANGSMITH_PROJECT = os.getenv("LANGSMITH_PROJECT")  
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

(b) config.py generado.

Figura 6.36: Ficheros de configuración generados por `configGenerator.ts`.

Soporte para persistencia. El soporte para la persistencia se consigue mediante el uso de los *checkpointers* de LangGraph. Un *checkerpointer* es un componente que LangGraph conecta al grafo en tiempo de compilación y que se encarga de guardar y restaurar el estado completo del grafo tras cada paso de ejecución. Gracias a esto, cada conversación queda asociada a un identificador de hilo (*thread_id*), de forma que el grafo puede retomarse en el punto exacto donde se dejó en ejecuciones anteriores sin perder ningún valor del estado. El generador produce el fichero `checkerpointer.py` con una función `generate_checkerpointer()`. Su implementación concreta depende del tipo de persistencia declarado en el modelo: `InMemorySaver` para persistencia volátil (el estado se pierde al terminar el proceso, útil durante el desarrollo), `AsyncPostgresSaver` para PostgreSQL y `MongoDBSaver` para MongoDB. Los dos últimos leen la cadena de conexión desde la variable de entorno `DB_URI` definida en `config.py`.

El código generado es estático para cada tipo de persistencia y se toma directamente de `templates/dbBlock.ts`, sin transformación. Los dos casos más representativos se muestran a continuación:



```
@contextlib.contextmanager
def generate_checkpoint():
    checkpointer = InMemorySaver()
    yield checkpointer
```

(a) InMemorySaver.

```
@contextlib.asynccontextmanager
async def generate_checkpoint():
    conn = await AsyncConnection.connect(
        DB_URI,
        autocommit=True,
        row_factory=dict_row
    )
    try:
        checkpointer = AsyncPostgresSaver(conn)
        await checkpointer.setup()
        yield checkpointer
    finally:
        await conn.close()
```

(b) AsyncPostgresSaver.

Figura 6.37: Implementaciones de `generate_checkpoint()` generadas.

Soporte LangSmith. `jsonGenerator.ts` genera el fichero `langgraph.json`, que configura el servidor de LangGraph y es el punto de entrada que LangSmith Studio utiliza para depurar y visualizar la ejecución del grafo en su interfaz gráfica.

```
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./graph.py:graph"
  },
  "checkpointer": {
    "path": "./checkpointer.py:generate_checkpoint"
  },
  "env": ".env"
}
```

Figura 6.38: Ejemplo de `langgraph.json` generado.

El fichero puede contener hasta cuatro campos:

- **dependencies:** indica al CLI de LangGraph que instale el directorio actual como paquete Python (`pip install -e .`) antes de arrancar el servidor, de forma que



todos los módulos generados sean importables entre sí.

- **graphs:** registra el grafo bajo el nombre `.agent`, con el que se expone en la API REST. El valor `./graph.py:graph` señala el fichero y la variable que contiene el grafo compilado (`graph = builder.compile()`).
- **checkpointer:** aparece únicamente cuando la persistencia es PostgreSQL o MongoDB. Apunta a `generate_checkpointer()` en `checkpointer.py`, que LangGraph invoca para obtener el *checkpointer* configurado e inyectarlo en el grafo al arrancarlo. Para `InMemorySaver` este campo se omite porque LangGraph lo gestiona internamente.
- **env:** ruta al fichero `.env` con las claves de API. LangGraph lo carga automáticamente antes de iniciar el servidor, equivalente a ejecutar `load_dotenv()` de forma manual.

6.3. Implementación de la extensión

6.3.1. Resaltado y autocompletado

Langium genera automáticamente una *TextMate grammar* a partir de la definición del lenguaje, pero la gramática por defecto no es suficiente para un coloreado semánticamente preciso: asigna el mismo *scope keyword* a palabras clave y atributos indistintamente. Para mejorar la experiencia en el editor, la grammar fue extendida manualmente con las siguientes mejoras:

- **Dos categorías semánticas separadas:** las palabras estructurales (`agent`, `context`, `layered...`) se marcan con `keyword.control`, mientras que los atributos (`provider`, `model`, `temperature`, `stateContext...`) reciben el `scope support.type.property-name`, lo que les asigna colores distintos en la mayoría de temas de VSCode.
- **Lookaheads contextuales:** palabras como `profile` y `coordinator` pueden aparecer tanto como declaración de nodo como en forma de referencia de atributo.



En la grammar se distingue cada caso y se colorea de forma diferente según el contexto.

- **Valores de enumerado diferenciados:** los literales `openai`, `anthropic`, `ollama`, `inMemorySave`, `MongoDB`, `Postgre`, `True` y `False` tienen su propio scope `string.unquoted`, que les confiere un color distinto al de las palabras clave.
- **Operadores con número capturado:** el patrón para los comparadores (`equal`, `greater`, `lower`) captura el operador y el valor numérico como grupos separados, permitiendo que cada uno adopte un color diferente.

Sobre el autocompletado por defecto se ha implementado un `CompletionProvider` personalizado en `packages/language/src/multi-agent-dsl-completion-provider.ts` que extiende `DefaultCompletionProvider`: cuando el cursor se encuentra sobre el campo `model` de un agente, lee el `provider` ya declarado y filtra las sugerencias para ofrecer únicamente los modelos válidos de ese proveedor; en cualquier otra posición se delega en el comportamiento por defecto.

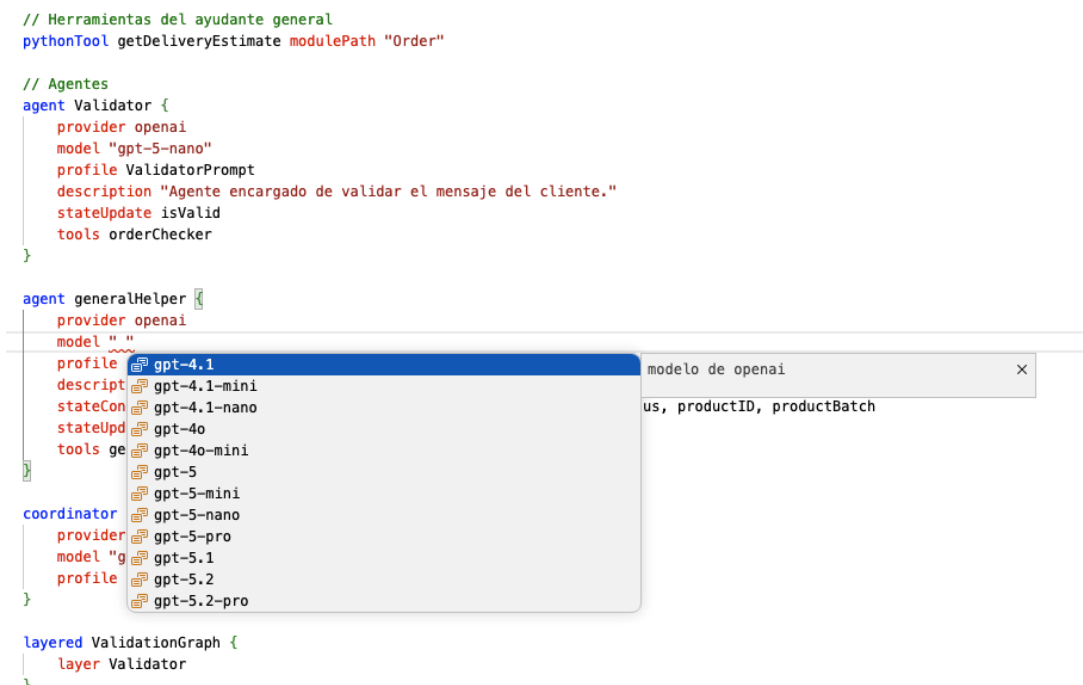


Figura 6.39: Resultado de sintaxis y autocompletado filtrado por proveedor en el editor VSCode.

6.3.2. Ayudas al usuario

Para facilitar el aprendizaje del DSL se han añadido dos mecanismos complementarios. El primero es un `HoverProvider` personalizado implementado en `packages/language/src/multi-agent-dsl-hover-provider.ts`, que muestra documentación contextual al situar el cursor sobre cualquier palabra clave del lenguaje o sobre nodos del AST como `Agent`, `Coordinator` o las distintas estructuras de comunicación; la descripción se renderiza en formato Markdown directamente en el editor. El segundo mecanismo es un conjunto de *snippets* declarados en el manifiesto del paquete `extension` que expanden plantillas predefinidas para los constructos más habituales (definición de agente, estructura centralizada, transición, etc.), reduciendo el coste cognitivo de recordar la sintaxis exacta.



Figura 6.40: Ayudas al usuario: hover provider y snippets predefinidos.

Además, se incluye un botón en el panel superior a través del cual el usuario podrá abrir el manual de usuario.



Figura 6.41: Botón correspondiente al manual de usuario.

6.3.3. Autocompletado de herramientas MCP

La declaración de un `mcpServer` incluye un campo `tools` con la lista de herramientas que se desean exponer al agente. Para evitar que el usuario tenga que consultar la documentación del servidor remoto y escribir los identificadores a mano, el paquete `extension` consulta el `endpoint` MCP declarado y obtiene su catálogo de herramientas disponibles, que expone como sugerencias filtradas en el autocompletado del campo `tools`. La lógica de red reside en el paquete `extension` —y no en `language`— porque depende del entorno concreto del usuario y de credenciales locales, contexto al que el `language server` no tiene acceso. De este modo, los errores tipográficos o las referencias a herramientas inexistentes se detectan en tiempo de edición, no en tiempo de ejecución del código generado.

6.3.4. Comando de generación

El paquete `extension` registra un comando de VSCode asociado a un botón en la barra superior del editor, visible únicamente cuando el archivo activo es un fichero `.mad`. Al pulsarlo, la extensión invoca el generador del paquete `cli` pasándole la ruta del modelo activo; el generador parsea el fichero, ejecuta las validaciones definidas en `packages/language` y produce el proyecto Python en el directorio de salida configurado. De esta forma el usuario no necesita abandonar el editor ni recordar comandos de terminal

para obtener el sistema multiagente ejecutable.



Figura 6.42: Botón de generación en la barra superior del editor, activo sobre un fichero .mad.

6.3.5. Diagrama del grafo

Durante el diseño del paquete `extension` surgió la pregunta de si convenía ofrecer un editor visual al estilo *drag-and-drop* que permitiese construir el sistema multiagente arrastrando nodos sobre un lienzo. Tras analizarlo se descartó esta vía: implementar un editor gráfico bidireccional —que sincronizase modificaciones en el lienzo con el texto del .mad y viceversa— habría supuesto un coste desproporcionado dado el alcance del trabajo, y habría desviado el foco de los objetivos centrales del lenguaje y el generador.

En su lugar se optó por una solución más ligera pero suficiente: mostrar al usuario un diagrama del grafo general del sistema que está modelando, de manera que pueda comprobar visualmente si la estructura resultante se corresponde con la que tenía en mente. La extensión recorre el AST del fichero activo, identifica las estructuras de comunicación, los agentes y las transiciones entre ellos, y produce un fragmento Mermaid que describe el grafo correspondiente. Este fragmento se inserta en un fichero Markdown auxiliar que se actualiza cada vez que el usuario guarda el modelo y se muestra en un panel WebView lateral. La apertura del panel se activa mediante un botón en la barra superior del editor, análogo al de generación de código.



Figura 6.43: Botón para abrir el panel de diagrama del grafo en la barra superior del editor.

La implementación inicial del visor utilizaba un *WebView* propio con la librería Mermaid empaquetada dentro de la extensión. Este enfoque resultó inestable: el SVG se generaba con sus dimensiones reales pero el *WebView* no llegaba a pintarse en pantalla. Ante este problema, se optó por delegar el renderizado en el *preview* de Markdown nativo de VSCode, escribiendo el diagrama como un bloque “`mermaid`” en un fichero temporal. El renderizado lo aporta la extensión `bierner.markdown-mermaid`, declarada como `extensionDependency` en el `package.json`, por lo que VSCode la instala automáticamente junto con la extensión principal.

```
// Herramientas del ayudante de pago
pythonTool requestRefund modulePath "Order"
pythonTool applyDiscount modulePath "Order"

// Herramientas del ayudante de producto
pythonTool requestReplacement modulePath "Order"
pythonTool requestReturn modulePath "Order"

// Herramientas del ayudante general
pythonTool getDeliveryEstimate modulePath "Order"

// Agentes
agent Validator {
  provider openai
  model "gpt-5-nano"
  profile ValidatorPrompt
  description "Agente encargado de validar el mensaje del cliente."
  stateUpdate isValid
  tools orderChecker
}

agent infoExtractor {
  provider openai
  model "gpt-5-nano"
  profile infoExtractorPrompt
  description "Agente encargado de rellenar el estado con la información del pedido"
  stateUpdate orderId, issueDescription, preferredSolution, orderStatus, productID,
  productBatch, trackingNumber, paymentAmount, paymentStatus
  tools getInfoOrder
}

agent answerWriterNode {
  provider openai
  model "gpt-5-nano"
  profile answerWriterPrompt
  description "Agente encargado de responder al usuario"
  stateContext isValid, decision
}

agent shippingHelper {
  provider openai
  model "gpt-5-nano"
  profile shippingHelperPrompt
  description "Especialista en problemas de envíos."
  stateContext orderId, issueDescription, preferredSolution, orderStatus, productID,
  productBatch, trackingNumber, paymentAmount, paymentStatus
  stateUpdate decision
  tools reopenShipmentCase, requestDuplicateShipment
}
```

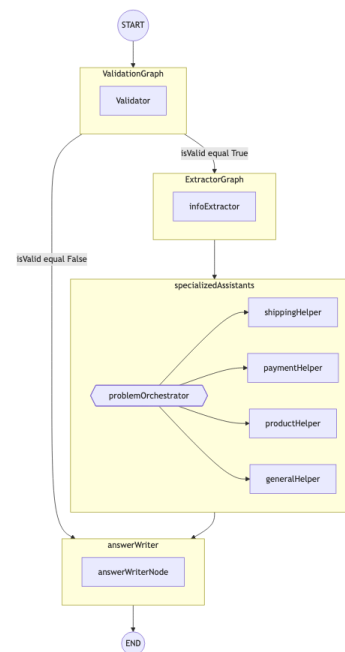


Figura 6.44: Panel lateral de VSCode con el diagrama Mermaid generado a partir de un modelo de ejemplo.

Capítulo 7

Pruebas

Contenidos

7.1. Pruebas del validador	124
7.2. Pruebas del generador	129
7.2.1. Selección de modelos: pairwise testing	129
7.2.2. Validación manual y corrección	130
7.3. Pruebas de la extensión	134
7.4. Validación con casos de uso	137
7.4.1. Respondedor automático de incidencias SRE	137
7.4.2. Asesor legal automatizado	140

En este capítulo se presentan las pruebas realizadas sobre el proyecto, con el fin de establecer un control de calidad sistemático que, además, sirva de apoyo al desarrollador en futuras iteraciones de cualquiera de los paquetes.

7.1. Pruebas del validador

El paquete `language` cumple dos funciones principales: definir el lenguaje mediante la gramática (sintaxis) y comprobar qué modelos son semánticamente válidos mediante el



validador (restricciones). La gramática en sí misma no admite pruebas significativas, ya que no encierra lógica que verificar más allá de aceptar o rechazar la sintaxis. Las restricciones del validador, en cambio, sí justifican una batería de pruebas que permite:

- Garantizar que el validador rechaza los modelos malformados.
- Detectar regresiones cuando se modifica la gramática o el propio validador.
- Garantizar que los modelos válidos no producen falsos positivos.
- Documentar el comportamiento esperado del validador a través de los modelos que lo ejercitan.

Para llevar a cabo esta fase de pruebas se utilizó el framework Vitest, ya que es el *runner* recomendado por Langium y por su compatibilidad nativa con el sistema de módulos empleado en el proyecto, sin requerir configuración adicional.

Las pruebas se organizan dentro del directorio `packages/language/test/`, donde se encuentran:

- `test/models/`: contiene los modelos necesarios para las pruebas.
- `test/restrictions.test.ts`: el conjunto de pruebas (*test suite*).

La pieza central es la función auxiliar `testRestriccion()`, que para cada restricción genera automáticamente dos pruebas: una sobre un modelo que debe producir el código de error R0X y otra sobre un modelo que no debe contenerlo. De este modo se evita repetir las mismas dos aserciones en cada una de las quince restricciones.



```
function testRestriccion(  
  codigo: string,  
  nombre: string,  
  modeloInvalido: string,  
  modeloValido: string = MODELO_BASE,  
) {  
  describe(`${codigo} - ${nombre}`, () => {  
    test("CASO INVALIDO", async () => {  
      expect(await validar(modeloInvalido)).toContain(codigo);  
    });  
    test("CASO VALIDO", async () => {  
      expect(await validar(modeloValido)).not.toContain(codigo);  
    });  
  });  
}
```

Figura 7.1: Función auxiliar `testRestriccion()` en `restrictions.test.ts`.

El caso inválido comprueba que la validación del modelo contiene el código de error correspondiente, mientras que el caso válido comprueba que no lo contiene. Cuando no se proporciona un modelo válido específico, se emplea `MODELO_BASE`, un modelo correcto que actúa como control negativo común a todas las restricciones.

Por tanto, para cada restricción se prueba un caso válido y un caso inválido. Además, algunas restricciones incluyen pruebas adicionales según su naturaleza, como la comprobación de los valores límite (R08) o la verificación de cada uno de los tipos y atributos afectados (R06 y R10). En total, las quince restricciones dan lugar a veintidós grupos de prueba y, por tanto, a cuarenta y cuatro casos individuales. El código `R0X` identifica cada restricción de forma uniforme en el validador, en el requisito y en el test, lo que permite trazar cada requisito hasta su prueba concreta. A continuación se muestra la matriz de trazabilidad con los casos de prueba definidos para cada restricción:



RF	Restr.	Qué se verifica	Casos
RF-L11	R01	Toda estructura de comunicación participa en al menos una transición	Válido e inválido
RF-L12	R02	Las URL de los servidores MCP no se repiten	Válido e inválido
RF-L13	R03	Existe exactamente un punto de inicio START	Válido e inválido
RF-L14	R04	No hay arcos con condición duplicados entre el mismo par origen-destino	Válido e inválido
RF-L15	R05	La transición única desde un origen no lleva condición	Válido e inválido
RF-L16	R06	El tipo del valor en una condición es compatible con el atributo referenciado	3 tipos: INT, BOOL, STRING
RF-L17	R07	No hay más de una transición sin condición desde un mismo origen	Válido e inválido
RF-L18	R08	La temperatura de los actores está entre 0.0 y 1.0	2 límites: inferior y superior
RF-L19	R09	Solo existe un nodo Summarizer en el sistema	Válido e inválido
RF-L20	R10	Los valores numéricos opcionales son estrictamente positivos	5 atributos
RF-L21	R11	Las estructuras layered no contienen ciclos	Válido e inválido
RF-L22	R12	Las apiKeyName de los servidores MCP son únicas	Válido e inválido
RF-L23	R13	Todos los elementos del sistema tienen nombres únicos	Válido e inválido



RF	Restr.	Qué se verifica	Casos
RF-L24	R14	El nodo Summarizer no tiene perfil asignado	Válido e inválido
RF-L25	R15	Cada agente pertenece a una única estructura de comunicación	Válido e inválido

Tabla 7.1: Matriz de trazabilidad de las pruebas del validador. Cada fila se implementa con el helper `testRestriccion()` en `restrictions.test.ts`.

La ejecución de la batería completa confirma que las cuarenta y cuatro pruebas pasan satisfactoriamente, como se muestra en la Figura 7.2. De este modo queda verificado el comportamiento de las quince restricciones del validador, tanto frente a modelos malformados como frente a modelos correctos.

```

RUN v3.1.4 /Users/pmoroeno/Documents/TFG/maasDSL/multiAgentDSL/packages/language
✓ test/restrictions.test.ts (44 tests) 36ms
✓ R01 - Estructuras de comunicación unidas > CASO INVALIDO 18ms
✓ R01 - Estructuras de comunicación unidas > CASO VALIDO 1ms
✓ R02 - Servidores MCP sin repetir > CASO INVALIDO 1ms
✓ R02 - Servidores MCP sin repetir > CASO VALIDO 1ms
✓ R03 - Punto de inicio único > CASO INVALIDO 1ms
✓ R03 - Punto de inicio único > CASO VALIDO 0ms
✓ R04 - Sin arcos duplicados > CASO INVALIDO 1ms
✓ R04 - Sin arcos duplicados > CASO VALIDO 1ms
✓ R05 - Las restricciones únicas no pueden tener restricción > CASO INVALIDO 1ms
✓ R05 - Las restricciones únicas no pueden tener restricción > CASO VALIDO 1ms
✓ R06 - Compatibilidad de tipos en la consición de transición (INT) > CASO INVALIDO 1ms
✓ R06 - Compatibilidad de tipos en la consición de transición (INT) > CASO VALIDO 1ms
✓ R06 - Compatibilidad de tipos en la consición de transición (BOOL) > CASO INVALIDO 1ms
✓ R06 - Compatibilidad de tipos en la consición de transición (BOOL) > CASO VALIDO 0ms
✓ R06 - Compatibilidad de tipos en la consición de transición (STRING) > CASO INVALIDO 0ms
✓ R06 - Compatibilidad de tipos en la consición de transición (STRING) > CASO VALIDO 0ms
✓ R07 - Un solo else por transición > CASO INVALIDO 0ms
✓ R07 - Un solo else por transición > CASO VALIDO 0ms
✓ R08 - Temperatura de los agentes (Limite Inferior) > CASO INVALIDO 0ms
✓ R08 - Temperatura de los agentes (Limite Inferior) > CASO VALIDO 0ms
✓ R08 - Temperatura de los agentes (Limite Inferior) > CASO INVALIDO 0ms
✓ R08 - Temperatura de los agentes (Limite Inferior) > CASO VALIDO 0ms
✓ R09 - Un solo nodo summarizer > CASO INVALIDO 0ms
✓ R09 - Un solo nodo summarizer > CASO VALIDO 0ms
✓ R10 - Valor positivo de maxMessages > CASO INVALIDO 0ms
✓ R10 - Valor positivo de maxMessages > CASO VALIDO 0ms
✓ R10 - Valor positivo de tokenTrigger > CASO INVALIDO 0ms
✓ R10 - Valor positivo de tokenTrigger > CASO VALIDO 0ms
✓ R10 - Valor positivo de maxToken > CASO INVALIDO 0ms
✓ R10 - Valor positivo de maxToken > CASO VALIDO 0ms
✓ R10 - Valor positivo de timeout > CASO INVALIDO 0ms
✓ R10 - Valor positivo de timeout > CASO VALIDO 0ms
✓ R10 - Valor positivo de maxRetries > CASO INVALIDO 0ms
✓ R10 - Valor positivo de maxRetries > CASO VALIDO 0ms
✓ R11 - Layered sin ciclos > CASO INVALIDO 0ms
✓ R11 - Layered sin ciclos > CASO VALIDO 0ms
✓ R12 - MCP con distintas API KEY > CASO INVALIDO 0ms
✓ R12 - MCP con distintas API KEY > CASO VALIDO 0ms
✓ R13 - Nombres únicos en el sistema > CASO INVALIDO 0ms
✓ R13 - Nombres únicos en el sistema > CASO VALIDO 0ms
✓ R14 - Sumarizer no puede tener perfil > CASO INVALIDO 0ms
✓ R14 - Sumarizer no puede tener perfil > CASO VALIDO 0ms
✓ R15 - Un nodo no puede pertenece a varias estructuras de comunicación > CASO INVALIDO 0ms
✓ R15 - Un nodo no puede pertenece a varias estructuras de comunicación > CASO VALIDO 0ms

Test Files 1 passed (1)
Tests 44 passed (44)
Start at 13:55:59
Duration 433ms (transform 66ms, setup 0ms, collect 202ms, tests 36ms, environment 0ms, prepare 43ms)

```

Figura 7.2: Resultado de la ejecución de las pruebas del validador con Vitest.



7.2. Pruebas del generador

Las pruebas del generador se basan en *snapshot testing*: se definen una serie de modelos `.mad` que cubren distintas combinaciones de funcionalidades, se genera el proyecto Python completo y se compara fichero a fichero contra una copia guardada previamente. Si la salida cambia respecto al *snapshot*, el test falla. Esto garantiza que ningún cambio en el generador altera silenciosamente el código producido.

7.2.1. Selección de modelos: pairwise testing

Lo primero fue determinar las dimensiones que influyen de forma independiente en el código generado:

- **Base de datos:** PostgreSQL, MongoDB, InMemory (3 valores).
- **Proveedor LLM:** OpenAI, Anthropic, Google, Ollama (4 valores).
- **Herramientas:** ninguna, MCP, Python (3 valores).
- **Estrategia de mensajes:** ninguna, recorte, resumen, mixta (4 valores).

Probar todas las combinaciones supondría $3 \times 4 \times 3 \times 4 = 144$ modelos. Para reducir ese número sin sacrificar cobertura significativa se aplicó la técnica de *pairwise testing*, que garantiza que cada par de valores de dos dimensiones distintas aparece al menos una vez. El resultado fue una matriz de 17 combinaciones, una reducción del 88 % respecto al producto cartesiano completo.



ID	Base de datos	Proveedor	Herramientas	Mensajes
MD01	PostgreSQL	Anthropic	Ninguna	Recorte
MD02	InMemory	Anthropic	MCP	Mixta
MD03	PostgreSQL	Google	Python	Resumen
MD04	MongoDB	OpenAI	MCP	Recorte
MD05	MongoDB	Ollama	Python	Mixta
MD06	InMemory	Ollama	Ninguna	Recorte
MD07	MongoDB	Anthropic	MCP	Resumen
MD08	InMemory	Anthropic	Python	Ninguna
MD09	MongoDB	OpenAI	Ninguna	Ninguna
MD10	PostgreSQL	Google	MCP	Ninguna
MD11	PostgreSQL	Ollama	MCP	Resumen
MD12	PostgreSQL	OpenAI	Ninguna	Mixta
MD13	InMemory	Google	Ninguna	Mixta
MD14	MongoDB	Google	Python	Recorte
MD15	PostgreSQL	Ollama	Python	Ninguna
MD16	InMemory	OpenAI	Python	Resumen
MD17	PostgreSQL	Google	Ninguna	Resumen

Tabla 7.2: Matriz de combinaciones pairwise para las pruebas del generador.

7.2.2. Validación manual y corrección

La validación funcional del código generado se realizó ejecutándolo contra LLM reales con API keys y bases de datos en Docker. Sin embargo, automatizar ese proceso en una batería de tests que se ejecute en cada cambio no es viable: implicaría coste económico por llamada, latencia elevada, no-determinismo en las respuestas y dependencia de red permanente. Antes de optar por la validación manual se consideraron alternativas intermedias:

- **Mocking de las llamadas al LLM:** habría eliminado las dependencias externas,



pero también habría enmascarado fallos reales de integración, como que los modelos de Anthropic rechazan conversaciones con más de un `SystemMessage`.

- **Compilación sintáctica** (`python -m py_compile`): barato y capaz de detectar errores de importación, pero con cobertura funcional muy baja.
- **Pipeline CI! (CI!) con contenedores**: levantar bases de datos en Docker, instalar dependencias y ejecutar el grafo con un mensaje de ejemplo habría proporcionado cobertura alta, pero el coste de configuración y mantenimiento era desproporcionado para el alcance del proyecto.

Por ello se optó por una validación manual puntual: los 17 modelos se ejecutaron contra LLM reales con bases de datos en Docker y API keys reales. Este proceso reveló cinco fallos en el generador:

- El proveedor `google_genai` no se reconocía en el generador de dependencias.
- El `Summarizer` guardaba el resumen como mensaje, provocando dos `SystemMessage` consecutivos (rechazado por Anthropic).
- Los agentes descentralizados sin salida estructurada no volcaban la respuesta al estado.
- Anthropic rechazaba conversaciones que terminaban en `AIMessage`.
- Los agentes centralizados no se identificaban en los mensajes al coordinador.

Todos los fallos se corrigieron antes de fijar los *snapshots*. Una vez aprobado el código de forma manual, se congelaron los *snapshots* con `toMatchSnapshot()`. A partir de ese momento, cualquier cambio en el generador que altere la salida rompe el test, actuando como detector de regresiones. Los tests dinámicos realizados durante la validación no se automatizaron: fueron un proceso puntual cuyo valor consistió en descubrir los fallos, corregirlos y aprobar la referencia; una vez cerrado ese ciclo, su función quedó transferida a los *snapshots*.



Cabe destacar que el *snapshot testing* verifica implícitamente el requisito RNF03: si el generador no fuera determinista, el mismo modelo produciría salidas distintas en cada ejecución y los *snapshots* fallarían de forma aleatoria. El hecho de que la batería sea estable confirma que el generador es determinista sin necesidad de una prueba adicional.

RF	Qué se verifica	Cobertura
RF-G01	Generación de <code>prompts.py</code> con los perfiles de los agentes	Snapshot: verifica generación del artefacto
RF-G02	Generación de <code>state.py</code> con el <code>TypedDict</code> del estado	Snapshot: verifica generación del artefacto
RF-G03	Generación de <code>agents.py</code> con los LLM y salida estructurada	Snapshot: verifica generación del artefacto
RF-G04	Generación de <code>graph.py</code> con nodos, aristas y enrutamiento	Snapshot: verifica generación del artefacto
RF-G05	<code>.env.template</code> según proveedores y herramientas usados	Dimensión Proveedor (4 valores) $\times$ Herramientas (3 valores)
RF-G06	<code>requirements.txt</code> con dependencias según el modelo	Snapshot: verifica generación del artefacto
RF-G07	<code>langgraph.json</code> para despliegue con <code>langgraph dev</code>	Snapshot: verifica generación del artefacto
RF-G08	<code>checkpointer.py</code> según la estrategia de persistencia declarada	Dimensión BD: PostgreSQL, MongoDB, InMemory
RF-G09	Código correcto para estructura <code>layered</code>	Casos de uso (Sección 7.4)
RF-G10	Código correcto para estructura <code>centralized</code>	Casos de uso (Sección 7.4)



RF	Qué se verifica	Cobertura
RF-G11	Código correcto para estructura <code>decentralized</code>	Casos de uso (Sección 7.4)
RF-G12	Múltiples estructuras combinadas como subgrafos	Casos de uso (Sección 7.4)
RF-G13	<code>mcpClients.py</code> por cada <code>mcpServer</code> declarado	Herramientas=MCP: MD02, MD04, MD07, MD10, MD11
RF-G14	CLI invocable con <code>multi-agent-dsl-cli generate</code>	Ejecución manual de los 17 modelos
RNF02	Código ejecutable con <code>langgraph dev</code> sin modificaciones	Validación manual de los 17 modelos
RNF03	Generador determinista: mismo modelo produce mismo código	Estabilidad de los snapshots
RNF04	Tests automatizados en <code>language</code> y <code>cli</code>	Batería del validador + batería del generador

Tabla 7.3: Matriz de trazabilidad de las pruebas del generador.

La ejecución de la batería de snapshots confirma que los 17 modelos producen la salida esperada, como se muestra en la siguiente figura.



```

RUN v3.1.4 /Users/pmoreno/Documents/TFG/maasDSL/multiAgentDSL/packages/cli
✓ test/generating/snapshots.test.ts (17 tests) 73ms
✓ Generator snapshots > MD01 24ms
✓ Generator snapshots > MD02 5ms
✓ Generator snapshots > MD03 3ms
✓ Generator snapshots > MD04 4ms
✓ Generator snapshots > MD05 3ms
✓ Generator snapshots > MD06 3ms
✓ Generator snapshots > MD07 3ms
✓ Generator snapshots > MD08 3ms
✓ Generator snapshots > MD09 2ms
✓ Generator snapshots > MD10 3ms
✓ Generator snapshots > MD11 4ms
✓ Generator snapshots > MD12 2ms
✓ Generator snapshots > MD13 2ms
✓ Generator snapshots > MD14 3ms
✓ Generator snapshots > MD15 3ms
✓ Generator snapshots > MD16 3ms
✓ Generator snapshots > MD17 3ms

Test Files 1 passed (1)
Tests 17 passed (17)
Start at 13:55:59
Duration 380ms (transform 69ms, setup 0ms, collect 148ms, tests 73ms, environment 0ms, prepare 36ms)

```

Figura 7.3: Resultado de la ejecución de las pruebas del generador con Vitest.

7.3. Pruebas de la extensión

Las pruebas de la extensión de VSCode se realizaron de forma manual. El framework oficial para automatizar este tipo de pruebas es `@vscode/test-electron`, que levanta una instancia real de VSCode en modo *headless*, carga la extensión sobre un workspace de prueba y permite disparar comandos y peticiones LSP programáticamente para verificar su comportamiento. Sin embargo, este enfoque tiene un coste de implementación significativo: cada test debe gestionar el tiempo de arranque del *language server*, trabajar con la API asíncrona de VSCode y mantener sincronía con los cambios de versión del editor. Dado que las funcionalidades de la extensión (resaltado, hover, snippets, botones de generación y diagrama) son fácilmente verificables en una sesión de edición real, se consideró que el esfuerzo de automatización no estaba justificado para el alcance del proyecto y se optó por la verificación manual.

Para estructurar la verificación manual se definió la siguiente lista de comprobación, ejecutada sobre una instancia real de VSCode con la extensión cargada:

- ✓ **V01** — Abrir un fichero `.mad` y comprobar que el Language Server se activa: aparecen sugerencias de autocompletado y los errores se subrayan en tiempo real.
- ✓ **V02** — Verificar que las palabras clave, atributos y valores del DSL reciben colores



distintos según su categoría semántica.

- ✓ **V03** — Comprobar que el autocompletado filtra las sugerencias por tipo en cada posición (por ejemplo, solo muestra modelos válidos para el proveedor declarado).
- ✓ **V04** — Situar el cursor sobre palabras clave y nodos del modelo y verificar que aparece la documentación contextual del hover provider.
- ✓ **V05** — Escribir un prefijo de snippet y verificar que la plantilla se expande correctamente.
- ✓ **V06** — Introducir un modelo semánticamente inválido y comprobar que los errores y advertencias del validador se subrayan en tiempo real, identificando el elemento afectado.
- ✓ **V07** — Pulsar el botón de generación sobre un modelo válido y verificar que el proyecto Python se produce en el directorio de salida.
- ✓ **V08** — Pulsar el botón de generación sobre un modelo con errores de validación y verificar que el proceso aborta y notifica al usuario.
- ✓ **V09** — Pulsar el botón de diagrama y verificar que se abre el panel lateral con el diagrama Mermaid del modelo activo.
- ✓ **V10** — Con el panel de diagrama abierto, guardar el fichero `.mad` y comprobar que el diagrama se actualiza automáticamente.
- ✓ **V11** — Usar una combinación de proveedor y estructura conocida como problemática y verificar que la extensión muestra el aviso correspondiente.

RF	Qué se verifica	Verificación
RF-E01	Language Server activo al abrir un fichero <code>.mad</code>	V01



RF	Qué se verifica	Verificación
RF-E02	Resaltado de sintaxis mediante gramática TextMate	V02
RF-E03	Autocompletado filtrado por tipo en cada posición	V03
RF-E04	Documentación contextual (hover) sobre palabras clave y nodos	V04
RF-E05	Snippets para las construcciones más habituales	V05
RF-E06	Errores y advertencias del validador subrayados en tiempo real	V06
RF-E07	Comando de generación accesible desde la barra del editor	V07
RF-E08	El comando de generación aborta si el modelo contiene errores	V08
RF-E09	Comando para visualizar el diagrama Mermaid en panel lateral	V09
RF-E10	El diagrama se actualiza automáticamente al guardar el fichero	V10
RF-E11	Aviso al usar combinaciones de proveedor y estructura problemáticas	V11
RNF01	El Language Server valida el modelo sin demoras perceptibles	V01, V06
RNF08	Los errores identifican el elemento afectado y describen la causa	V06

Tabla 7.4: Matriz de trazabilidad de las pruebas de la extensión.



7.4. Validación con casos de uso

Esta sección valida la expresividad del DSL mediante casos de uso reales ejecutados de extremo a extremo. El objetivo no es repetir las pruebas del generador —ya cubiertas en la sección anterior— sino demostrar qué tipo de sistemas es posible describir y generar con la herramienta, y qué relación existe entre la compacidad del modelo y el código producido.

7.4.1. Respondedor automático de incidencias SRE

El primer caso de uso modela un sistema de respuesta automática a incidencias de infraestructura software. El escenario de partida es habitual en equipos de operaciones: una alerta generada por un sistema de monitorización como Prometheus o Datadog llega al grafo con la descripción del problema, y el sistema decide de forma autónoma si puede diagnosticarlo y resolverlo o si debe escalar a un ingeniero humano de guardia.

Objetivo del sistema

Reducir el tiempo de respuesta ante incidencias de producción automatizando las dos primeras fases de cualquier proceso de respuesta: el triaje inicial y el diagnóstico técnico. Cuando la causa raíz es identificable con suficiente confianza, el sistema ejecuta también la remediación (rollback, reinicio del servicio, escalado de instancias) y notifica el resultado al canal de guardia. Cuando la incidencia supera un umbral de criticidad, escala inmediatamente a un operador humano mediante PagerDuty sin intentar actuar de forma autónoma.

Arquitectura del sistema

El grafo combina tres estructuras de comunicación. Una estructura `layered` inicial (`InitialTriage`) aloja al agente de triaje, cuya salida condiciona el flujo: si la severidad es crítica o el servicio afectado es de negocio crítico, el grafo se desvía a una segunda estructura `layered` (`EmergencyEscalation`) que escala al *oncall* humano.



En caso contrario, continúa hacia una estructura **decentralized** (**ParallelDiagnosis**) compuesta por tres analistas especializados que se pasan el control entre sí hasta completar el diagnóstico; una vez los tres han concluido, una estructura **layered** final (**AutomaticResolution**) sintetiza la causa raíz y ejecuta el plan de remediación. La persistencia del estado se configura en MongoDB, lo que permite retomar la conversación si el grafo se interrumpe y llevar un histórico de incidencias entre sesiones.

Agente	Responsabilidad	Herramientas
Triager	Analiza la alerta entrante y determina la severidad (<i>critical</i> , <i>high</i> , <i>medium</i> , <i>low</i>) y si requiere escalado humano inmediato.	—
LogAnalyst	Recupera y analiza los logs recientes del servicio afectado, identificando patrones de error y el primer momento de aparición.	<code>getRecentLogs</code>
MetricsAnalyst	Examina las métricas de la última hora (CPU, memoria, latencia p50/p95/p99, <i>error rate</i>) y detecta anomalías.	<code>getServiceMetrics</code>
DeployAnalyst	Revisa los despliegues recientes e identifica cambios con correlación temporal con el inicio de la incidencia.	<code>getRecentDeployments</code>
RootCause Synthesizer	Consolida los hallazgos de los tres analistas, determina la causa raíz con nivel de confianza explícito y propone un plan de remediación priorizado por riesgo.	—
Remediator	Ejecuta el plan de remediación. No actúa si la confianza es baja o el riesgo es alto. Notifica el resultado al canal de guardia.	<code>rollbackDeployment</code> , <code>restartService</code> , <code>scaleService</code> , <code>notifyChannel</code>
Escalator	En incidencias críticas, crea un incidente en PagerDuty y notifica al canal de guardia para que un ingeniero tome el control.	<code>createPagerDuty Incident</code> , <code>notifyChannel</code>

Tabla 7.5: Agentes del sistema respondedor de incidencias y sus herramientas.

Este caso es especialmente relevante para la validación del DSL porque combina cuatro estructuras de comunicación del metamodelo en un único modelo —**layered**, **decentralized** y dos **layered** adicionales— lo que ejercita los requisitos RF-G09,

RF-G11 y RF-G12 que la matriz *pairwise* no cubría de forma conjunta.

Ejecución y evidencia

El sistema se ejecutó contra LLMs reales usando herramientas mockeadas que simulan las respuestas de las APIs de infraestructura. La Figura 7.4 muestra la traza de LangSmith durante una ejecución en la que el agente **Remediator** invoca la herramienta `rollbackDeployment` tras recibir el plan de remediación sintetizado por **RootCauseSynthesizer**. El panel izquierdo de LangSmith muestra el grafo de ejecución generado por el DSL; el panel central, los pasos de razonamiento del agente y la llamada a la herramienta.

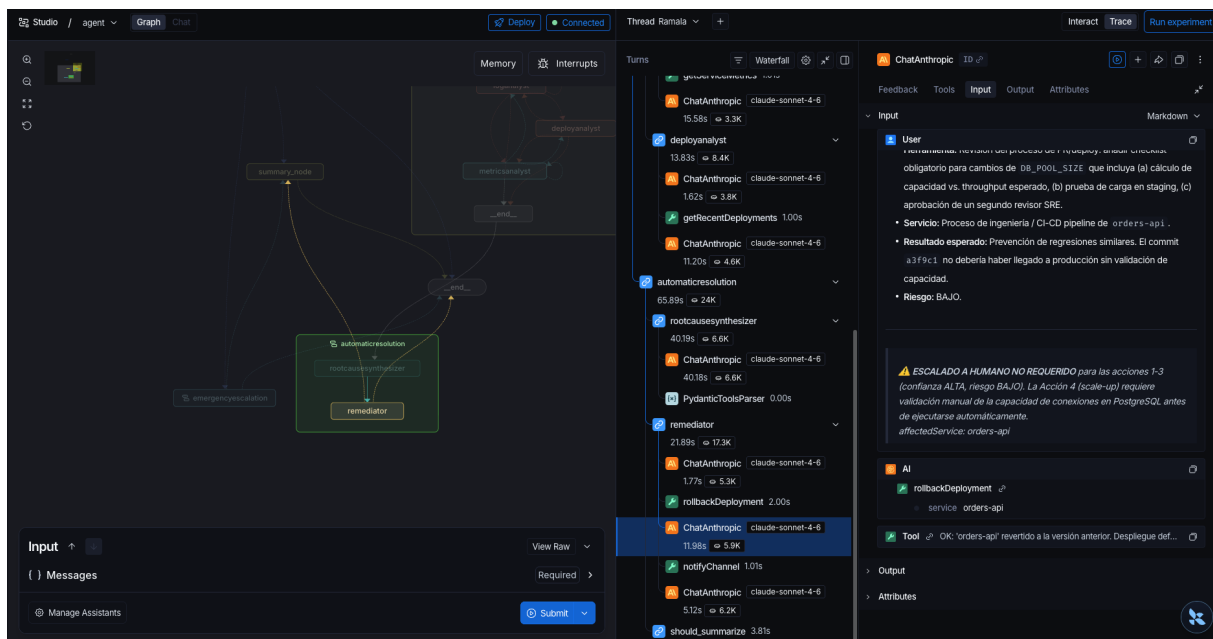


Figura 7.4: Traza de LangSmith durante la ejecución del respondedor de incidencias. El agente **Remediator** invoca `rollbackDeployment` como parte del plan de remediación.

Métricas de generación

Para cuantificar la ganancia del enfoque MDD, se midió la relación entre el tamaño del modelo `.mad` y el código Python generado mediante el script `scripts/count_lines.py` incluido en el repositorio, excluyendo en ambos lados el contenido de los *prompts* —que hay que escribir independientemente de si se usa el DSL o se programa directamente— y



las 110 líneas del fichero `tools/ops.py`, que contiene los mocks de herramientas escritos a mano y no forman parte de la salida del generador.

Artefacto	Líneas
Modelo <code>.mad</code> (líneas estructurales, sin <i>prompts</i>)	163
Código Python generado (sin <i>prompts</i> ni mocks manuales)	587
Ratio de generación	3,6x

Tabla 7.6: Relación entre el tamaño del modelo DSL y el código Python generado, excluyendo prompts de ambos lados.

Un ratio de 3,6x significa que por cada línea de modelo el generador produce 3,6 líneas de código Python funcional: estado tipado, nodos de agente con integración de herramientas, lógica de grafos, configuración de persistencia, infraestructura Docker y API REST. Más allá del volumen, el código generado sigue los principios de separación de responsabilidades descritos en el Capítulo 6, lo que lo hace directamente legible y modificable sin regenerar el proyecto completo.

El modelo `.mad` junto al código generado completo se puede consultar en:

[https://github.com/Pmoreno447/MAS-DSL/tree/main/examples/03-casosDeUso/in
cidentResponder](https://github.com/Pmoreno447/MAS-DSL/tree/main/examples/03-casosDeUso/in
cidentResponder)

7.4.2. Asesor legal automatizado

El segundo caso de uso modela un asistente jurídico para un despacho de abogados. El escenario es el de un cliente que expone una consulta legal en lenguaje natural, y el sistema debe clasificarla, investigarla, analizarla con un especialista del área correspondiente y devolver un asesoramiento comprensible, decidiendo de forma autónoma qué casos puede tratar y cuáles deben derivarse a un abogado humano.

Objetivo del sistema

Automatizar el primer nivel de atención jurídica de un despacho: clasificar la consulta por área del derecho, fundamentarla en normativa y jurisprudencia, y producir un análisis



especializado con plazos y pasos concretos. El sistema actúa como filtro y asistente del letrado, no como sustituto: ante casos graves (penales con detenidos, menores en riesgo o plazos inminentes) escala directamente a un abogado humano sin emitir asesoramiento autónomo.

Arquitectura del sistema

El grafo combina tres estructuras de comunicación. Una estructura **layered** inicial (**Triage**) clasifica el caso e investiga la normativa aplicable; su salida condiciona el flujo mediante una transición booleana: si el caso requiere intervención humana, la ejecución salta a la estructura **layered** final (**FinalWriting**), que redacta un aviso de derivación; en caso contrario, pasa a una estructura **centralized** (**SpecialistConsultation**) en la que un coordinador delega en el especialista del área correspondiente.

La especialización de los abogados no es meramente nominal: cada especialista dispone de su propia herramienta de búsqueda documental sobre el corpus normativo de su rama del derecho, de modo que su conocimiento se diferencia de forma estructural y no solo a través del *prompt*. En esta demostración dichas herramientas están simuladas, pero en un despliegue real corresponderían a un sistema de recuperación aumentada (RAG) sobre la legislación y jurisprudencia indexada de cada área. El sistema incorpora además un **ConversationSummarizer** que comprime el historial cuando supera un umbral de tokens, y persiste el estado en PostgreSQL, lo que permite retomar consultas largas entre sesiones.

Agente	Responsabilidad	Herramientas
Classifier	Analiza la consulta y determina el área legal (<i>laboral, civil, penal, fiscal</i>), la urgencia y si requiere derivación obligatoria a un abogado humano.	—
LegalResearcher	Busca normativa y jurisprudencia española relevante en fuentes oficiales (BOE, Tribunal Supremo) para fundamentar el análisis.	Tavily (MCP)
LaborLawyer	Analiza el caso desde el Derecho Laboral y la Seguridad Social.	<code>searchLaborLaw</code> , <code>checkLegalDeadlines</code>
CivilLawyer	Analiza el caso desde el Derecho Civil (contratos, familia, sucesiones, arrendamientos).	<code>searchCivilLaw</code> , <code>checkLegalDeadlines</code>
CriminalLawyer	Analiza el caso desde el Derecho Penal, realizando una calificación jurídica provisional.	<code>searchCriminalLaw</code> , <code>checkLegalDeadlines</code>
TaxLawyer	Analiza el caso desde el Derecho Fiscal y Tributario (IRPF, IVA, sanciones, recursos).	<code>searchTaxLaw</code> , <code>checkLegalDeadlines</code>
Specialist Orchestrator	Coordinador que delega el caso en el especialista correspondiente según el área clasificada.	—
Writer	Transforma el análisis técnico en un asesoramiento claro y estructurado para el cliente (situación jurídica, plazos y próximos pasos).	—

Tabla 7.7: Agentes del sistema asesor legal y sus herramientas.

Ejecución y evidencia

El sistema se ejecutó con herramientas mockeadas que simulan las respuestas del corpus normativo. La Figura 7.5 muestra la traza de LangSmith durante una ejecución en la que uno de los abogados especialistas invoca su herramienta de búsqueda documental para fundamentar el análisis antes de remitirlo al redactor final.

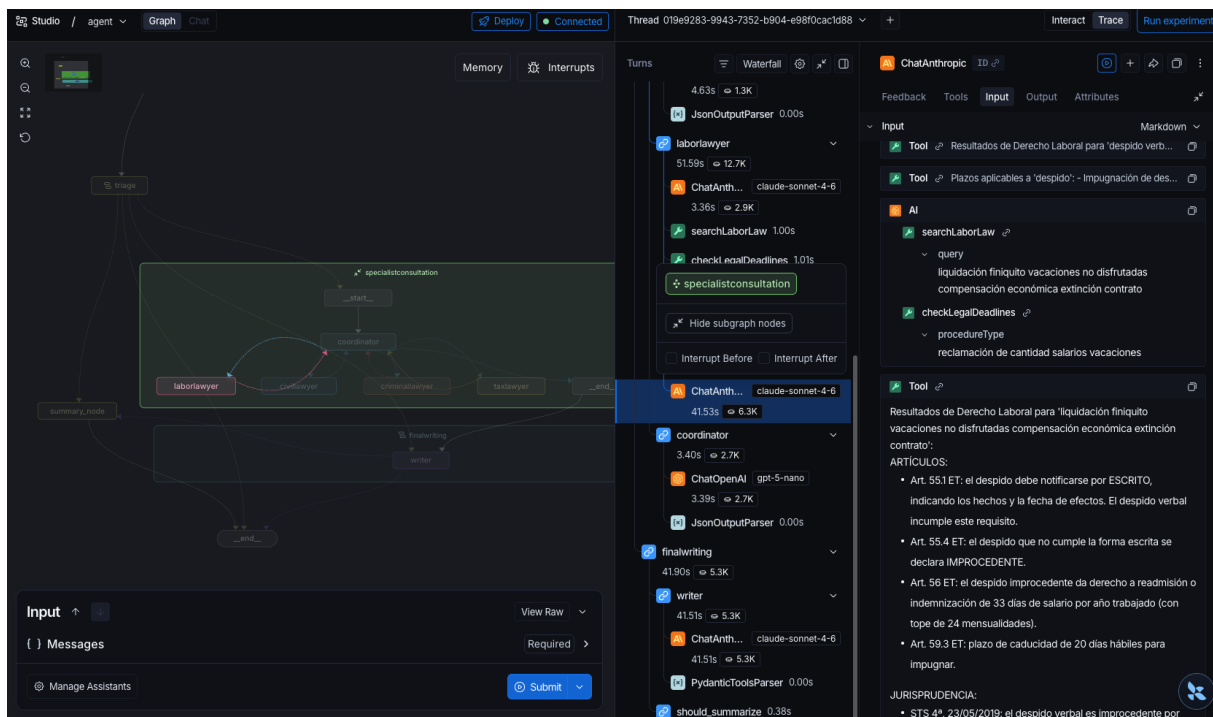


Figura 7.5: Traza de LangSmith durante la ejecución del asesor legal. Un abogado especialista invoca su herramienta de búsqueda documental.

Métricas de generación

Artefacto	Líneas
Modelo .mad (líneas estructurales, sin <i>prompts</i>)	170
Código Python generado (sin <i>prompts</i> ni mocks manuales)	672
Ratio de generación	3,9x

Tabla 7.8: Relación entre el modelo DSL y el código Python generado en el asesor legal, obtenida con `scripts/count_lines.py`.

El ratio de 3,9x es consistente con el del caso anterior (3,6x), lo que sugiere que la relación entre modelo y código generado es estable independientemente de la complejidad del sistema. Ambos casos confirman que el DSL permite definir sistemas multiagente reales en menos de 200 líneas estructurales, delegando en el generador la producción de toda la infraestructura de ejecución.

El modelo .mad junto al código generado completo se puede consultar en:



<https://github.com/Pmoreno447/MAS-DSL/tree/main/examples/03-casosDeUso/legadAdvisor>

Capítulo 8

Conclusiones y trabajos futuros

Contenidos

8.1. Conclusiones	145
8.2. Trabajos futuros	148

8.1. Conclusiones

El código fuente completo del proyecto, incluyendo el DSL, el generador, la extensión de VSCode y los casos de uso, está disponible en:

<https://github.com/Pmoreno447/MAS-DSL>

Este TFG ha dado lugar a un sistema funcional compuesto por tres piezas complementarias: un DSL para la descripción de sistemas multi-agente basados en LLMs, un generador que traduce esos modelos a código Python ejecutable sobre LangGraph, y una extensión de VSCode que integra ambas capacidades en el entorno de desarrollo. Los requisitos funcionales definidos al inicio del proyecto quedan cubiertos en su totalidad, dentro del alcance deliberadamente acotado que se estableció desde el comienzo.



Objetivos cumplidos

Los ocho objetivos definidos en la Sección 1.3 quedan cubiertos en su totalidad. A continuación se recoge de forma resumida cómo se ha dado respuesta a cada uno de ellos.

1. **Diseñar el metamodelo del DSL.** El metamodelo se diseñó mediante un diagrama UML que evolucionó a lo largo de tres iteraciones, incorporando los conceptos de agente, perfil, herramienta, servidor MCP, contexto y las cuatro estructuras de comunicación soportadas. El resultado final se recoge en la Subsección 5.1.2.
2. **Implementar la gramática y la semántica del DSL en Langium.** La gramática Langium se implementó a partir del metamodelo, añadiendo quince restricciones semánticas en el validador para capturar invariantes no expresables en la gramática libre de contexto.
3. **Desarrollar un editor asistido.** La extensión de VSCode proporciona resaltado de sintaxis, autocompletado filtrado por proveedor, mensajes de error contextualizados, *hover* informativo, *snippets* predefinidos y autocompletado de herramientas MCP a partir de ficheros de configuración locales.
4. **Implementar un generador de código.** El generador transforma modelos *.mad* en código Python ejecutable sobre LangGraph: estado, agentes, *prompts*, herramientas, grafos por estructura y grafo principal.
5. **Garantizar la mantenibilidad del sistema generado.** El código generado sigue los principios de separación de responsabilidades y bajo acoplamiento: cada concepto del modelo se traduce a un módulo independiente y los *prompts* están desacoplados de la lógica de los agentes, lo que facilita su modificación sin regenerar el conjunto completo.
6. **Preparar el código generado para el despliegue.** El generador produce un *Dockerfile*, un fichero *docker-compose.yml*, un *requirements.txt* y una API



REST lista para servir el grafo. El sistema puede contenerizarse y desplegarse sin ninguna modificación adicional.

7. **Verificar la corrección de la herramienta mediante pruebas.** Se desarrollaron 44 tests unitarios sobre el validador —dos por cada una de las quince restricciones— y tests de *snapshot* sobre los diecisiete modelos de la matriz *pairwise*. Las matrices de trazabilidad del Capítulo 7 vinculan cada requisito funcional con su prueba correspondiente.
8. **Validar el DSL mediante casos de uso reales.** Los modelos de la matriz *pairwise* se ejecutaron manualmente contra LLMs reales con claves de API y bases de datos activas, lo que permitió detectar y corregir cinco errores antes de congelar los *snapshots* de referencia.

Valor de la herramienta

El sistema resultante responde a una necesidad concreta: permitir a cualquier persona definir un sistema multi-agente sencillo con rapidez, sin necesidad de aprender LangGraph en profundidad. LangGraph es un *framework* potente pero con una curva de aprendizaje notable; el DSL abstrae esa complejidad y ofrece un vocabulario orientado al dominio, más próximo a cómo se razona sobre un sistema de agentes que a cómo se implementa. Cabe señalar que esta herramienta no pretende competir con la generación de código asistida por IA. Un modelo de lenguaje puede producir código LangGraph a partir de una descripción en lenguaje natural, pero carece de una visión conceptual global del sistema: las limitaciones de contexto hacen que sea más adecuado para generar fragmentos que para diseñar un sistema completo con soporte de producción, al menos por ahora. El DSL, en cambio, parte de un modelo estructurado y completo, lo que garantiza la coherencia del código generado independientemente de la complejidad del sistema. Además, la generación se ejecuta localmente: el código resultante no pasa por servidores externos y pertenece íntegramente al usuario. Por último, el propio archivo `.mad` actúa como documentación viva del sistema, algo que una sesión de generación con IA no proporciona de forma



estructurada.

Reflexión personal

Este trabajo ha resultado especialmente enriquecedor por la combinación de disciplinas que pone en juego. La ingeniería de modelos y los sistemas agenticos son, a priori, ámbitos alejados entre sí; unirlos en un mismo proyecto ha exigido comprender en profundidad tanto el diseño de lenguajes con Langium como el modelo de ejecución de LangGraph, dos tecnologías punteras cuyo aprendizaje considero de gran valor de cara al futuro profesional. En conjunto, el sistema obtenido supera las expectativas iniciales y constituye una base sólida sobre la que continuar trabajando en las líneas descritas en la sección siguiente.

8.2. Trabajos futuros

El desarrollo de este TFG ha permitido delimitar con claridad las capacidades actuales del sistema y, al mismo tiempo, identificar extensiones que ampliarían de forma natural su utilidad. A continuación se describen las líneas de trabajo futuro más relevantes.

Tipos de atributo enriquecidos: literales y listas

El sistema de tipos del estado se limita actualmente a tres tipos escalares (`int`, `string`, `boolean`). Una extensión natural sería permitir que un atributo de tipo cadena declare sus valores posibles como un conjunto de literales, convirtiéndolo en un tipo enumerado. Esto facilitaría la escritura de condiciones de transición —el DSL podría advertir en tiempo de edición cuando el valor de una condición no pertenece al conjunto declarado— y haría el modelo más autodocumentado. En la misma línea, dar soporte a listas permitiría acumular mensajes o resultados intermedios directamente en el estado sin recurrir a campos de texto libre, lo que abriría la puerta a patrones como la agregación de respuestas de múltiples agentes.



Ejecución en paralelo (*map-reduce*)

El paradigma *map-reduce* es un patrón recurrente en sistemas multi-agente: un nodo coordinador despacha una misma tarea a n instancias de agente que trabajan en paralelo (fase *map*) y, una vez completadas, sus resultados se fusionan en el estado antes de continuar (fase *reduce*). LangGraph soporta este patrón de forma nativa mediante la API **Send**, que permite despachar mensajes a nodos de forma dinámica en tiempo de ejecución. Añadir esta capacidad al DSL requeriría introducir una nueva construcción para declarar el número de instancias —o vincularlo a una lista del estado—, un nodo reductor que reciba todas las salidas parciales y una función que las consolide en el estado compartido. La ganancia práctica es significativa: casos de uso como el análisis paralelo de documentos, la búsqueda simultánea en múltiples fuentes o la generación y evaluación concurrente de alternativas pasarían a ser expresables directamente en el modelo sin código adicional.

Estado por subgrafo

LangGraph permite descomponer un grafo complejo en subgrafos, cada uno con su propio estado interno, comunicándose con el grafo padre mediante un estado compartido parcial. Esta capacidad es relevante en modelos con múltiples estructuras de comunicación: actualmente, todos los agentes del modelo comparten un único estado global definido en el bloque **context**. Permitir que cada estructura declare atributos de estado propios —invisibles para el resto del modelo— reduciría el acoplamiento entre estructuras, evitaría colisiones de nombres en modelos complejos y se alinearía con el modelo de ejecución de LangGraph. El principal reto de diseño es definir qué atributos se propagan al estado padre y cuáles permanecen locales al subgrafo.

Generación de código para `SharedMessagePool`

La estructura de comunicación `SharedMessagePool` está definida en la gramática del DSL y validada por el compilador, pero el generador de código no la traduce aún: cualquier modelo que la utilice es sintácticamente válido pero no produce código ejecutable. Esta



estructura representa el paradigma de pizarra (*blackboard*): los agentes publican sus resultados en un pool común que cualquier otro agente puede leer, desacoplando emisores y receptores. Implementar su generación requeriría definir cómo se representa ese pool compartido en el estado de LangGraph y qué orden de ejecución se asigna a los agentes cuando no existen dependencias explícitas entre ellos.

Autenticación extendida en servidores MCP

Cuando se modela un servidor MCP en el DSL, el generador produce un cliente `MultiServerMCPClient` que resuelve las herramientas declaradas y las pone a disposición de los agentes, exponiendo únicamente las solicitadas para reducir el consumo de tokens y fallando de forma temprana si alguna no existe en el servidor. La autenticación se resuelve sustituyendo en tiempo de ejecución el marcador `{key}` presente en la URL por el valor de la variable de entorno correspondiente, de modo que la credencial nunca queda codificada en el código generado.

La limitación de este enfoque es que solo cubre los servidores que aceptan la credencial como parámetro dentro de la URL —como Tavily—, pero no las dos formas de autenticación más extendidas en servidores corporativos: las cabeceras HTTP (por ejemplo `Authorization: Bearer`, que usan servicios como Context7 o GitHub) y los flujos OAuth. El soporte de cabeceras sería una extensión sencilla, ya que el cliente generado las admite de forma nativa y bastaría con añadir un nuevo campo opcional al bloque `mcpServer` del DSL; el soporte de OAuth queda fuera del alcance inmediato al requerir un flujo de autorización interactivo incompatible con la ejecución desatendida del grafo.

Human in the Loop

LangGraph ofrece soporte nativo para pausar la ejecución del grafo en puntos concretos y transferir el control a un operador humano mediante *breakpoints* (`interrupt_before` / `interrupt_after`). Integrar este mecanismo en el DSL permitiría declarar en el modelo qué nodos requieren aprobación humana antes de continuar —por ejemplo, antes de



ejecutar una herramienta con efectos externos o de enviar una respuesta al usuario— y el generador produciría automáticamente la configuración de interrupción correspondiente en LangGraph. El principal reto de diseño es decidir cómo expresar en el modelo la reanudación tras la interrupción y qué información se expone al operador durante la pausa.

Editor gráfico de modelos

El flujo de trabajo actual exige que el usuario describa el sistema escribiendo texto en el DSL. Una alternativa complementaria sería ofrecer un editor gráfico en el que los agentes, las estructuras de comunicación y las transiciones se representen como nodos y aristas en un diagrama interactivo. El usuario arrastraría elementos del vocabulario del DSL sobre un lienzo, configuraría sus propiedades mediante formularios y el editor generaría automáticamente el archivo `.mad` correspondiente. Este enfoque reduciría la barrera de entrada para perfiles menos técnicos y haría más inmediata la comprensión topológica del sistema. Langium ofrece soporte experimental para este tipo de integraciones mediante *Langium Sprotty*, una biblioteca que conecta el LSP con el *framework* de diagramas Sprotty y permite construir editores gráficos que comparten el mismo modelo semántico que el editor textual.

Segmentación de módulos generados por estructura

Cuando un modelo combina varias estructuras de comunicación, el generador vuelca todos los agentes en un único `agents.py` y todos los *prompts* en un único `prompts.py`, independientemente de la estructura a la que pertenecen. Refactorizar el generador para que produzca un módulo por estructura (`agents_layered1.py`, `agents_centralized2.py`, etc.) no afectaría a la corrección del código generado, pero mejoraría notablemente su legibilidad, facilitaría la depuración de grafos complejos y alinearía la estructura de ficheros con la organización conceptual del modelo.



Opciones de generación selectiva en el CLI

El comando `generate` produce siempre el conjunto completo de ficheros. Añadir los flags `-only` y `-skip` permitiría regenerar únicamente los módulos que han cambiado —por ejemplo, solo los *prompts* tras ajustar los perfiles— sin sobrescribir el resto. Esto resulta especialmente útil cuando el usuario ha realizado modificaciones manuales en parte del código generado y no desea perderlas al regenerar.

Integración continua y pruebas automatizadas ampliadas

La suite de pruebas actual cubre el validador y el generador mediante tests unitarios y de *snapshot*. Tres áreas quedan fuera de la automatización: las pruebas de la extensión de VSCode —que requerirían `@vscode/test-electron` tal como se describe en la ??—, la verificación de que el código Python generado es ejecutable y la integración de todo ello en un *pipeline* de **CI!**. Para el código Python, una aproximación viable sería combinar análisis estático (`pyflakes` o `mypy`) con la ejecución de los *fixtures* pairwise contra modelos locales ligeros (Ollama) en lugar de LLMs de producción. Con un *pipeline* de **CI!** —por ejemplo, GitHub Actions— cualquier cambio en la gramática, el generador o la extensión quedaría validado automáticamente antes de integrarse en la rama principal.

Anexos

Anexo A

Manual de instalación y manual de usuario

Contenidos

A.1. Manual de instalación	154
A.2. Manual de usuario	155
A.3. Versiones de las herramientas utilizadas	162

A.1. Manual de instalación

Opción A — Extensión VSCode (recomendado)

Instala **multiAgentDSL** desde el Visual Studio Marketplace buscando `pmorenoc.vscode-multiagentdsl` en el panel de extensiones de VSCode o ejecutando en la terminal:

```
code --install-extension pmorenoc.vscode-multiagentdsl
```

La extensión incluye el generador, el validador en tiempo real, autocompletado, *hover* y el diagrama del grafo. No requiere clonar el repositorio ni instalar dependencias adicionales.



Opción B — Desde el código fuente

```
git clone https://github.com/Pmoreno447/MAS-DSL
cd MAS-DSL
npm install
npm run build
```

Abre la carpeta raíz del repositorio en VSCode —aquella desde la que son visibles las carpetas `docs`, `examples`, `packages`, etc.— y pulsa F5 para lanzar la extensión en modo desarrollo. Es importante que la carpeta abierta sea exactamente la raíz del proyecto; si se abre una subcarpeta, la tarea de compilación configurada en `.vscode/launch.json` no encontrará los artefactos necesarios y el modo de desarrollo no arrancará correctamente.

A.2. Manual de usuario

Estructura de un modelo

Un sistema se define entre llaves `{ }` respetando el siguiente orden de bloques:

```
{
    context      ...
    profile      ...    (0 o mas)
    pythonTool / mcpServer  ...    (0 o mas)
    agent / summarizer / coordinator  ...    (1 o mas)
    layered / centralized / decentralized  ...    (1 o mas)
    from         ...    (transiciones, 0 o mas)
}
```

context

Define el estado compartido del sistema y su mecanismo de persistencia.

```
context MiContexto {
```



```

attribute nombreAttr type string description "descripcion"
maxMessages 10 // opcional: max. mensajes retenidos
persistence inMemorySave
}

```

Tipos de atributo: int · string · boolean

Valor	Descripción
inMemorySave	En memoria; se pierde al terminar la ejecución
Postgre	PostgreSQL. Requiere la variable POSTGRES_URI
MongoDB	MongoDB. Requiere la variable MONGODB_URI

Tabla A.1: Opciones de persistencia del contexto.

profile

System prompt reutilizable por cualquier agente.

```

profile nombrePerfil description "Eres un asistente util y conciso."

```

agent

Unidad LLM básica. Es el actor más común del sistema.

```

agent MiAgente {
  provider openai
  model "gpt-4o"
  profile miPerfil // opcional
  temperature 0.7 // opcional: 0.0-1.0
  description "..." // opcional
  stateContext attr1, attr2 // atributos que lee del estado
  stateUpdate attr1 // atributos que escribe en el estado
  statusMessage "Pensando..." // opcional: mensaje de progreso
  maxToken 2000 // opcional
  timeOut 30 // opcional, segundos
  maxRetries 3 // opcional
}

```



```
tools herramienta1 // opcional  
}
```

coordinator

Orquesta a los agentes de una estructura **centralized**. Decide a quién delegar en cada turno.

```
coordinator MiCoordinador {  
  provider openai  
  model "gpt-4o"  
  profile miPerfil // opcional  
  temperature 0.0 // opcional  
}
```

summarizer

Resume la conversación cuando supera el umbral de tokens. Solo puede haber uno por sistema.

```
summarizer MiSummarizer {  
  provider anthropic  
  model "claude-sonnet-4-6"  
  profile miPerfil // opcional  
  tokenTrigger 3000 // opcional: umbral en tokens  
}
```

pythonTool

Herramienta implementada en Python local.

```
pythonTool miHerramienta modulePath "tools/mi_modulo"
```



mcpServer

Herramienta remota accesible vía protocolo MCP.

```
mcpServer miServidor {  
    url "https://mcp.ejemplo.com/mcp?key={key}"  
    transport "streamable_http"  
    apiKeyName "MI_API_KEY" // nombre de la var. de entorno  
    tools "tool1", "tool2"  
}
```

Cuando se declara `apiKeyName`, el generador sustituye el marcador `{key}` de la URL por el valor de la variable de entorno en tiempo de ejecución. La clave nunca queda *hardcodeada* en el código generado.

Estructuras de comunicación

layered — los agentes se ejecutan en cadena. El último `layer` no lleva `next`.

```
layered miCadena {  
    layer Agente1 next Agente2  
    layer Agente2 next Agente3  
    layer Agente3  
}
```

centralized — un coordinador decide qué agente actúa en cada turno.

```
centralized miEstructura {  
    coordinator MiCoordinador  
    agents Agente1, Agente2, Agente3  
}
```

decentralized — los agentes se comunican entre sí sin jerarquía. Mínimo dos agentes.

```
decentralized miEstructura {  
    agents Agente1, Agente2  
}
```



Transiciones (from)

Conectan estructuras de comunicación entre sí. El grafo necesita al menos un `from START` y un `to END`.

```
from START to estructural1
from estructural1 to estructura2 when miAtributo equal "listo"
from estructura2 to END
```

Operadores de condición: equal · greather · lower

Las condiciones solo pueden comparar atributos del `context` con literales del mismo tipo (True / False, número entero o cadena entre comillas).

Proveedores y modelos disponibles

Proveedor	Modelos
openai	gpt-4o, gpt-4o-mini, gpt-4.1, gpt-4.1-mini, gpt-4.1-nano, gpt-5, gpt-5-mini, gpt-5-nano, o1, o1-pro, o3, o3-pro, o3-mini, o4-mini
anthropic	claude-opus-4-7, claude-sonnet-4-6, claude-haiku-4-5, claude-opus-4-6, claude-sonnet-4-5, claude-opus-4-5, claude-opus-4-1
google_genai	gemini-2.5-pro, gemini-2.5-flash, gemini-2.5-flash-lite, gemini-3.1-pro-preview, gemini-3-flash-preview
ollama	llama3.3, llama3.2, llama3.1, qwen3, qwen2.5, deepseek-r1, mistral, gemma2, phi3

Tabla A.2: Proveedores y modelos soportados por el DSL.

ollama no requiere API key pero necesita el servidor en ejecución (ollama serve). El resto requieren la variable de entorno correspondiente: OPENAI_API_KEY,



ANTHROPIC_API_KEY o GOOGLE_API_KEY.

Snippets

Escribe el prefijo y pulsa Tab para insertar el esqueleto completo con los puntos de edición ya colocados.

Prefijo	Descripción
template	Sistema completo de ejemplo con contexto, perfiles, dos agentes y estructura <code>layered</code>
agent	Declaración de agente con selector de proveedor
mcpServer	MCP Server genérico (URL, transport y tools)
mcpServer	MCP Server preconfigurado para Tavily (búsqueda web)
transition	Transición condicional con selector de operador
layered	Estructura <code>layered</code> con dos capas
centralized	Estructura <code>centralized</code> con coordinador y dos agentes
decentralized	Estructura <code>decentralized</code> con dos agentes

Tabla A.3: Snippets disponibles en la extensión.

Ayudas en tiempo real

Hover — al pasar el cursor sobre una palabra clave (`agent`, `layered`, `openai...`) se muestra su descripción y esquema de sintaxis. Sobre un nodo concreto del modelo (un agente, un perfil, un atributo) se muestra un resumen con sus datos.

Autocompletado — al escribir el campo `model`, el editor sugiere los modelos disponibles para el `provider` declarado en ese nodo. Al referenciar agentes, perfiles, atributos o herramientas, las sugerencias se filtran por tipo.

Validación en tiempo real — los errores semánticos se marcan con subrayado rojo mientras se escribe, sin necesidad de lanzar el generador.

Comandos de la extensión

Los tres botones aparecen en la barra superior del editor siempre que haya un fichero .mad abierto.

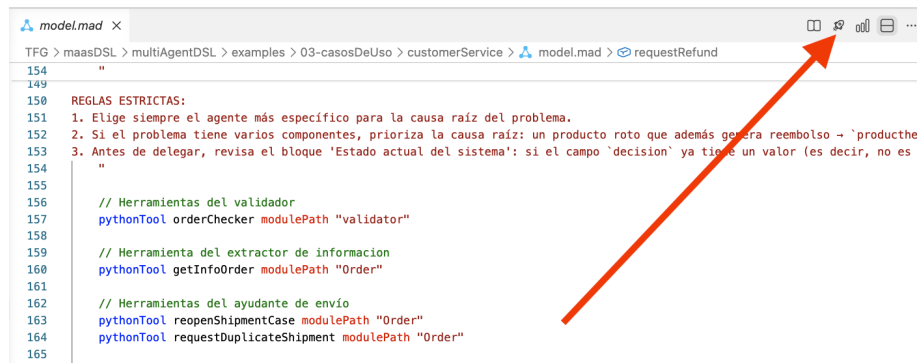


Figura A.1: Botón de generación de código (cohetes). Al pulsarlo se genera el proyecto Python en `generated/`.

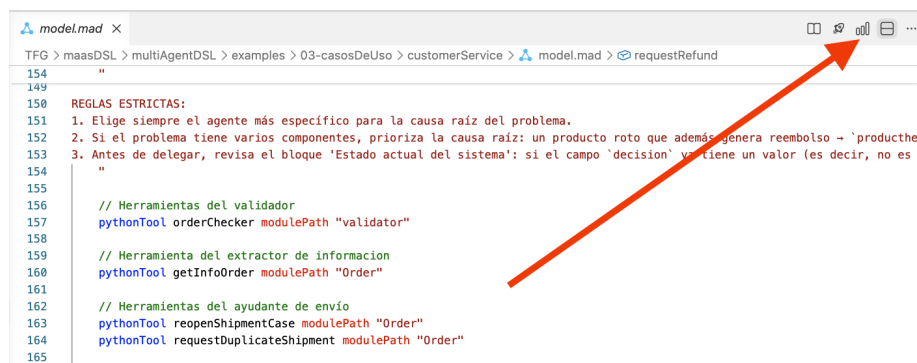


Figura A.2: Botón de diagrama (gráfico de barras). Abre el diagrama Mermaid del grafo generado a partir del modelo actual en un panel lateral.

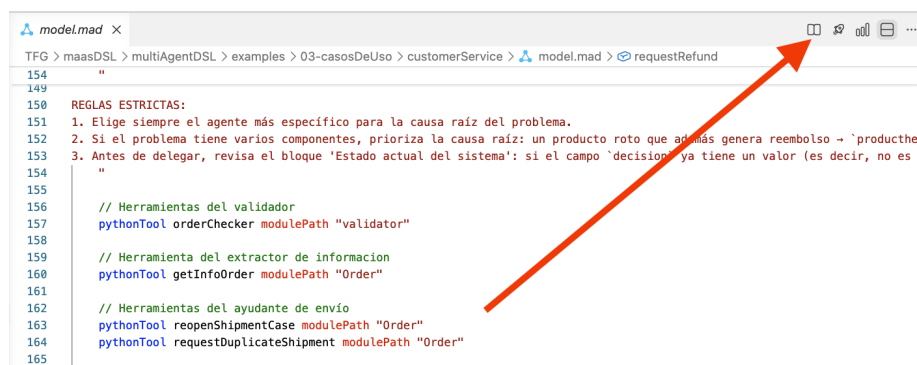


Figura A.3: Botón de manual (libro). Abre el manual de usuario en un panel lateral.



Ejecutar el código generado

El código se genera en una carpeta `generated/` junto al fichero `.mad`. Para ejecutarlo:

1. Crear el entorno virtual (solo la primera vez)

```
python3.12 -m venv venv
source venv/bin/activate          # macOS / Linux
venv\Scripts\activate            # Windows
```

2. Instalar dependencias

```
pip install -r requirements.txt
```

3. Configurar las API keys

```
cp .env.template .env
# editar .env con las claves necesarias
```

4a. Modo desarrollo (LangGraph Studio)

```
langgraph dev
```

Lanza un servidor local con interfaz gráfica para probar e inspeccionar el grafo paso a paso.

4b. Construir imagen Docker

```
langgraph build -t nombre-imagen
```

A.3. Versiones de las herramientas utilizadas

Herramienta	Versión
Node.js	24
Python	3.12

Tabla A.4: Versiones de las herramientas requeridas.

Bibliografía

- [1] Jose A. Barriga, Arturo Barriga, and Pedro J. Clemente. Model-driven development toward LLM-based multi-agent systems. In Loli Burgueño, editor, *Actas de las XXIX Jornadas de Ingeniería del Software y Bases de Datos (JISBD 2025)*. Sistedes, 2025. URL: <https://hdl.handle.net/11705/JISBD/2025/109>.
- [2] Xixun Lin, Yucheng Ning, Jingwen Zhang, Yan Dong, Yilong Liu, Yongxuan Wu, Xiaohua Qi, Nan Sun, Yanmin Shang, Kun Wang, Pengfei Cao, Qingyue Wang, Lixin Zou, Xu Chen, Chuan Zhou, Jia Wu, Peng Zhang, Qingsong Wen, Shirui Pan, Bin Wang, Yanan Cao, Kai Chen, Songlin Hu, and Li Guo. LLM-based agents suffer from hallucinations: A survey of taxonomy, methods, and directions. *arXiv preprint arXiv:2509.18970*, 2025. URL: <https://arxiv.org/abs/2509.18970>.
- [3] Yangxiao Cai, Peng Liang, et al. Methods and techniques of agentic software engineering: A systematic literature review. *IEEE Access*, 2026. URL: <https://ieeexplore.ieee.org/abstract/document/11343819>, doi:10.1109/ACCESS.2025.11343819.
- [4] Mailyn Moreno Espino, Alejandro Rosete Suárez, and Alfredo J. Simón Cuevas. Ingeniería de software orientada a agentes: Roles y metodologías. *Ingeniería Industrial*, 27(2-3):25–28, 2006. URL: https://www.researchgate.net/publication/49594853_INGENIERIA_DE_SOFTWARE_ORIENTADA_A_AGENTES_ROLES_Y_METODOLOGIAS.
- [5] Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems*

- (*ICMAS-95*), pages 312–319. AAAI Press, 1995. URL: <https://cdn.aaai.org/ICMAS/1995/ICMAS95-042.pdf>.
- [6] Franco Zambonelli, Nicholas R. Jennings, and Michael Wooldridge. Developing multiagent systems: The Gaia methodology. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 12(3):317–370, 2003. URL: <http://www.agentgroup.unimore.it/Zambonelli/PDF/MSEASchapter.pdf>.
- [7] Jaelson Castro, Manuel Kolp, and John Mylopoulos. Tropos: An agent-oriented software development methodology. *Autonomous Agents and Multi-Agent Systems*, 8(3):203–236, 2002. URL: <https://link.springer.com/article/10.1023/B:AGNT.0000018806.20944.ef>.
- [8] Scott A. DeLoach, Mark F. Wood, and Clint H. Sparkman. Multiagent systems engineering. *International Journal of Software Engineering and Knowledge Engineering*, 11(03):231–258, 2001. URL: <https://apps.dtic.mil/sti/pdfs/ADA449860.pdf>.
- [9] Yangxiao Cai, Ruiyin Li, Peng Liang, Mojtaba Shahin, and Zengyang Li. Designing LLM-based multi-agent systems for software engineering tasks: Quality attributes, design patterns and rationale. *arXiv preprint arXiv:2511.08475*, 2025. URL: <https://arxiv.org/abs/2511.08475>.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [11] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in*

- Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. URL: <https://arxiv.org/abs/2203.02155>.
- [12] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. URL: <https://arxiv.org/abs/2206.07682>.
- [13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. 35, 2022. URL: <https://arxiv.org/abs/2201.11903>.
- [14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. 2023. URL: <https://arxiv.org/abs/2210.03629>.
- [15] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model-based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. URL: <https://arxiv.org/abs/2308.11432>, [doi:10.1007/s11704-024-40231-1](https://doi.org/10.1007/s11704-024-40231-1).
- [16] Anthropic. Model context protocol, 2024. Accedido: 2025-01-10. URL: <https://www.anthropic.com/news/model-context-protocol>.
- [17] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995. [doi:10.1017/S0269888900008122](https://doi.org/10.1017/S0269888900008122).
- [18] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024. URL: <https://arxiv.org/abs/2402.01680>.

- [19] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. URL: <https://arxiv.org/abs/2308.00352>.
- [20] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL: <https://arxiv.org/abs/2307.07924>.
- [21] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. URL: <https://arxiv.org/abs/2304.03442>, doi:10.1145/3586183.3606763.
- [22] LangChain Inc. LangChain: Building applications with LLMs through composability, 2022. Accedido: 2025-01-10. URL: <https://github.com/langchain-ai/langchain>.
- [23] LangChain Inc. LangGraph: Building stateful, multi-actor applications with LLMs, 2024. Accedido: 2025-01-10. URL: <https://github.com/langchain-ai/langgraph>.
- [24] João Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents, 2024. Accedido: 2025-01-10. URL: <https://github.com/crewAIInc/crewAI>.
- [25] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications

- via multi-agent conversation. In *First Conference on Language Modeling (COLM)*, 2024. URL: <https://arxiv.org/abs/2308.08155>.
- [26] Douglas C. Schmidt. Model-driven engineering. *IEEE Computer*, 39(2):25–31, 2006. doi:10.1109/MC.2006.58.
- [27] Martin Fowler. *Domain-Specific Languages*. Addison-Wesley Professional, 2010.
- [28] Dave Steinberg, Frank Budinsky, Marcelo Paternostro, and Ed Merks. *EMF: Eclipse Modeling Framework*. Addison-Wesley Professional, 2 edition, 2008.
- [29] Moritz Eysholdt and Heiko Behrens. Xtext: Implement your language faster than the quick and dirty way. In *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications (SPLASH/OOPSLA)*, pages 307–309, 2010. doi:10.1145/1869542.1869625.
- [30] Dimitrios S. Kolovos, Antonio García-Domínguez, Louis M. Rose, and Richard F. Paige. Eugenia: Towards disciplined and automated development of GMF-based graphical model editors. *Software & Systems Modeling*, 16(1):229–255, 2017. doi:10.1007/s10270-015-0455-3.
- [31] TypeFox. Langium: Language engineering tool in the Node.js ecosystem, 2024. Accedido: 2025-01-10. URL: <https://langium.org>.
- [32] Markus Voelter. Language and IDE development, modularization and composition with MPS. In *Generative and Transformational Techniques in Software Engineering IV (GTTSE 2011)*, volume 7680 of *Lecture Notes in Computer Science*, pages 383–430. Springer, 2013. doi:10.1007/978-3-642-35992-7_9.
- [33] IBM Research. Agent Communication Protocol (ACP). <https://agentcommunicationprotocol.dev/>, 2025. Accessed: 2025.